Chapter 12 ▪ Advanced Swing and Graphics

In this chapter, we continue our discussion of the Swing user interface toolkit and AWT graphics from Volume I. We focus on techniques that are applicable to both client-side user interfaces and server-side generation of graphics and images. Swing has sophisticated components for rendering tables and trees. With the 2D graphics API, you can produce vector art of arbitrary complexity. The ImageIO API lets you manipulate raster images. Finally, you can use the printing API to generate printouts and PostScript files.

12.1. Tables

The `JTable` component displays a two-dimensional grid of objects. Tables are common in user interfaces, and the Swing team has put a lot of effort into the table control. Tables are inherently complex, but—perhaps more successfully than other Swing classes—the `JTable` component hides much of that complexity. You can produce fully functional tables with rich behavior by writing a few lines of code. You can also write more code and customize the display and behavior for your specific applications.

In the following sections, I'll explain how to make simple tables, how the user interacts with them, and how to make some of the most common adjustments. As with the other complex Swing controls, it is impossible to cover all aspects in complete detail. For more information, look in *Graphic Java™, Third Edition*, by David M. Geary (Prentice Hall, 1999), or *Core Swing* by Kim Topley (Prentice Hall, 1999).

12.1.1. A Simple Table

A `JTable` does not store its own data but obtains them from a *table model*. The `JTable` class has a constructor that wraps a two-dimensional array of objects into a default model. That is the strategy that we use in our first example; later in this chapter, we will turn to table models.

Figure 12.1 shows a typical table, describing the properties of the planets of the solar system. (A planet is *gaseous* if it consists mostly of hydrogen and helium. You should take the "Color" entries with a grain of salt—that column was added because it will be useful in later code examples.)

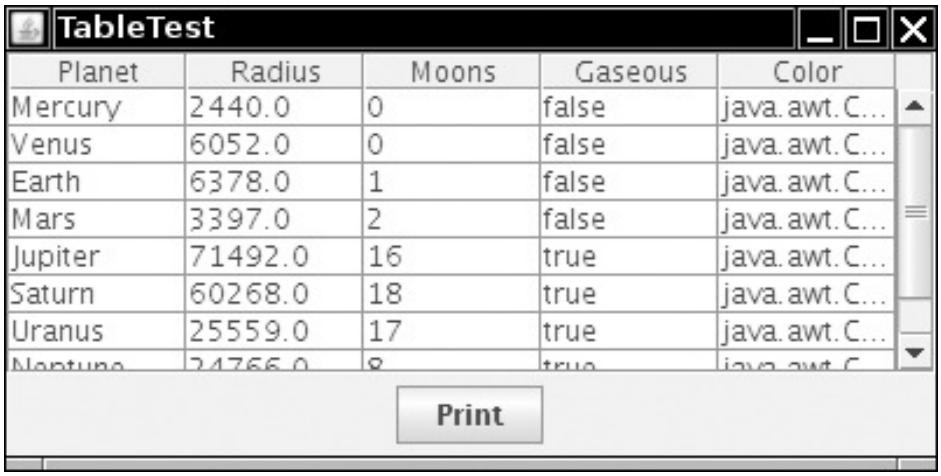


Figure 12.1: A simple table

As you can see from the code in Listing 12.1, the data of the table are stored as a two-dimensional array of `Object` values:

```
Object[][] cells = {
    { "Mercury", 2440.0, 0, false, Color.YELLOW },
    { "Venus", 6052.0, 0, false, Color.YELLOW },
    . . .
};
```



Note: Here, we take advantage of autoboxing. The entries in the second, third, and fourth columns are automatically converted into objects of type `Double`, `Integer`, and `Boolean`.

The table simply invokes the `toString` method on each object to display it. That's why the colors show up as `java.awt.Color[r=. . .,g=. . .,b=. . .]`.

Supply the column names in a separate array of strings:

```
String[] columnNames = { "Planet", "Radius", "Moons", "Gaseous", "Color" };
```

Then, construct a table from the cell and column name arrays:

```
var table = new JTable(cells, columnNames);
```

You can add scroll bars in the usual way—by wrapping the table in a `JScrollPane`:

```
var pane = new JScrollPane(table);
```

When you scroll the table, the table header doesn't scroll out of view.

Next, click on one of the column headers and drag it to the left or right. See how the entire column becomes detached (see Figure 12.2). You can drop it in a different location. This rearranges the columns *in the view only*. The data model is not affected.

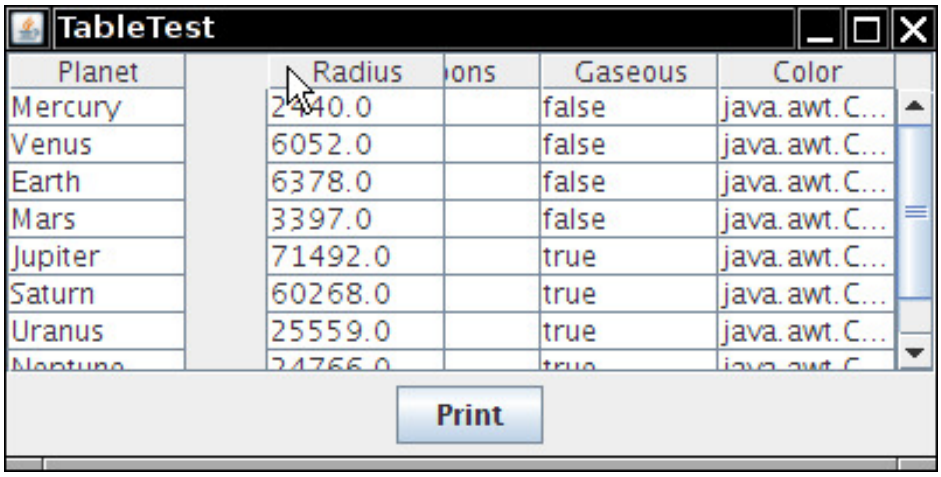


Figure 12.2: Moving a column

To *resize* columns, simply place the cursor between two columns until the cursor shape changes to an arrow. Then, drag the column boundary to the desired place (see Figure 12.3).

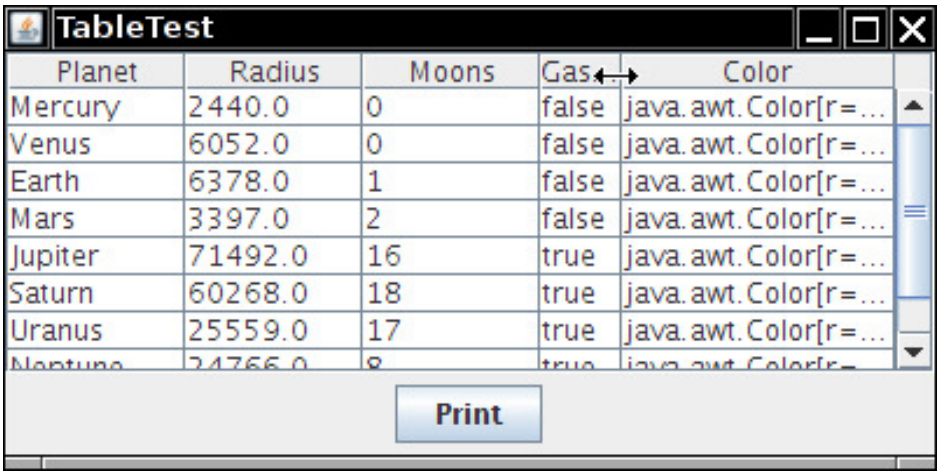


Figure 12.3: Resizing columns

Users can select rows by clicking anywhere in a row. The selected rows are highlighted; you will see later how to get selection events. Users can also edit the table entries by clicking on a cell and typing into it. However, in this code example, the edits do not change the underlying data. In your programs, you should either make cells uneditable or handle cell editing events and update your model. We will discuss those topics later in this section.

Finally, click on a column header. The rows are automatically sorted. Click again, and the sort order is reversed. This behavior is activated by the call

```
table.setAutoCreateRowSorter(true);
```

You can print a table with the call

```
table.print();
```



Caution: If you don't wrap a table into a scroll pane, you need to explicitly add the header:

```
add(table.getTableHeader(), BorderLayout.NORTH);
```

Listing 12.1 v2ch12/table/TableDemo.java

```
1  package v2ch12.table;
2
3  import module java.desktop;
4
5  /**
6   * This program demonstrates how to show a simple table.
7   */
8  class TableDemo {
9      void main() {
10         EventQueue.invokeLater(() -> {
11             var frame = new PlanetTableFrame();
12             frame.setTitle("TableTest");
13             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14             frame.setVisible(true);
15         });
16     }
17 }
```

```
18
19  /**
20   * This frame contains a table of planet data.
21   */
22  class PlanetTableFrame extends JFrame {
23      private String[] columnNames = { "Planet", "Radius", "Moons", "Gaseous", "Color" };
24      private Object[][] cells = { { "Mercury", 2440.0, 0, false, Color.YELLOW },
25              { "Venus", 6052.0, 0, false, Color.YELLOW },
26              { "Earth", 6378.0, 1, false, Color.BLUE }, { "Mars", 3397.0, 2, false, Color.RED },
27              { "Jupiter", 71492.0, 16, true, Color.ORANGE },
28              { "Saturn", 60268.0, 18, true, Color.ORANGE },
29              { "Uranus", 25559.0, 17, true, Color.BLUE },
30              { "Neptune", 24766.0, 8, true, Color.BLUE },
31              { "Pluto", 1137.0, 1, false, Color.BLACK } };
32
33      public PlanetTableFrame() {
34          var table = new JTable(cells, columnNames);
35          table.setAutoCreateRowSorter(true);
36          add(new JScrollPane(table), BorderLayout.CENTER);
37          var printButton = new JButton("Print");
38          printButton.addActionListener(_ -> {
39              try {
40                  table.print();
41              }
42              catch (SecurityException | PrinterException e) {
43                  e.printStackTrace();
44              }
45          });
46          var buttonPanel = new JPanel();
47          buttonPanel.add(printButton);
48          add(buttonPanel, BorderLayout.SOUTH);
49          pack();
50      }
51  }
```

`javax.swing.JTable` 1.2

- `JTable(Object[][] entries, Object[] columnNames)`
  constructs a table with a default table model.
- `void print()` 5.0
  displays a print dialog box and prints the table.
- `boolean getAutoCreateRowSorter()` 6
- `void setAutoCreateRowSorter(boolean newValue)` 6
  get or set the `autoCreateRowSorter` property. The default is `false`. When set, a default row sorter is automatically set whenever the model changes.
- `boolean getFillsViewportHeight()` 6
- `void setFillsViewportHeight(boolean newValue)` 6
  get or set the `fillsViewportHeight` property. The default is `false`. When set, the table always fills the enclosing viewport.

12.1.2. Table Models

In the preceding example, the table data were stored in a two-dimensional array. However, you should generally not use that strategy in your own code. Instead of dumping data into an array to display it as a table, consider im-

plementing your own table model.

Table models are particularly simple to implement because you can take advantage of the `AbstractTableModel` class that implements most of the required methods. You only need to supply three methods:

```
public int getRowCount();
public int getColumnCount();
public Object getValueAt(int row, int column);
```

There are many ways of implementing the `getValueAt` method. For example, if you want to display the contents of a `RowSet` that contains the result of a database query, simply provide this method:

```
public Object getValueAt(int r, int c) {
    try {
        rowSet.absolute(r + 1);
        return rowSet.getObject(c + 1);
    }
    catch (SQLException e) {
        e.printStackTrace();
        return null;
    }
}
```

Our sample program is even simpler. We construct a table that shows some computed values—namely, the growth of an investment under different interest rate scenarios (see **Figure 12.4**).

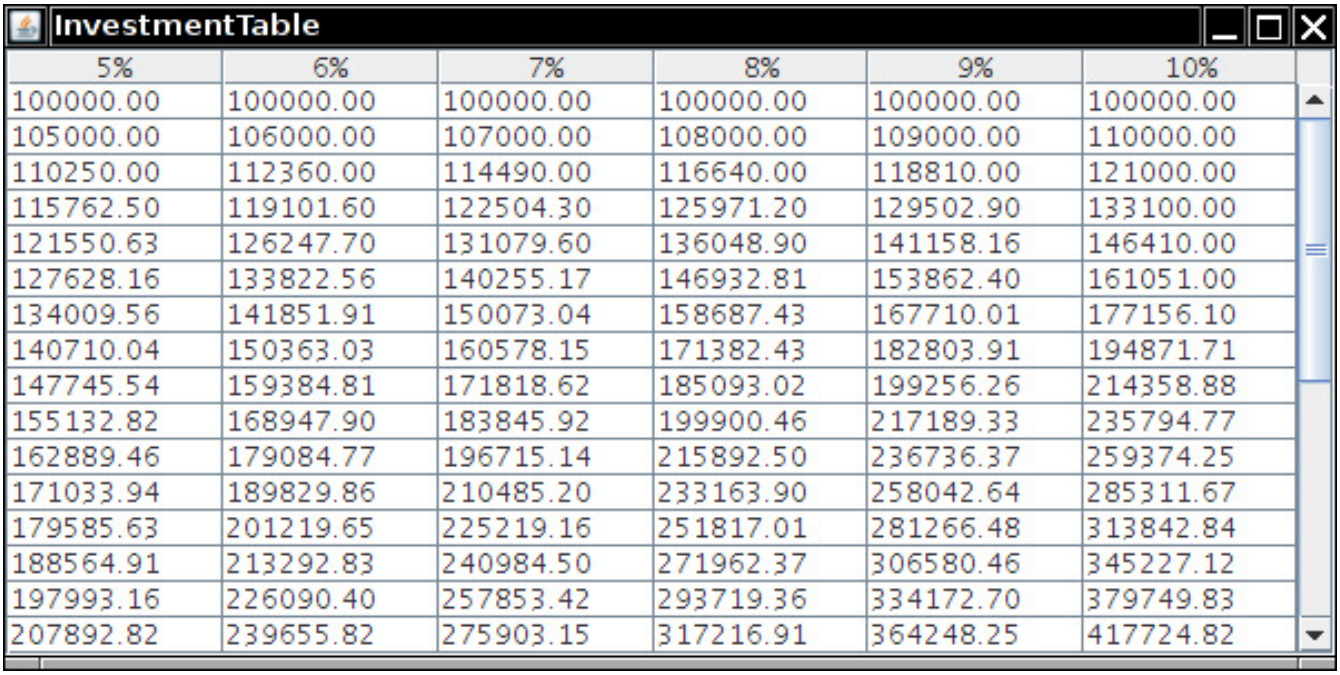


Figure 12.4: Growth of an investment

The `getValueAt` method computes the appropriate value and formats it:

```
public Object getValueAt(int r, int c) {
    double rate = (c + minRate) / 100.0;
    int nperiods = r;
```

```
    double futureBalance = INITIAL_BALANCE * Math.pow(1 + rate, nperiods);
    return "%.2f".formatted(futureBalance);
}
```

The `getRowCount` and `getColumnCount` methods simply return the number of rows and columns:

```
public int getRowCount() { return years; }
public int getColumnCount() {  return maxRate - minRate + 1; }
```

If you don't supply column names, the `getColumnName` method of the `AbstractTableModel` names the columns A, B, C, and so on. To change the default column names, override the `getColumnName` method. In this example, we simply label each column with the interest rate.

```
public String getColumnName(int c) { return (c + minRate) + "%"; }
```

You can find the complete source code in **Listing 12.2**.

Listing 12.2 v2ch12/tableModel/InvestmentTable.java

```
1  package v2ch12.tableModel;
2
3  import module java.desktop;
4
5  /**
6   * This program shows how to build a table from a table model.
7   */
8  class InvestmentTable {
9      void main() {
10         EventQueue.invokeLater(() -> {
11             var frame = new InvestmentTableFrame();
12             frame.setTitle("InvestmentTable");
13             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14             frame.setVisible(true);
15         });
16     }
17 }
18
19 /**
20  * This frame contains the investment table.
21  */
22 class InvestmentTableFrame extends JFrame {
23     public InvestmentTableFrame() {
24         var model = new InvestmentTableModel(30, 5, 10);
25         var table = new JTable(model);
26         add(new JScrollPane(table));
27         pack();
28     }
29 }
30
31 /**
32  * This table model computes the cell entries each time they are requested. The table contents
33  * shows the growth of an investment for a number of years under different interest rates.
34  */
```

```
35 class InvestmentTableModel extends AbstractTableModel {
36     private static double INITIAL_BALANCE = 100000.0;
37
38     private int years;
39     private int minRate;
40     private int maxRate;
41
42     /**
43      * Constructs an investment table model.
44      * @param y the number of years
45      * @param r1 the lowest interest rate to tabulate
46      * @param r2 the highest interest rate to tabulate
47      */
48     public InvestmentTableModel(int y, int r1, int r2) {
49         years = y;
50         minRate = r1;
51         maxRate = r2;
52     }
53
54     public int getRowCount() {
55         return years;
56     }
57
58     public int getColumnCount() {
59         return maxRate - minRate + 1;
60     }
61
62     public Object getValueAt(int r, int c) {
63         double rate = (c + minRate) / 100.0;
64         int nperiods = r;
65         double futureBalance = INITIAL_BALANCE * Math.pow(1 + rate, nperiods);
66         return "%.2f".formatted(futureBalance);
67     }
68
69     public String getColumnName(int c) {
70         return (c + minRate) + "%";
71     }
72 }
```

javax.swing.table.TableModel **1.2**

- `int getRowCount()`
- `int getColumnCount()`
  get the number of rows and columns in the table model.
- `Object getValueAt(int row, int column)`
  gets the value at the given row and column.
- `void setValueAt(Object newValue, int row, int column)`
  sets a new value at the given row and column.
- `boolean isCellEditable(int row, int column)`
  returns `true` if the cell at the given row and column is editable.
- `String getColumnName(int column)`
  gets the column title.

12.2. Working with Rows and Columns

In the following subsections, you will see how to manipulate the rows and columns in a table. As you read through this material, keep in mind that a Swing table is quite asymmetric—the operations that you can carry out on rows and columns are different. The table component was optimized to display rows of information with the same structure, such as the result of a database query, not an arbitrary two-dimensional grid of objects. You will see this asymmetry throughout this subsection.

12.2.1. Column Classes

In the next example, we again display our planet data, but this time we want to give the table more information about the column types. This is achieved by defining the method

```
Class<?> getColumnClass(int columnIndex)
```

of the table model to return the class that describes the column type.

The `JTable` class uses this information to pick an appropriate renderer for the class. **Table 12.1** shows the default rendering actions.

Table 12.1: Default Rendering Actions

Type	Rendered As
`Boolean`	Checkbox
`Icon`	Image
`Object`	String

You can see the checkboxes and images in **Figure 12.5**. (Thanks to Jim Evins for providing the planet images!)

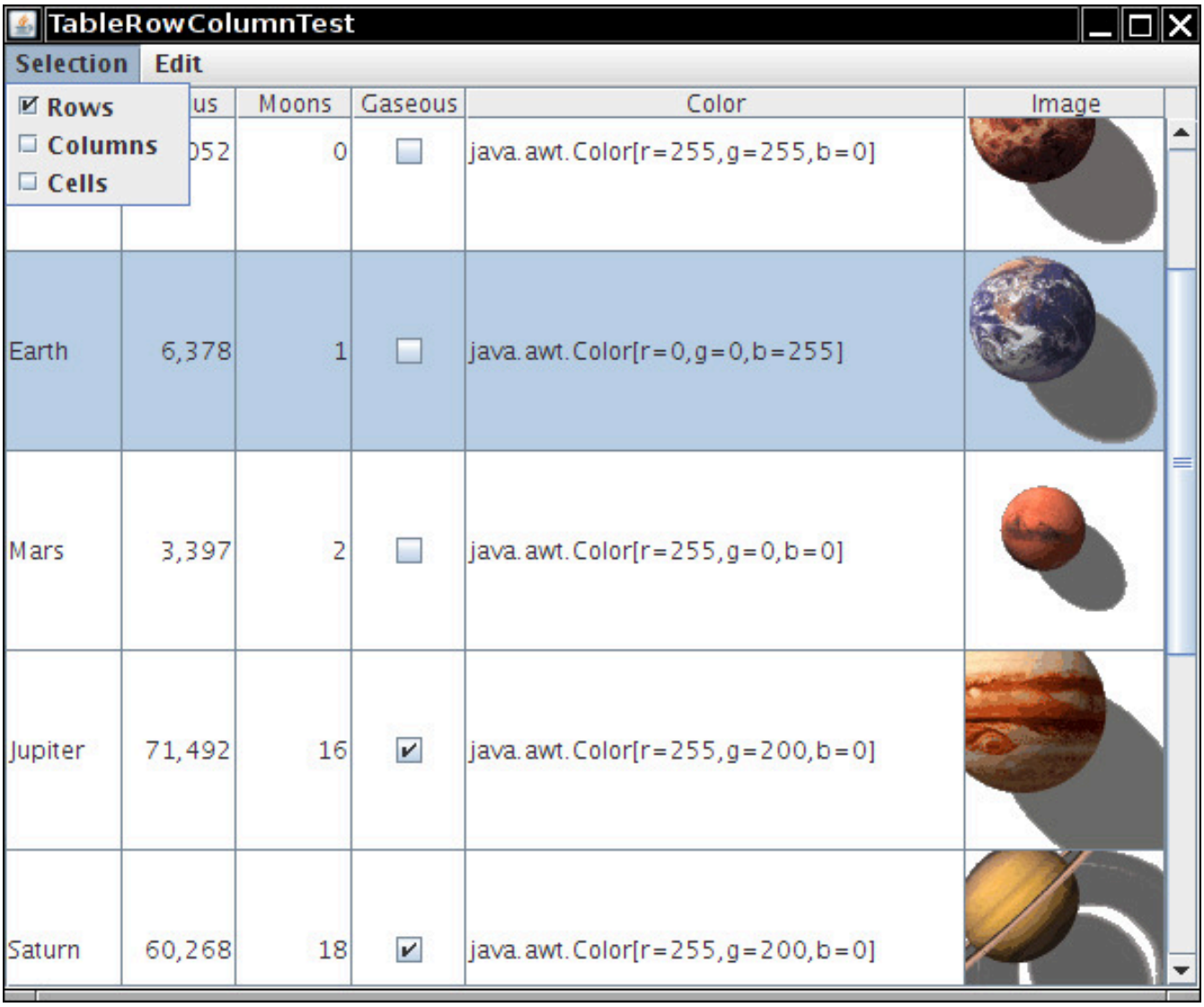


Figure 12.5: A table with planet data

To render other types, you can install a custom renderer—see **Section 12.3**.

12.2.2. Accessing Table Columns

The `JTable` class stores information about table columns in objects of type `TableColumn`. A `TableColumnModel` object manages the columns. (**Figure 12.6** shows the relationships among the most important table classes.) If you don't want to insert or remove columns dynamically, you won't use the column model much. The most common use for the column model is simply to get a `TableColumn` object:

```
int columnIndex = . . .;
TableColumn column = table.getColumnModel().getColumn(columnIndex);
```

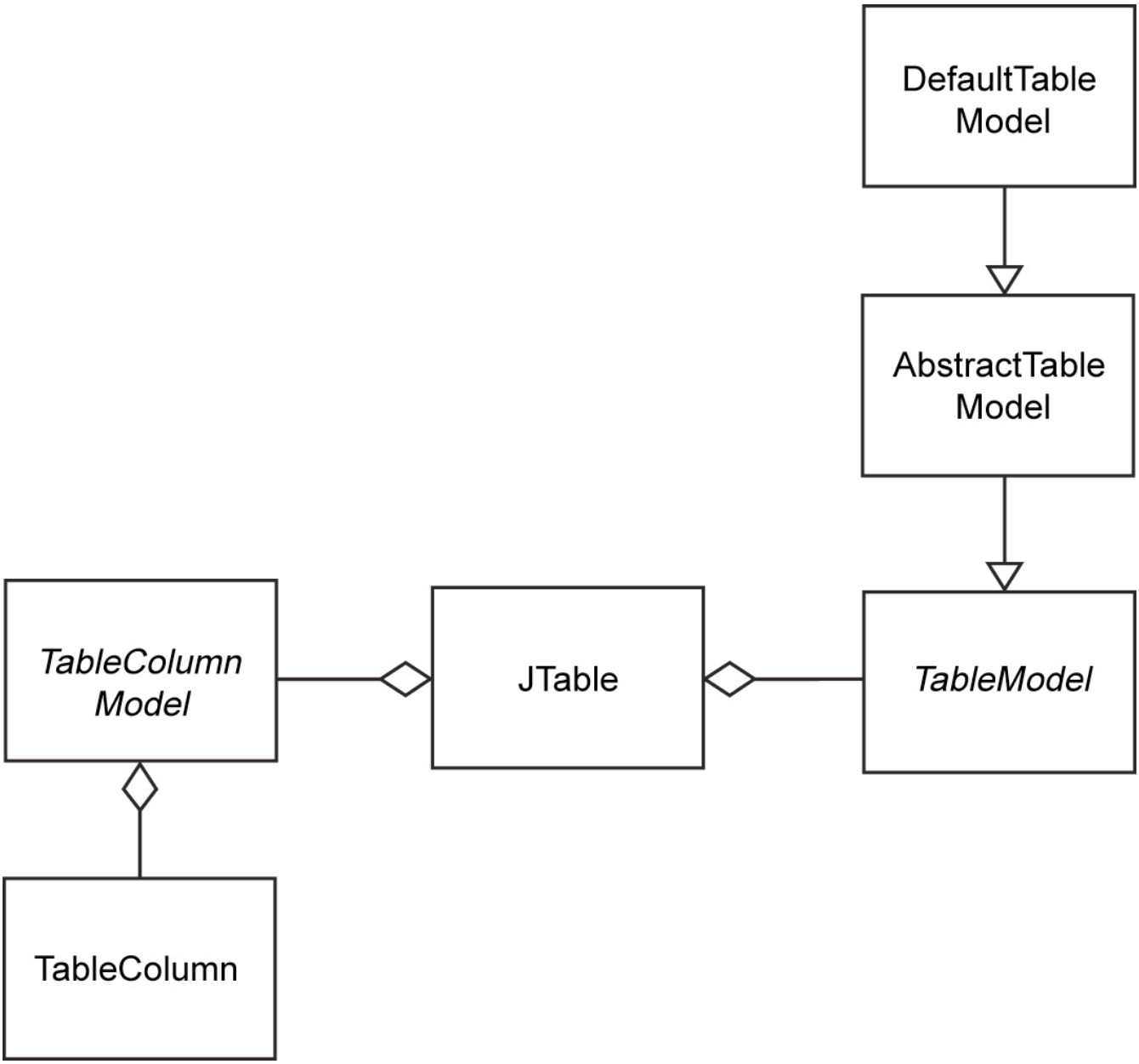


Figure 12.6: Relationships between table classes

12.2.3. Resizing Columns

The `TableColumn` class gives you control over the resizing behavior of columns. You can set the preferred, minimum, and maximum width with the methods

```
void setPreferredWidth(int width)
void setMinWidth(int width)
void setMaxWidth(int width)
```

This information is used by the table component to lay out the columns.

Use the method

```
void setResizable(boolean resizable)
```

to control whether the user is allowed to resize the column.

You can programmatically resize a column with the method

```
void setWidth(int width)
```

When a column is resized, the default is to leave the total size of the table unchanged. Of course, the width increase or decrease of the resized column must then be distributed over other columns. The default behavior is to change the size of all columns to the right of the resized column. That's a good default because it allows a user to adjust all columns to a desired width, moving from left to right.

You can set another behavior from **Table 12.2** by using the method

```
void setAutoResizeMode(int mode)
```

of the `JTable` class.

Table 12.2: Resize Modes

Mode	Behavior
`AUTO_RESIZE_OFF`	Don't resize other columns; change the table width.
`AUTO_RESIZE_NEXT_COLUMN`	Resize the next column only.
`AUTO_RESIZE_SUBSEQUENT_COLUMNS`	Resize all subsequent columns equally; this is the default behavior.
`AUTO_RESIZE_LAST_COLUMN`	Resize the last column only.
`AUTO_RESIZE_ALL_COLUMNS`	Resize all columns in the table; this is not a good choice because it prevents the user from adjusting multiple columns to a desired size.

12.2.4. Resizing Rows

Row heights are managed directly by the `JTable` class. If your cells are taller than the default, you may want to set the row height:

```
table.setRowHeight(height);
```

By default, all rows of the table have the same height. You can set the heights of individual rows with the call

```
table.setRowHeight(row, height);
```

The actual row height equals the row height set with these methods, reduced by the row margin. The default row margin is 1 pixel, but you can change it with the call

```
table.setRowMargin(margin);
```

12.2.5. Selecting Rows, Columns, and Cells

Depending on the selection mode, the user can select rows, columns, or individual cells in the table. By default, row selection is enabled. Clicking inside a cell selects the entire row (see **Figure 12.5**). Call

```
table.setRowSelectionAllowed(false);
```

to disable row selection.

When row selection is enabled, you can control whether the user is allowed to select a single row, a contiguous set of rows, or any set of rows. You need to retrieve the *selection model* and use its `setSelectionMode` method:

```
table.getSelectionModel().setSelectionMode(mode);
```

Here, `mode` is one of the three values:

```
ListSelectionModel.SINGLE_SELECTION
ListSelectionModel.SINGLE_INTERVAL_SELECTION
ListSelectionModel.MULTIPLE_INTERVAL_SELECTION
```

Column selection is disabled by default. You can turn it on with the call

```
table.setColumnSelectionAllowed(true);
```

Enabling both row and column selection is equivalent to enabling cell selection. The user then selects ranges of cells (see **Figure 12.7**). You can also enable that setting with the call

```
table.setCellSelectionEnabled(true);
```

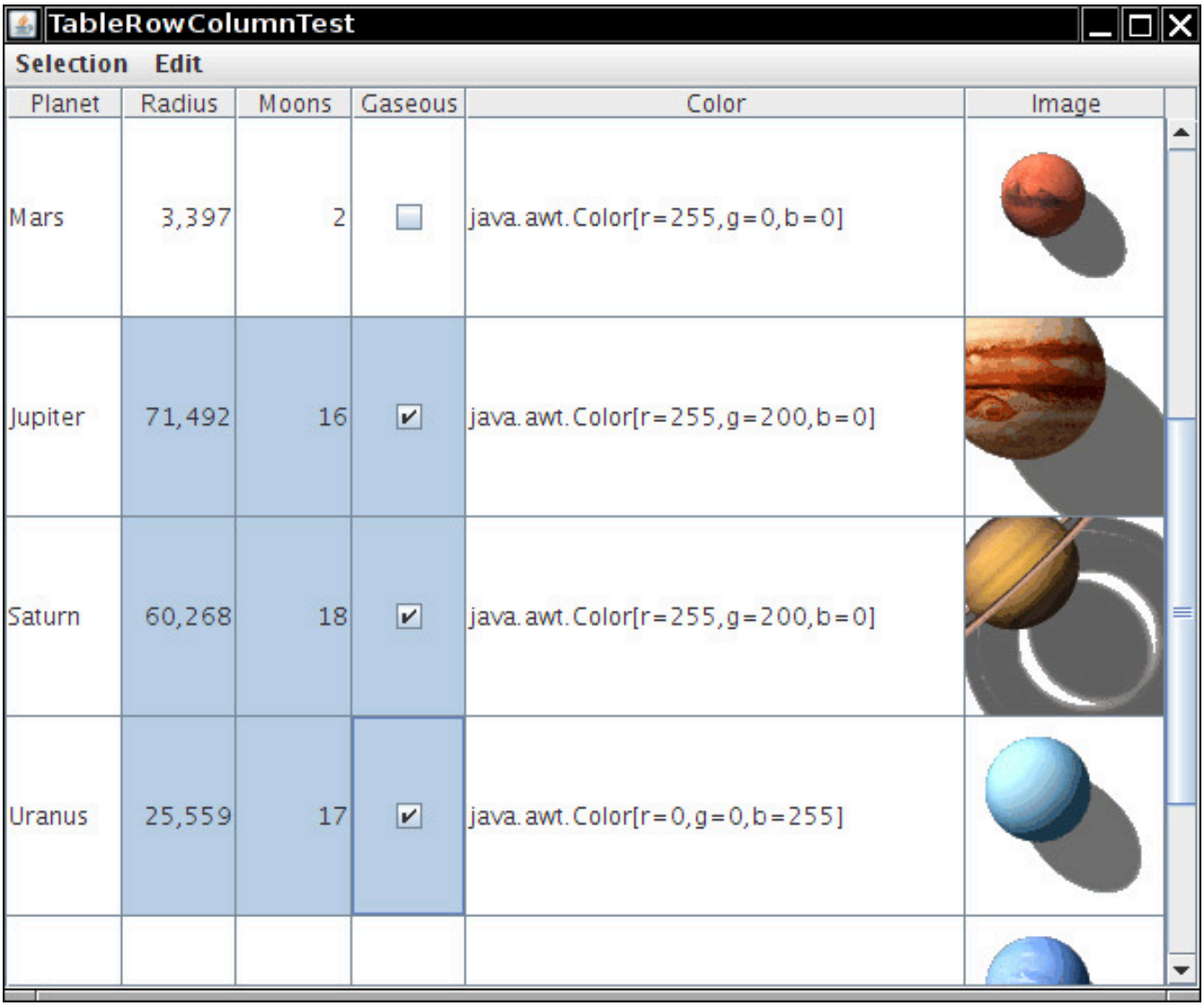


Figure 12.7: Selecting a range of cells

Run the program in **Listing 12.3** to watch cell selection in action. Enable row, column, or cell selection in the Selection menu and watch how the selection behavior changes.

You can find out which rows and columns are selected by calling the `getSelectedRows` and `getSelectedColumns` methods. Both return an `int[]` array of the indexes of the selected items. Note that the index values are those of the table view, not the underlying table model. Try selecting rows and columns, then drag columns to different places and sort the rows by clicking on column headers. Use the Print Selection menu item to see which rows and columns are reported as selected.

If you need to translate the table index values to table model index values, use the `JTable` methods `convertRowIndexToModel` and `convertColumnIndexToModel`.

12.2.6. Sorting Rows

As you have seen in our first table example, it is easy to add row sorting to a `JTable` simply by calling the `setAutoCreateRowSorter` method. However, to have finer-grained control over the sorting behavior, install a `TableRowSorter<M>` object into a `JTable` and customize it. The type parameter `M` denotes the table model; it needs to be a subtype of the `TableModel` interface.

```
var sorter = new TableRowSorter<TableModel>(model);
table.setRowSorter(sorter);
```

Some columns should not be sortable, such as the image column in our planet data. Turn sorting off by calling

```
sorter.setSortable(IMAGE_COLUMN, false);
```

You can install a custom comparator for each column. In our example, we will sort the colors in the Color column by preferring blue and green over red. When you click on the Color column, you will see that the blue planets go to the bottom of the table. This is achieved with the following call:

```
sorter.setComparator(COLOR_COLUMN, new Comparator<Color>() {
    public int compare(Color c1, Color c2) {
        int d = c1.getBlue() - c2.getBlue();
        if (d != 0) return d;
        d = c1.getGreen() - c2.getGreen();
        if (d != 0) return d;
        return c1.getRed() - c2.getRed();
    }
});
```

If you do not specify a comparator for a column, the sort order is determined as follows:

1. If the column class is `String`, use the default collator returned by `Collator.getInstance()`. It sorts strings in a way that is appropriate for the current locale. (See **Chapter 7** for more information about locales and collators.)
2. If the column class implements `Comparable`, use its `compareTo` method.
3. If a `TableStringConverter` has been set for the sorter, sort the strings returned by the converter's `toString` method with the default collator. If you want to use this approach, define a converter as follows:

   ```
   sorter.setStringConverter(new TableStringConverter() {
       public String toString(TableModel model, int row, int column) {
           Object value = model.getValueAt(row, column);
           convert value to a string and return it
       }
   });
   ```

4. Otherwise, call the `toString` method on the cell values and sort them with the default collator.

12.2.7. Filtering Rows

In addition to sorting rows, the `TableRowSorter` can also selectively hide rows—a process called *filtering*. To activate filtering, set a `RowFilter`. For example, to include all rows that contain at least one moon, call

```
sorter.setRowFilter(RowFilter.numberFilter(ComparisonType.NOT_EQUAL, 0, MOONS_COLUMN));
```

Here, we use a predefined number filter. To construct a number filter, supply

- The comparison type (one of `EQUAL`, `NOT_EQUAL`, `AFTER`, or `BEFORE`).
- An object of a subclass of `Number` (such as an `Integer` or `Double`). Only objects that have the same class as the given `Number` object are considered.
- Zero or more column index values. If no index values are supplied, all columns are searched.

The static `RowFilter.dateFilter` method constructs a date filter in the same way; you need to supply a `Date` object instead of the `Number` object.

Finally, the static `RowFilter.regexFilter` method constructs a filter that looks for strings matching a regular expression. For example,

```
sorter.setRowFilter(RowFilter.regexFilter(".*[^s]$", PLANET_COLUMN));
```

only displays those planets whose name doesn't end with an "s". (See **Chapter 2** for more information on regular expressions.)

You can also combine filters with the `andFilter`, `orFilter`, and `notFilter` methods. To filter for planets not ending in an "s" with at least one moon, you can use this filter combination:

```
sorter.setRowFilter(RowFilter.andFilter(List.of(
    RowFilter.regexFilter(".*[^s]$", PLANET_COLUMN),
    RowFilter.numberFilter(ComparisonType.NOT_EQUAL, 0, MOONS_COLUMN))));
```

To implement your own filter, provide a subclass of `RowFilter` and implement an `include` method to indicate which rows should be displayed. This is easy to do, but the glorious generality of the `RowFilter` class makes it a bit scary.

The `RowFilter<M, I>` class has two type parameters—the types for the model and for the row identifier. When dealing with tables, the model is always a subtype of `TableModel` and the identifier type is `Integer`. (At some point in the future, other components might also support row filtering. For example, to filter rows in a `JTree`, one might use a `RowFilter<TreeModel, TreePath>`.)

A row filter must implement the method

```
public boolean include(RowFilter.Entry<? extends M, ? extends I> entry)
```

The `RowFilter.Entry` class supplies methods to obtain the model, the row identifier, and the value at a given index. Therefore, you can filter both by row identifier and by the contents of the row.

For example, this filter displays every other row:

```
var filter = new RowFilter<TableModel, Integer>() {
    public boolean include(Entry<? extends TableModel, ? extends Integer> entry) {
        return entry.getIdentifier() % 2 == 0;
    }
};
```

If you wanted to include only those planets with an even number of moons, you would instead test for

```
((Integer) entry.getValue(MOONS_COLUMN)) % 2 == 0
```

In our sample program, we allow the user to hide arbitrary rows. We store the hidden row indexes in a set. The row filter shows all rows whose indexes are not in that set.

The filtering mechanism wasn't designed for filters with criteria changing over time. In our sample program, we keep calling

```
sorter.setRowFilter(filter);
```

whenever the set of hidden rows changes. Setting a filter causes it to be applied immediately.

12.2.8. Hiding and Displaying Columns

As you saw in the preceding section, you can filter table rows by either their contents or their row identifier. Hiding table columns uses a completely different mechanism.

The `removeColumn` method of the `JTable` class removes a column from the table view. The column data are not actually removed from the model—they are just hidden from view. The `removeColumn` method has a parameter of type `TableColumn`. If you have the column number (for example, from a call to `getSelectedColumns`), you need to ask the table model for the actual table column object:

```
TableColumnModel columnModel = table.getColumnModel();
TableColumn column = columnModel.getColumn(i);
table.removeColumn(column);
```

If you remember the column, you can later add it back in:

```
table.addColumn(column);
```

This method adds the column to the end. If you want it to appear elsewhere, call the `moveColumn` method.

You can also add a new column that corresponds to a column index in the table model, by adding a new `TableColumn` object:

```
table.addColumn(new TableColumn(modelColumnIndex));
```

You can have multiple table columns that view the same column of the model.

The program in **Listing 12.3** demonstrates selection and filtering of rows and columns.

Listing 12.3 v2ch12/tableRowColumn/PlanetTableFrame.java

```
1  package v2ch12.tableRowColumn;
2
3  import module java.base;
4  import module java.desktop;
5
6  /**
7   * This frame contains a table of planet data.
8   */
9  public class PlanetTableFrame extends JFrame {
10     private static final int DEFAULT_WIDTH = 600;
11     private static final int DEFAULT_HEIGHT = 500;
12
```

```
    public static final int COLOR_COLUMN = 4;
    public static final int IMAGE_COLUMN = 5;

    private JTable table;
    private HashSet<Integer> removedRowIndices;
    private ArrayList<TableColumn> removedColumns;
    private JCheckBoxMenuItem rowsItem;
    private JCheckBoxMenuItem columnsItem;
    private JCheckBoxMenuItem cellsItem;

    private String[] columnNames = { "Planet", "Radius", "Moons", "Gaseous", "Color",
            "Image" };

    private Object[][] cells = {
            { "Mercury", 2440.0, 0, false, Color.YELLOW,
                    new ImageIcon("images/Mercury.gif") },
            { "Venus", 6052.0, 0, false, Color.YELLOW,
                    new ImageIcon("images/Venus.gif") },
            { "Earth", 6378.0, 1, false, Color.BLUE,
                    new ImageIcon("images/Earth.gif") },
            { "Mars", 3397.0, 2, false, Color.RED,
                    new ImageIcon("images/Mars.gif") },
            { "Jupiter", 71492.0, 16, true, Color.ORANGE,
                    new ImageIcon("images/Jupiter.gif") },
            { "Saturn", 60268.0, 18, true, Color.ORANGE,
                    new ImageIcon("images/Saturn.gif") },
            { "Uranus", 25559.0, 17, true, Color.BLUE,
                    new ImageIcon("images/Uranus.gif") },
            { "Neptune", 24766.0, 8, true, Color.BLUE,
                    new ImageIcon("images/Neptune.gif") },
            { "Pluto", 1137.0, 1, false, Color.BLACK,
                    new ImageIcon("images/Pluto.gif") } };

    public PlanetTableFrame() {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        var model = new DefaultTableModel(cells, columnNames) {
            public Class<?> getColumnClass(int c) {
                return cells[0][c].getClass();
            }
        };

        table = new JTable(model);

        table.setRowHeight(100);
        table.getColumnModel().getColumn(COLOR_COLUMN).setMinWidth(250);
        table.getColumnModel().getColumn(IMAGE_COLUMN).setMinWidth(100);

        var sorter = new TableRowSorter<TableModel>(model);
        table.setRowSorter(sorter);
        sorter.setComparator(COLOR_COLUMN, Comparator.comparing(Color::getBlue)
                .thenComparing(Color::getGreen).thenComparing(Color::getRed));
        sorter.setSortable(IMAGE_COLUMN, false);
        add(new JScrollPane(table), BorderLayout.CENTER);

        removedRowIndices = new HashSet<>();
        removedColumns = new ArrayList<>();

```

```
        var filter = new RowFilter<TableModel, Integer>() {
            public boolean include(Entry<? extends TableModel, ? extends Integer> entry) {
                return !removedRowIndices.contains(entry.getIdentifier());
            }
        };

        // create menu

        var menuBar = new JMenuBar();
        setJMenuBar(menuBar);

        var selectionMenu = new JMenu("Selection");
        menuBar.add(selectionMenu);

        rowsItem = new JCheckBoxMenuItem("Rows");
        columnsItem = new JCheckBoxMenuItem("Columns");
        cellsItem = new JCheckBoxMenuItem("Cells");

        rowsItem.setSelected(table.getRowSelectionAllowed());
        columnsItem.setSelected(table.getColumnSelectionAllowed());
        cellsItem.setSelected(table.getCellSelectionEnabled());

        rowsItem.addActionListener(_ -> {
            table.clearSelection();
            table.setRowSelectionAllowed(rowsItem.isSelected());
            updateCheckboxMenuItems();
        });
        selectionMenu.add(rowsItem);

        columnsItem.addActionListener(_ -> {
            table.clearSelection();
            table.setColumnSelectionAllowed(columnsItem.isSelected());
            updateCheckboxMenuItems();
        });
        selectionMenu.add(columnsItem);

        cellsItem.addActionListener(_ -> {
            table.clearSelection();
            table.setCellSelectionEnabled(cellsItem.isSelected());
            updateCheckboxMenuItems();
        });
        selectionMenu.add(cellsItem);

        var tableMenu = new JMenu("Edit");
        menuBar.add(tableMenu);

        var hideColumnsItem = new JMenuItem("Hide Columns");
        hideColumnsItem.addActionListener(_ -> {
            int[] selected = table.getSelectedColumns();
            TableColumnModel columnModel = table.getColumnModel();

            // remove columns from view, starting at the last
            // index so that column numbers aren't affected

            for (int i = selected.length - 1; i >= 0; i--) {
                TableColumn column = columnModel.getColumn(selected[i]);
                table.removeColumn(column);

```

```
                // store removed columns for "show columns" command

                removedColumns.add(column);
            }
        });
        tableMenu.add(hideColumnsItem);

        var showColumnsItem = new JMenuItem("Show Columns");
        showColumnsItem.addActionListener(_ -> {
            // restore all removed columns
            for (TableColumn tc : removedColumns)
                table.addColumn(tc);
            removedColumns.clear();
        });
        tableMenu.add(showColumnsItem);

        var hideRowsItem = new JMenuItem("Hide Rows");
        hideRowsItem.addActionListener(_ -> {
            int[] selected = table.getSelectedRows();
            for (int i : selected)
                removedRowIndices.add(table.convertRowIndexToModel(i));
            sorter.setRowFilter(filter);
        });
        tableMenu.add(hideRowsItem);

        var showRowsItem = new JMenuItem("Show Rows");
        showRowsItem.addActionListener(_ -> {
            removedRowIndices.clear();
            sorter.setRowFilter(filter);
        });
        tableMenu.add(showRowsItem);

        var printSelectionItem = new JMenuItem("Print Selection");
        printSelectionItem.addActionListener(_ -> {
            int[] selected = table.getSelectedRows();
            System.out.println("Selected rows: " + Arrays.toString(selected));
            selected = table.getSelectedColumns();
            System.out.println("Selected columns: " + Arrays.toString(selected));
        });
        tableMenu.add(printSelectionItem);
    }

    private void updateCheckboxMenuItems() {
        rowsItem.setSelected(table.getRowSelectionAllowed());
        columnsItem.setSelected(table.getColumnSelectionAllowed());
        cellsItem.setSelected(table.getCellSelectionEnabled());
    }
}
```

javax.swing.table.TableModel **1.2**

- `Class getColumnClass(int columnIndex)`
  gets the class for the values in this column. This information is used for sorting and rendering.

`javax.swing.JTable` 1.2

- `TableColumnModel getColumnModel()`
  gets the column model that describes the arrangement of the table columns.
- `void setAutoResizeMode(int mode)`
  sets the mode for automatic resizing of table columns. . The `mode` parameter is one of `AUTO_RESIZE_OFF`, `AUTO_RESIZE_NEXT_COLUMN`, `AUTO_RESIZE_SUBSEQUENT_COLUMNS`, `AUTO_RESIZE_LAST_COLUMN`, and `AUTO_RESIZE_ALL_COLUMNS`
- `int getRowMargin()`
- `void setRowMargin(int margin)`
  get or set the amount of empty space between cells in adjacent rows.
- `int getRowHeight()`
- `void setRowHeight(int height)`
  get or set the default height of all rows of the table.
- `int getRowHeight(int row)`
- `void setRowHeight(int row, int height)`
  get or set the height of the given row of the table.
- `ListSelectionModel getSelectionModel()`
  returns the list selection model. You need that model to choose between row, column, and cell selection.
- `boolean getRowSelectionAllowed()`
- `void setRowSelectionAllowed(boolean b)`
  get or set the `rowSelectionAllowed` property. If `true`, rows are selected when the user clicks on cells.
- `boolean getColumnSelectionAllowed()`
- `void setColumnSelectionAllowed(boolean b)`
  get or set the `columnSelectionAllowed` property. If `true`, columns are selected when the user clicks on cells.
- `boolean getCellSelectionEnabled()`
  returns `true` if both `rowSelectionAllowed` and `columnSelectionAllowed` are `true`.
- `void setCellSelectionEnabled(boolean b)`
  sets both `rowSelectionAllowed` and `columnSelectionAllowed` to `b`.
- `void addColumn(TableColumn column)`
  adds a column as the last column of the table view.
- `void moveColumn(int from, int to)`
  moves the column whose table index is `from` so that its index becomes `to`. Only the view is affected.
- `void removeColumn(TableColumn column)`
  removes the given column from the view.
- `int convertRowIndexToModel(int index)` 6
- `int convertColumnIndexToModel(int index)`
  return the model index of the row or column with the given index. This value is different from `index` when rows are sorted or filtered, or when columns are moved or removed.
- `void setRowSorter(RowSorter<? extends TableModel> sorter)`
  sets the row sorter.

javax.swing.table.TableColumnModel 1.2

- `TableColumn getColumn(int index)`
  gets the table column object that describes the column with the given view index.

`javax.swing.table.TableColumn` 1.2

- `TableColumn(int modelColumnIndex)`
  constructs a table column for viewing the model column with the given index.
- `void setPreferredWidth(int width)`
- `void setMinWidth(int width)`
- `void setMaxWidth(int width)`
  set the preferred, minimum, and maximum width of this table column to `width`.
- `void setWidth(int width)`
  sets the actual width of this column to `width`.
- `void setResizable(boolean b)`
  If `b` is `true`, this column is resizable.

javax.swing.ListSelectionModel 1.2

- `void setSelectionMode(int mode)`
  sets the selection mode to one of `SINGLE_SELECTION`, `SINGLE_INTERVAL_SELECTION`, and `MULTIPLE_INTERVAL_SELECTION`

`javax.swing.DefaultRowSorter<M, I>` 6

- `void setComparator(int column, Comparator<?> comparator)`
  sets the comparator to be used with the given column.
- `void setSortable(int column, boolean enabled)`
  enables or disables sorting for the given column.
- `void setRowFilter(RowFilter<? super M,? super I> filter)`
  sets the row filter.

`javax.swing.table.TableRowSorter<M extends TableModel>` 6

- `void setStringConverter(TableStringConverter stringConverter)`
  sets the string converter used for sorting and filtering.

`javax.swing.table.TableStringConverter` 6

- `abstract String toString(TableModel model, int row, int column)`
  converts the model value at the given location to a string; you can override this method.

`javax.swing.RowFilter<M, I>` 6

- `boolean include(RowFilter.Entry<? extends M,? extends I> entry)`
  specifies the rows that are retained; you can override this method.
- `static <M,I> RowFilter<M,I> numberFilter(RowFilter.ComparisonType type, Number number, int... indices)`
- `static <M,I> RowFilter<M,I> dateFilter(RowFilter.ComparisonType type, Date date, int... indices)`
  return a filter that includes rows containing values that match the given comparison to the given number or date. The comparison type is one of `EQUAL`, `NOT_EQUAL`, `AFTER`, or `BEFORE`. If any column model in-

dexes are given, only those columns are searched; otherwise, all columns are searched. For the number filter, the class of the cell value must match the class of `number`.

- `static <M,I> RowFilter<M,I> regexFilter(String regex, int... indices)`
  returns a filter that includes rows that have a string value matching the given regular expression. If any column model indexes are given, only those columns are searched; otherwise, all columns are searched. Note that the string returned by the `getStringValue` method of `RowFilter.Entry` is matched.
- `static <M,I> RowFilter<M,I> andFilter(Iterable<? extends RowFilter<? super M,? super I>> filters)`
- `static <M,I> RowFilter<M,I> orFilter(Iterable<? extends RowFilter<? super M,? super I>> filters)`
  return a filter that includes the entries included by all filters, or at least one of the filters.
- `static <M,I> RowFilter<M,I> notFilter(RowFilter<M,I> filter)`
  returns a filter that includes the entries not included by the given filter.

`javax.swing.RowFilter.Entry<M, I>` 6

- `I getIdentifier()`
  returns the identifier of this row entry.
- `M getModel()`
  returns the model of this row entry.
- `Object getValue(int index)`
  returns the value stored at the given index of this row.
- `int getValueCount()`
  returns the number of values stored in this row.
- `String getStringValue(int index)`
  returns the value stored at the given index of this row, converted to a string. The `TableRowSorter` produces entries whose `getStringValue` calls the sorter's string converter.

12.3. Cell Rendering and Editing

As you saw in **Section 12.2.2**, the column type determines how the cells are rendered. There are default renderers for the types `Boolean` and `Icon` that render a checkbox or icon. For all other types, you need to install a custom renderer.

12.3.1. Rendering Cells

Table cell renderers are similar to the list cell renderers that you saw earlier. They implement the `TableCellRenderer` interface that has a single method:

```
Component getTableCellRendererComponent(JTable table, Object value,
      boolean isSelected, boolean hasFocus, int row, int column)
```

That method is called when the table needs to draw a cell. You return a component whose `paint` method is then invoked to fill the cell area.

The table in **Figure 12.8** contains cells of type `Color`. The renderer simply returns a panel with a background color that is the color object stored in the cell. The color is passed as the `value` parameter.

```
class ColorTableCellRenderer extends JPanel implements TableCellRenderer {
    public Component getTableCellRendererComponent(JTable table, Object value,
            boolean isSelected, boolean hasFocus, int row, int column) {
        setBackground((Color) value);
        if (hasFocus)
            setBorder(UIManager.getBorder("Table.focusCellHighlightBorder"));
        else
            setBorder(null);
        return this;
    }
}
```

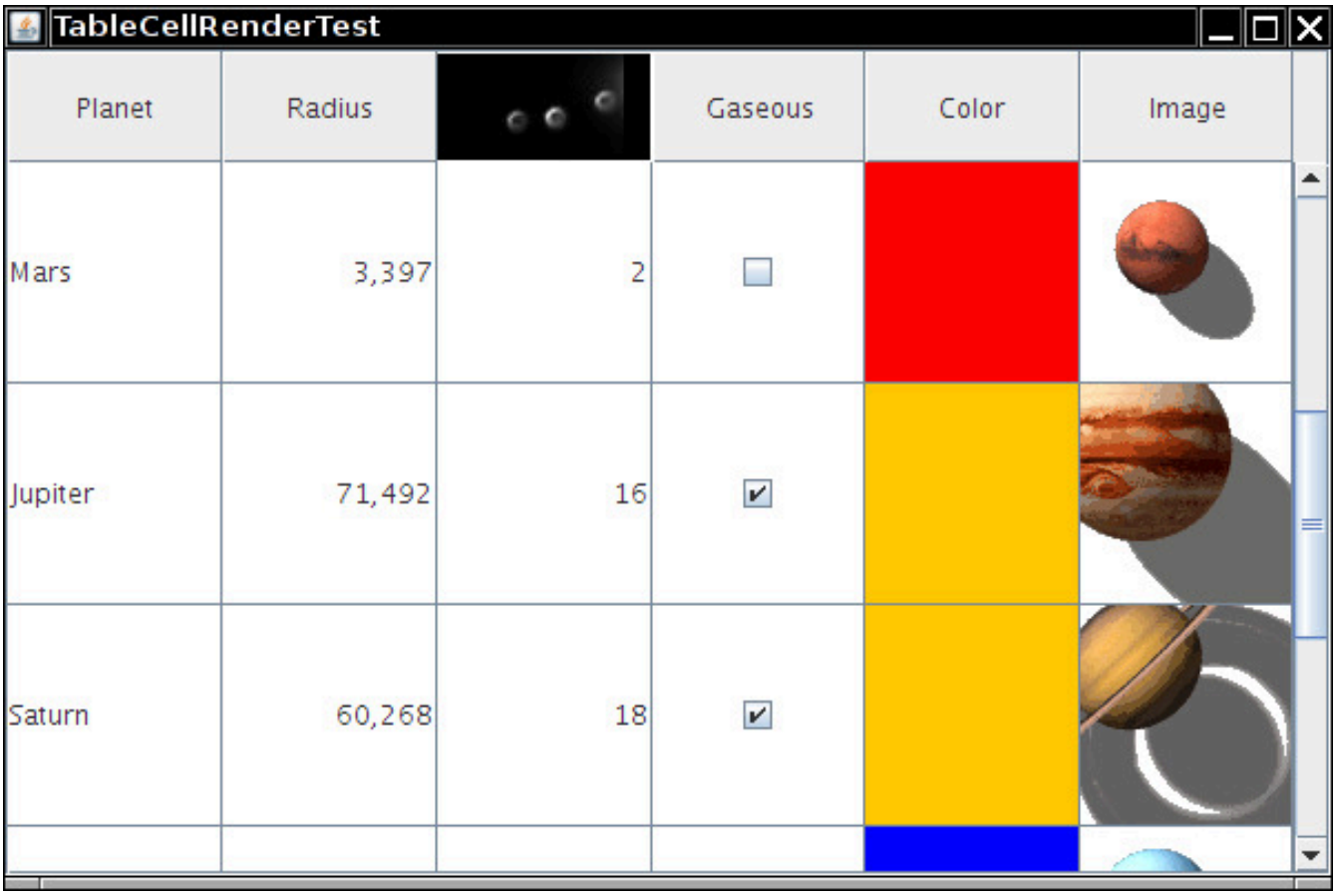


Figure 12.8: A table with cell renderers

As you can see, the renderer draws a border when the cell has focus. (We ask the `UIManager` for the correct border. To find the lookup key, we peeked into the source code of the `DefaultTableCellRenderer` class.)

Tip: If your renderer simply draws a text string or an icon, you can extend the `DefaultTableCellRenderer` class. It takes care of rendering the focus and selection status for you.

You need to tell the table to use this renderer with all objects of type `Color`. The `setDefaultRenderer` method of the `JTable` class lets you establish this association. Supply a `Class` object and the renderer:

```
table.setDefaultRenderer(Color.class, new ColorTableCellRenderer());
```

That renderer is now used for all objects of the given type in this table.

If you want to select a renderer based on some other criterion, you need to subclass the `JTable` class and override the `getCellRenderer` method.

12.3.2. Rendering the Header

To display an icon in the header, set the header value:

```
moonColumn.setHeaderValue(new ImageIcon("Moons.gif"));
```

However, the table header isn't smart enough to choose an appropriate renderer for the header value. You have to install the renderer manually. For example, to show an image icon in a column header, call

```
moonColumn.setHeaderRenderer(table.getDefaultRenderer(ImageIcon.class));
```

12.3.3. Editing Cells

To enable cell editing, the table model must indicate which cells are editable by defining the `isCellEditable` method. Most commonly, you will want to make certain columns editable. In the example program, we allow editing in four columns.

```
public boolean isCellEditable(int r, int c) {
    return c == PLANET_COLUMN || c == MOONS_COLUMN || c == GASEOUS_COLUMN
        || c == COLOR_COLUMN;
}
```



Note: The `AbstractTableModel` defines the `isCellEditable` method to always return `false`. The `DefaultTableModel` overrides the method to always return `true`.

If you run the sample program at the end of this section, note that you can click the checkboxes in the Gaseous column and turn the check marks on and off. If you click a cell in the Moons column, a combo box appears (see **Figure 12.9**). You will shortly see how to install such a combo box as a cell editor.

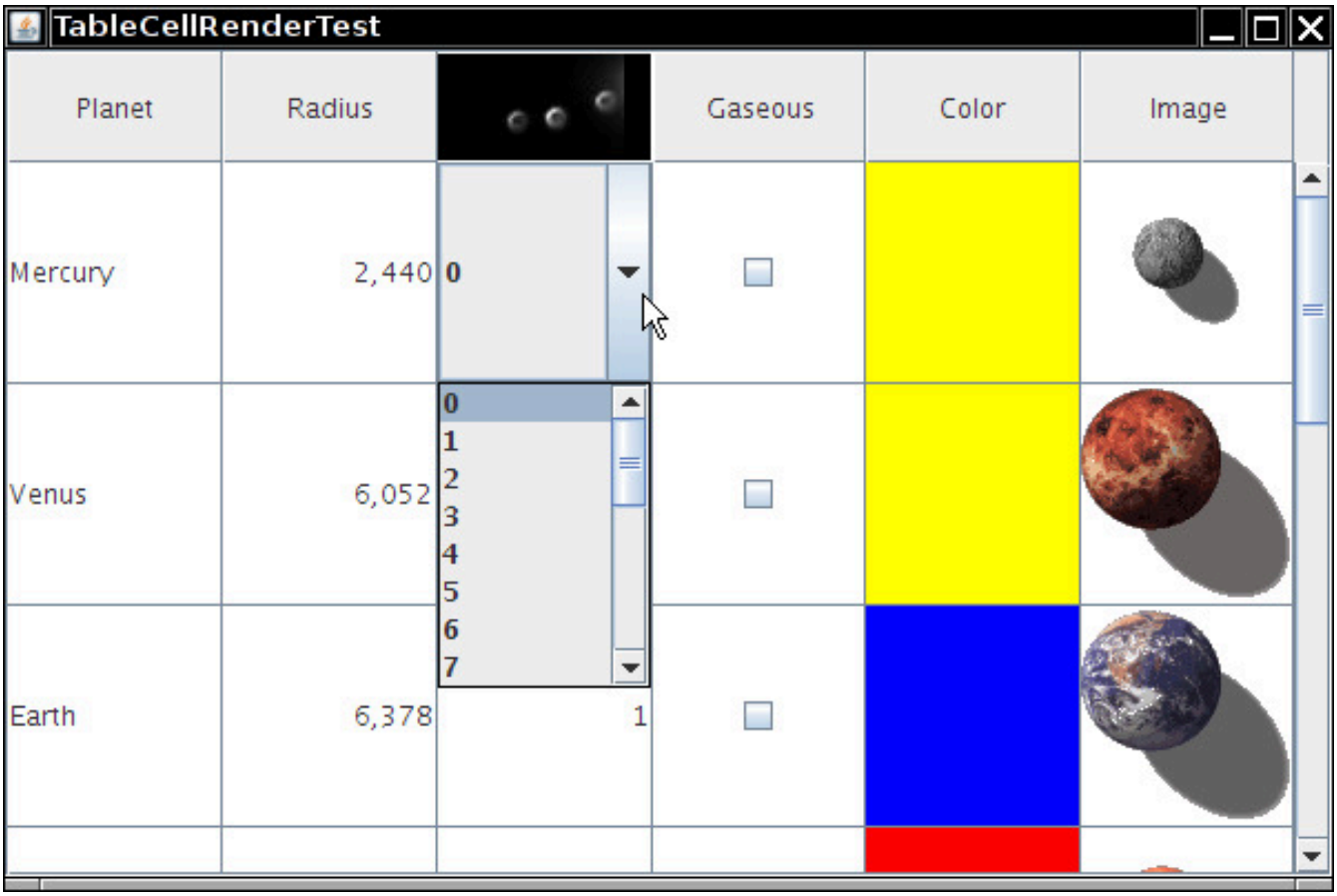


Figure 12.9: A cell editor

Finally, click a cell in the first column. The cell gains focus. You can start typing, and the cell contents change.

What you just saw in action are the three variations of the `DefaultCellEditor` class. A `DefaultCellEditor` can be constructed with a `JTextField`, a `JCheckBox`, or a `JComboBox`. The `JTable` class automatically installs a checkbox editor for `Boolean` cells and a text field editor for all editable cells that don't supply their own renderer. The text fields let the user edit the strings that result from applying `toString` to the return value of the `getValueAt` method of the table model.

When the edit is complete, the edited value is retrieved by calling the `getCellEditorValue` method of your editor. That method should return a value of the correct type (that is, the type returned by the `getColumnType` method of the model).

To get a combo box editor, set a cell editor manually—the `JTable` component has no idea what values might be appropriate for a particular type. For the Moons column, we wanted to enable the user to pick any value between 0 and 20. Here is the code for initializing the combo box:

```
var moonCombo = new JComboBox();
for (int i = 0; i <= 20; i++)
    moonCombo.addItem(i);
```

To construct a `DefaultCellEditor`, supply the combo box in the constructor:

```
var moonEditor = new DefaultCellEditor(moonCombo);
```

Next, we need to install the editor. Unlike the color cell renderer, this editor does not depend on the object *type*—we don't necessarily want to use it for all objects of type `Integer`. Instead, we need to install it into a particular

column:

```
moonColumn.setCellEditor(moonEditor);
```

12.3.4. Custom Editors

Run the example program again and click a color. A *color chooser* pops up and lets you pick a new color for the planet. Select a color and click OK. The cell color is updated (see **Figure 12.10**).

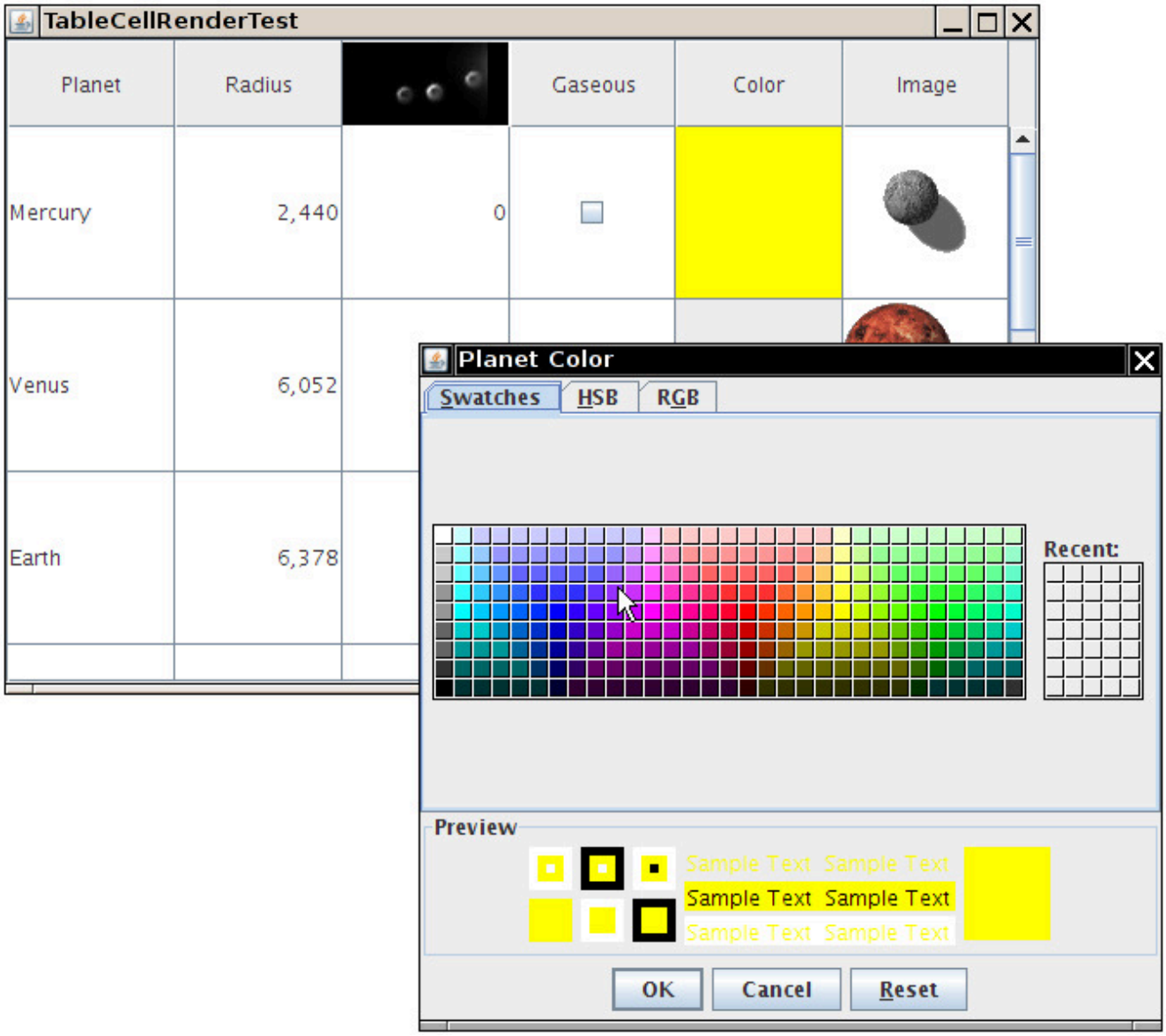


Figure 12.10: Editing the cell color with a color chooser

The color cell editor is not a standard table cell editor but a custom implementation. To create a custom cell editor, implement the `TableCellEditor` interface. That interface is a bit tedious, and an `AbstractCellEditor` class is provided to take care of the event handling details.

The `getTableCellEditorComponent` method of the `TableCellEditor` interface requests a component to render the cell. It is exactly the same as the `getTableCellRendererComponent` method of the `TableCellRenderer` interface, except that there is no `focus` parameter. When the cell is being edited, it is presumed to have focus. The editor component temporarily *replaces* the renderer when the editing is in progress. In our example, we return a blank panel that is not colored. This is an indication to the user that the cell is currently being edited.

Next, you want to have your editor pop up when the user clicks on the cell.

The `JTable` class calls your editor with an event (such as a mouse click) to find out if that event is acceptable to initiate the editing process. The `AbstractCellEditor` class defines the method to accept all events.

```
public boolean isCellEditable(EventObject anEvent) {
     return true;
}
```

However, if you override this method to return `false`, the table would not go through the trouble of inserting the editor component.

Once the editor component is installed, the `shouldSelectCell` method is called, presumably with the same event. You should initiate editing in this method—for example, by popping up an external edit dialog box.

```
public boolean shouldSelectCell(EventObject anEvent) {
    colorDialog.setVisible(true);
    return true;
}
```

If the user cancels the edit, the table calls the `cancelCellEditing` method. If the user has clicked on another table cell, the table calls the `stopCellEditing` method. In both cases, you should hide the dialog box. When your `stopCellEditing` method is called, the table would like to use the partially edited value. You should return `true` if the current value is valid. In the color chooser, any value is valid. But if you edit other data, you can ensure that only valid data are retrieved from the editor.

Also, you should call the superclass methods that take care of event firing—otherwise, the editing won't be properly canceled.

```
public void cancelCellEditing() {
     colorDialog.setVisible(false);
     super.cancelCellEditing();
}
```

Finally, you need a method that yields the value that the user supplied in the editing process:

```
public Object getCellEditorValue() {
     return colorChooser.getColor();
}
```

To summarize, your custom editor should do the following:

1. Extend the `AbstractCellEditor` class and implement the `TableCellEditor` interface.
2. Define the `getTableCellEditorComponent` method to supply a component. This can either be a dummy component (if you pop up a dialog box) or a component for in-place editing such as a combo box or text field.
3. Define the `shouldSelectCell`, `stopCellEditing`, and `cancelCellEditing` methods to handle the start, completion, and cancellation of the editing process. The `stopCellEditing` and `cancelCellEditing` methods should call the superclass methods to ensure that listeners are notified.
4. Define the `getCellEditorValue` method to return the value that is the result of the editing process.

Finally, indicate when the user is finished editing by calling the `stopCellEditing` and `cancelCellEditing` methods. When constructing the color dialog box, we install the accept and cancel callbacks that fire these events.

```
colorDialog = JColorChooser.createDialog(null, "Planet Color", false, colorChooser,
    EventHandler.create(ActionListener.class, this, "stopCellEditing"),
    EventHandler.create(ActionListener.class, this, "cancelCellEditing"));
```

This completes the implementation of the custom editor.

You now know how to make a cell editable and how to install an editor. There is one remaining issue—how to update the model with the value that the user edited. When editing is complete, the `JTable` class calls the following method of the table model:

```
void setValueAt(Object value, int r, int c)
```

You need to override the method to store the new value. The `value` parameter is the object that was returned by the cell editor. If you implemented the cell editor, you know the type of the object you return from the `getCellEditorValue` method. In the case of the `DefaultCellEditor`, there are three possibilities for that value. It is a `Boolean` if the cell editor is a checkbox, a string if it is a text field, and, if the value comes from a combo box, it is the object that the user selected.

If the `value` object does not have the appropriate type, you need to convert it. That happens most commonly when a number is edited in a text field. In our example, we populated the combo box with `Integer` objects so that no conversion is necessary.

Listing 12.4 v2ch12/tableCellRender/TableCellRenderFrame.java

```
1  package v2ch12.tableCellRender;
2
3  import module java.desktop;
4
5  /**
6   * This frame contains a table of planet data.
7   */
8  public class TableCellRenderFrame extends JFrame {
9      private static final int DEFAULT_WIDTH = 600;
10     private static final int DEFAULT_HEIGHT = 400;
11
12     public TableCellRenderFrame() {
13         setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
14
15         var model = new PlanetTableModel();
16         var table = new JTable(model);
17         table.setRowSelectionAllowed(false);
18
19         // set up renderers and editors
20
21         table.setDefaultRenderer(Color.class, new ColorTableCellRenderer());
22         table.setDefaultEditor(Color.class, new ColorTableCellEditor());
23
24         var moonCombo = new JComboBox<Integer>();
25         for (int i = 0; i <= 20; i++)
26             moonCombo.addItem(i);
```

```

        TableColumnModel columnModel = table.getColumnModel();
        TableColumn moonColumn = columnModel.getColumn(PlanetTableModel.MOONS_COLUMN);
        moonColumn.setCellEditor(new DefaultCellEditor(moonCombo));
        moonColumn.setHeaderRenderer(table.getDefaultRenderer(ImageIcon.class));
        moonColumn.setHeaderValue(new ImageIcon("images/Moons.gif"));

        // show table

        table.setRowHeight(100);
        add(new JScrollPane(table), BorderLayout.CENTER);
    }
}
```

Listing 12.5 v2ch12/tableCellRender/PlanetTableModel.java

```
package v2ch12.tableCellRender;

import module java.desktop;

/**
 * The planet table model specifies the values, rendering and editing properties for the planet
 * data.
 */
public class PlanetTableModel extends AbstractTableModel {
    public static final int PLANET_COLUMN = 0;
    public static final int MOONS_COLUMN = 2;
    public static final int GASEOUS_COLUMN = 3;
    public static final int COLOR_COLUMN = 4;

    private Object[][] cells = {
            { "Mercury", 2440.0, 0, false, Color.YELLOW,
                    new ImageIcon("images/Mercury.gif") },
            { "Venus", 6052.0, 0, false, Color.YELLOW,
                    new ImageIcon("images/Venus.gif") },
            { "Earth", 6378.0, 1, false, Color.BLUE,
                    new ImageIcon("images/Earth.gif") },
            { "Mars", 3397.0, 2, false, Color.RED,
                    new ImageIcon("images/Mars.gif") },
            { "Jupiter", 71492.0, 16, true, Color.ORANGE,
                    new ImageIcon("images/Jupiter.gif") },
            { "Saturn", 60268.0, 18, true, Color.ORANGE,
                    new ImageIcon("images/Saturn.gif") },
            { "Uranus", 25559.0, 17, true, Color.BLUE,
                    new ImageIcon("images/Uranus.gif") },
            { "Neptune", 24766.0, 8, true, Color.BLUE,
                    new ImageIcon("images/Neptune.gif") },
            { "Pluto", 1137.0, 1, false, Color.BLACK,
                    new ImageIcon("images/Pluto.gif") } };

    private String[] columnNames = { "Planet", "Radius", "Moons", "Gaseous", "Color",
            "Image" };

    public String getColumnName(int c) {
        return columnNames[c];
    }

```

```
    public Class<?> getColumnClass(int c) {
        return cells[0][c].getClass();
    }

    public int getColumnCount() {
        return cells[0].length;
    }

    public int getRowCount() {
        return cells.length;
    }

    public Object getValueAt(int r, int c) {
        return cells[r][c];
    }

    public void setValueAt(Object obj, int r, int c) {
        cells[r][c] = obj;
    }

    public boolean isCellEditable(int r, int c) {
        return c == PLANET_COLUMN || c == MOONS_COLUMN || c == GASEOUS_COLUMN
                || c == COLOR_COLUMN;
    }
}
```

Listing 12.6 v2ch12/tableCellRender/ColorTableCellRenderer.java

```
package v2ch12.tableCellRender;

import module java.desktop;

/**
 * This renderer renders a color value as a panel with the given color.
 */
public class ColorTableCellRenderer extends JPanel implements TableCellRenderer {
    public Component getTableCellRendererComponent(JTable table, Object value,
            boolean isSelected, boolean hasFocus, int row, int column) {
        setBackground((Color) value);
        if (hasFocus)
            setBorder(UIManager.getBorder("Table.focusCellHighlightBorder"));
        else
            setBorder(null);
        return this;
    }
}
```

Listing 12.7 v2ch12/tableCellRender/ColorTableCellEditor.java

```
package v2ch12.tableCellRender;

import module java.base;
import module java.desktop;

/**
 * This editor pops up a color dialog to edit a cell value.
```

```
 */
public class ColorTableCellEditor extends AbstractCellEditor implements TableCellEditor {
    private JColorChooser colorChooser;
    private JDialog colorDialog;
    private JPanel panel;

    public ColorTableCellEditor() {
        panel = new JPanel();
        // prepare color dialog

        colorChooser = new JColorChooser();
        colorDialog = JColorChooser.createDialog(null, "Planet Color", false, colorChooser,
                EventHandler.create(ActionListener.class, this, "stopCellEditing"),
                EventHandler.create(ActionListener.class, this, "cancelCellEditing"));
    }

    public Component getTableCellEditorComponent(JTable table, Object value,
            boolean isSelected, int row, int column) {
        // this is where we get the current Color value. We store it in the dialog in case the
        // user starts editing
        colorChooser.setColor((Color) value);
        return panel;
    }

    public boolean shouldSelectCell(EventObject anEvent) {
        // start editing
        colorDialog.setVisible(true);

        // tell caller it is ok to select this cell
        return true;
    }

    public void cancelCellEditing() {
        // editing is canceled--hide dialog
        colorDialog.setVisible(false);
        super.cancelCellEditing();
    }

    public boolean stopCellEditing() {
        // editing is complete--hide dialog
        colorDialog.setVisible(false);
        super.stopCellEditing();

        // tell caller it is ok to use color value
        return true;
    }

    public Object getCellEditorValue() {
        return colorChooser.getColor();
    }
}
```

`javax.swing.JTable` 1.2

- `TableCellRenderer getDefaultRenderer(Class<?> type)`
  gets the default renderer for the given type.

- `TableCellEditor getDefaultEditor(Class<?> type)`
  gets the default editor for the given type.

javax.swing.table.TableCellRenderer **1.2**

- `Component getTableCellRendererComponent(JTable table, Object value, boolean selected, boolean hasFocus, int row, int column)`
  returns a component whose `paint` method is invoked to render a table cell.

`javax.swing.table.TableColumn` **1.2**

- `void setCellEditor(TableCellEditor editor)`
- `void setCellRenderer(TableCellRenderer renderer)`
  set the cell editor or renderer for all cells in this column.
- `void setHeaderRenderer(TableCellRenderer renderer)`
  sets the cell renderer for the header cell in this column.
- `void setHeaderValue(Object value)`
  sets the value to be displayed for the header in this column.

`javax.swing.DefaultCellEditor` **1.2**

- `DefaultCellEditor(JComboBox comboBox)`
  constructs a cell editor that presents the combo box for selecting cell values.

javax.swing.table.TableCellEditor **1.2**

- `Component getTableCellEditorComponent(JTable table, Object value, boolean selected, int row, int column)`
  returns a component whose `paint` method renders a table cell.

javax.swing.CellEditor **1.2**

- `boolean isCellEditable(EventObject event)`
  returns `true` if the event is suitable for initiating the editing process for this cell.
- `boolean shouldSelectCell(EventObject anEvent)`
  starts the editing process. Returns `true` if the edited cell should be *selected*. Normally, you want to return `true`, but you can return `false` if you don't want the editing process to change the cell selection.
- `void cancelCellEditing()`
  cancels the editing process. You can abandon partial edits.
- `boolean stopCellEditing()`
  stops the editing process, with the intent of using the result. Returns `true` if the edited value is in a proper state for retrieval.
- `Object getCellEditorValue()`
  returns the edited result.
- `void addCellEditorListener(CellEditorListener l)`
- `void removeCellEditorListener(CellEditorListener l)`
  add or remove the obligatory cell editor listener.

12.4. Trees

Every computer user who has worked with a hierarchical file system has seen tree displays. Of course, directories and files are only one of the many examples of tree-like organizations. Many tree structures arise in everyday life, such as the hierarchy of countries, states, and cities shown in **Figure 12.11**.

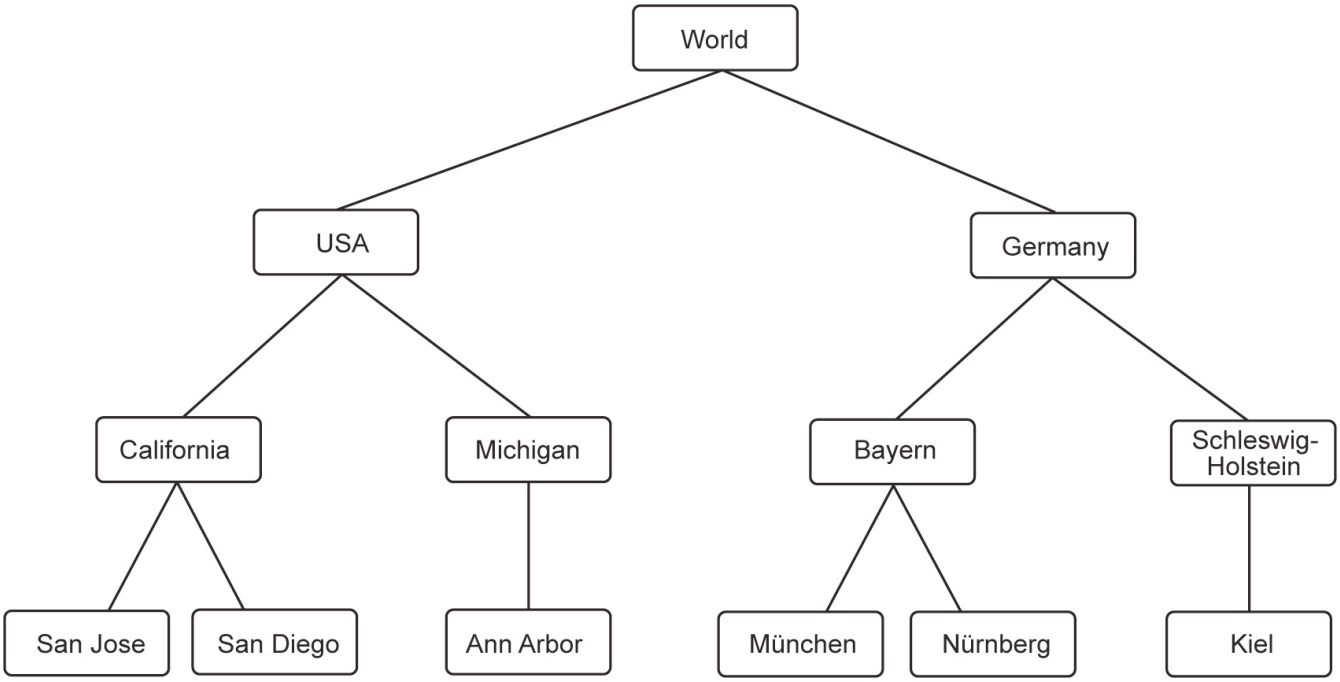


Figure 12.11: A hierarchy of countries, states, and cities

As programmers, we often need to display tree structures. Fortunately, the Swing library has a `JTree` class for this purpose. The `JTree` class (together with its helper classes) takes care of laying out the tree and processing user requests for expanding and collapsing nodes. In this section, you will learn how to put the `JTree` class to use.

As with the other complex Swing components, we must focus on the common and useful cases and cannot cover every nuance. If you want to achieve something unusual, we recommend that you consult *Graphic Java™, Third Edition*, by David M. Geary or *Core Swing* by Kim Topley.

Before going any further, let's settle on some terminology (see **Figure 12.12**). A *tree* is composed of *nodes*. Every node is either a *leaf* or it has *child nodes*. Every node, with the exception of the root node, has exactly one *parent*. A tree has exactly one root node. Sometimes you have a collection of trees, each with its own root node. Such a collection is called a *forest*.

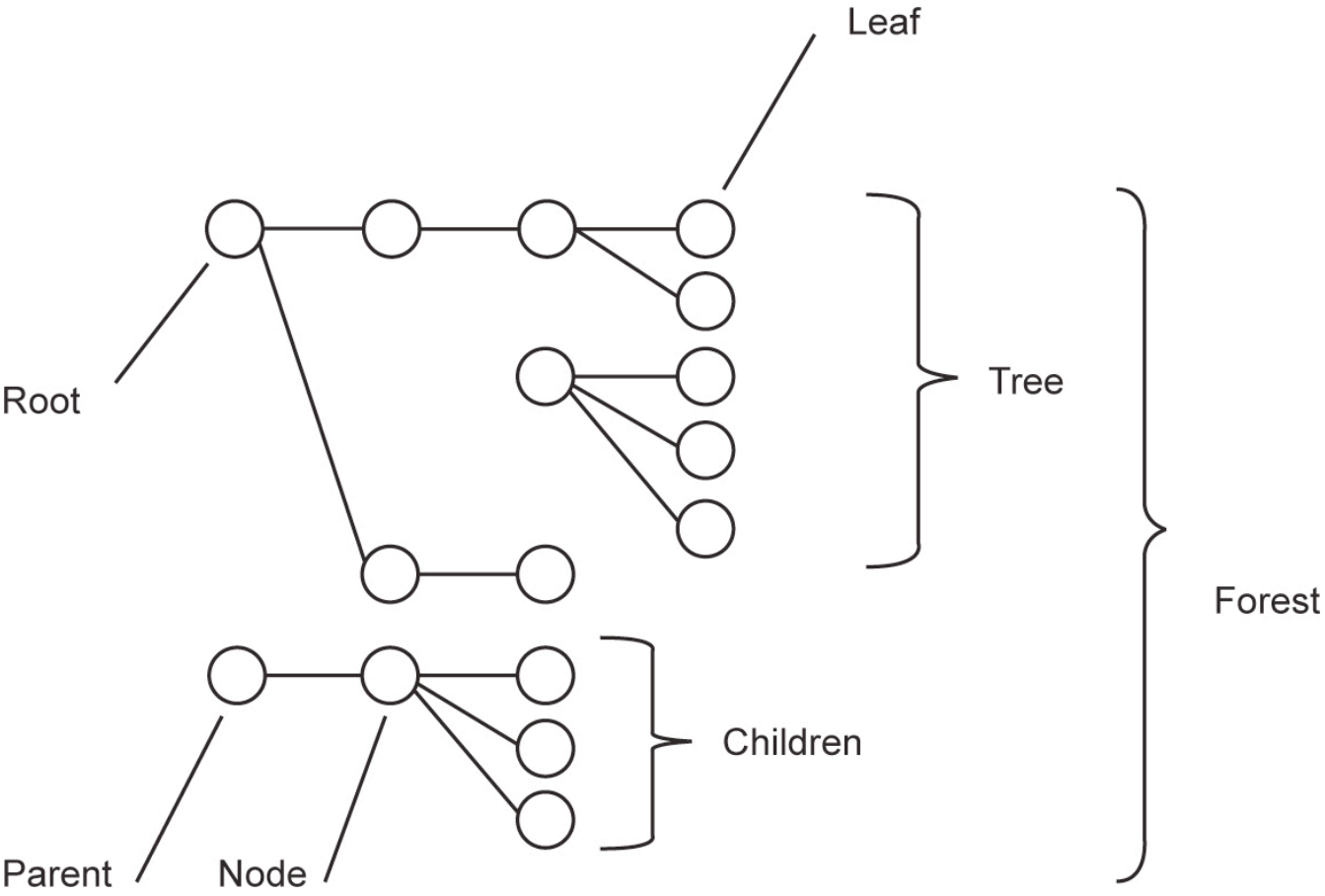


Figure 12.12: Tree terminology

12.4.1. Simple Trees

In our first example program, we will simply display a tree with a few nodes (see **Figure 12.14**). As with many other Swing components, you need to provide a model of the data, and the component displays it for you. To construct a `JTree`, supply the tree model in the constructor:

```
TreeModel model = . . .;
var tree = new JTree(model);
```



Note: There are also constructors that construct trees out of a collection of elements:

```
JTree(Object[] nodes)
JTree(Vector<?> nodes)
JTree(Hashtable<?, ?> nodes) // the values become the nodes
```

These constructors are not very useful. They merely build a forest of trees, each with a single node. The third constructor seems particularly useless because the nodes appear in the seemingly random order determined by the hash codes of the keys.

How do you obtain a tree model? You can construct your own model by creating a class that implements the `TreeModel` interface. You will see later in this chapter how to do that. For now, we will stick with the `DefaultTreeModel` that the Swing library supplies.

To construct a default tree model, you must supply a root node.

```
TreeNode root = . . .;
var model = new DefaultTreeModel(root);
```

`TreeNode` is another interface. Populate the default tree model with objects of any class that implements the interface. For now, we will use the concrete node class that Swing supplies—namely, `DefaultMutableTreeNode`. This class implements the `MutableTreeNode` interface, a subinterface of `TreeNode` (see **Figure 12.13**).

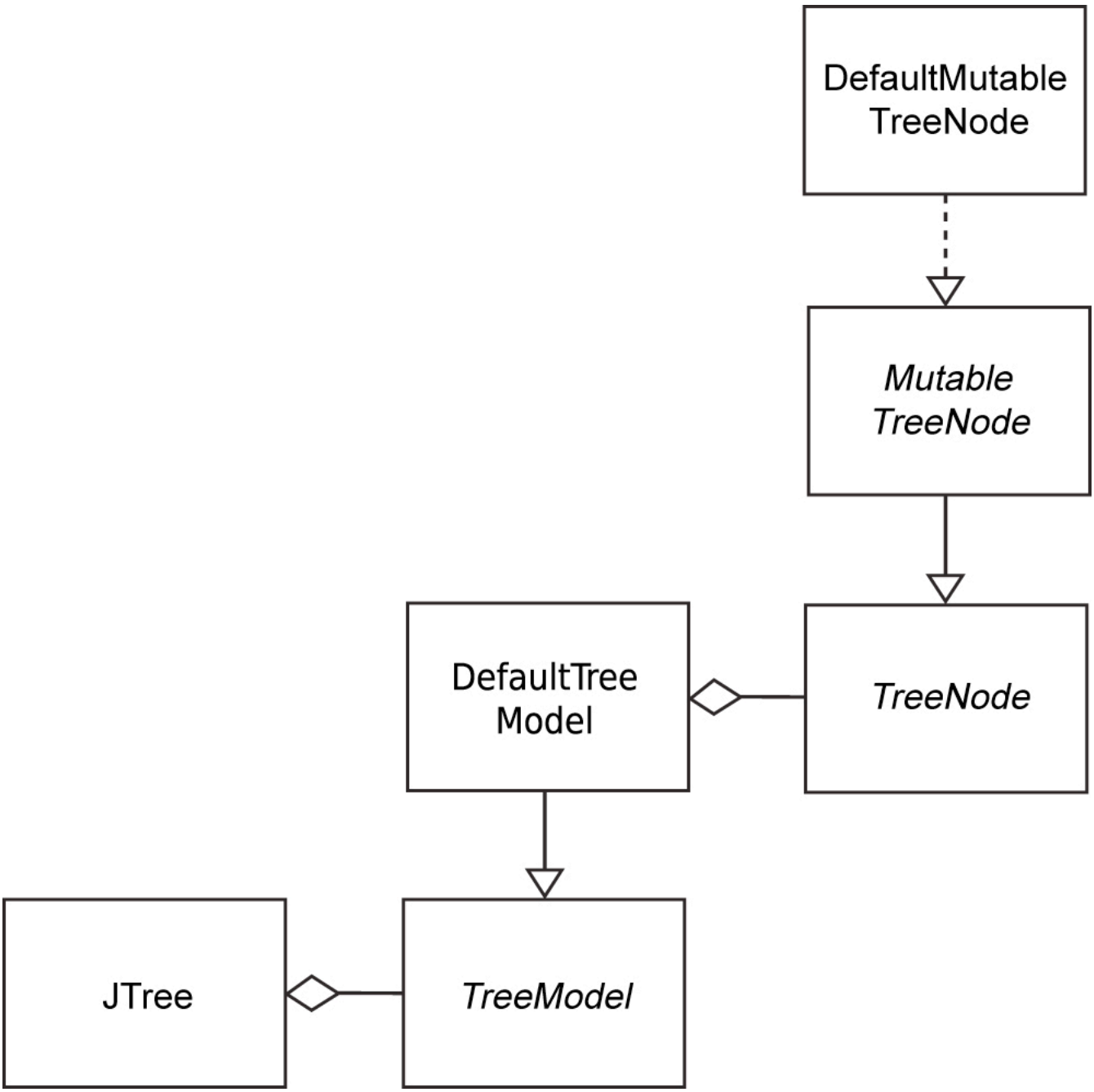


Figure 12.13: Tree classes

A default mutable tree node holds an object—the *user object*. The tree renders the user objects for all nodes. Unless you specify a renderer, the tree displays the string that is the result of the `toString` method.

In our first example, we use strings as user objects. In practice, you would usually populate a tree with more expressive user objects. For example, when displaying a directory tree, it makes sense to use `File` objects for the nodes.

You can specify the user object in the constructor, or you can set it later with the `setUserObject` method.

```
var node = new DefaultMutableTreeNode("Texas");
. . .
node.setUserObject("California");
```

Next, you need to establish the parent/child relationships between the nodes. Start with the root node and use the `add` method to add the children:

```
var root = new DefaultMutableTreeNode("World");
var country = new DefaultMutableTreeNode("USA");
root.add(country);
var state = new DefaultMutableTreeNode("California");
country.add(state);
```

Figure 12.14 illustrates how the tree will look.



Figure 12.14: A simple tree

Link up all nodes in this fashion. Then, construct a `DefaultTreeModel` with the root node. Finally, construct a `JTree` with the tree model.

```
var treeModel = new DefaultTreeModel(root);
var tree = new JTree(treeModel);
```

Or, as a shortcut, you can simply pass the root node to the `JTree` constructor. Then the tree automatically constructs a default tree model:

```
var tree = new JTree(root);
```

Listing 12.8 contains the complete code.

Listing 12.8 v2ch12/tree/SimpleTreeFrame.java

```
1  package v2ch12.tree;
2
3  import module java.desktop;
4
5  /**
6   * This frame contains a simple tree that displays a manually constructed tree model.
```

```
 */
class SimpleTreeFrame extends JFrame {
    private static final int DEFAULT_WIDTH = 300;
    private static final int DEFAULT_HEIGHT = 200;

    public SimpleTreeFrame() {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // set up tree model data

        var root = new DefaultMutableTreeNode("World");
        var country = new DefaultMutableTreeNode("USA");
        root.add(country);
        var state = new DefaultMutableTreeNode("California");
        country.add(state);
        var city = new DefaultMutableTreeNode("San Jose");
        state.add(city);
        city = new DefaultMutableTreeNode("Cupertino");
        state.add(city);
        state = new DefaultMutableTreeNode("Michigan");
        country.add(state);
        city = new DefaultMutableTreeNode("Ann Arbor");
        state.add(city);
        country = new DefaultMutableTreeNode("Germany");
        root.add(country);
        state = new DefaultMutableTreeNode("Schleswig-Holstein");
        country.add(state);
        city = new DefaultMutableTreeNode("Kiel");
        state.add(city);

        // construct tree and put it in a scroll pane

        var tree = new JTree(root);
        add(new JScrollPane(tree));
    }
}
```

When you run the program, the tree first looks as in **Figure 12.15**. Only the root node and its children are visible. Click on the circle icons (the *handles*) to open up the subtrees. The line sticking out from the handle icon points to the right when the subtree is collapsed and down when the subtree is expanded (see **Figure 12.16**). We don't know what the designers of the Metal look-and-feel had in mind, but we think of the icon as a door handle. You push down on the handle to open the subtree.



Figure 12.15: The initial tree display

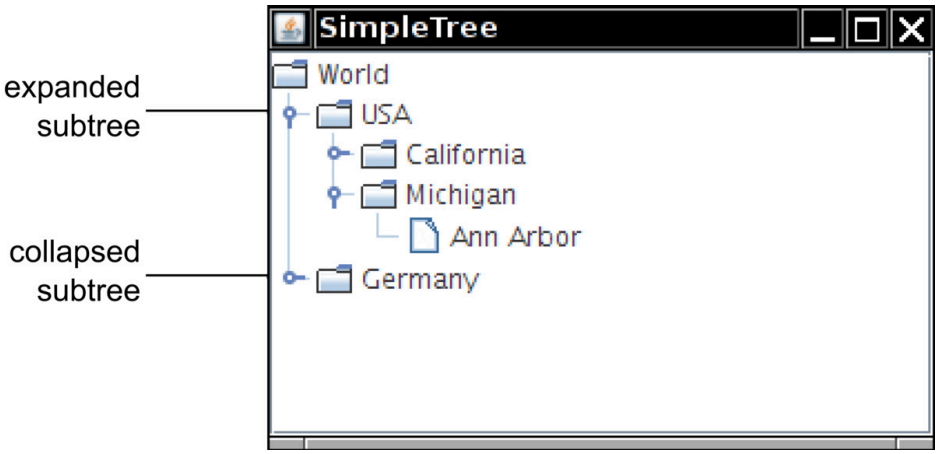


Figure 12.16: Collapsed and expanded subtrees

Note: Of course, the display of the tree depends on the selected look-and-feel. We just described the Metal look-and-feel. In the Windows look-and-feel, the handles have the more familiar look—a "`-`" or "`+`" in a box (see **Figure 12.17**).

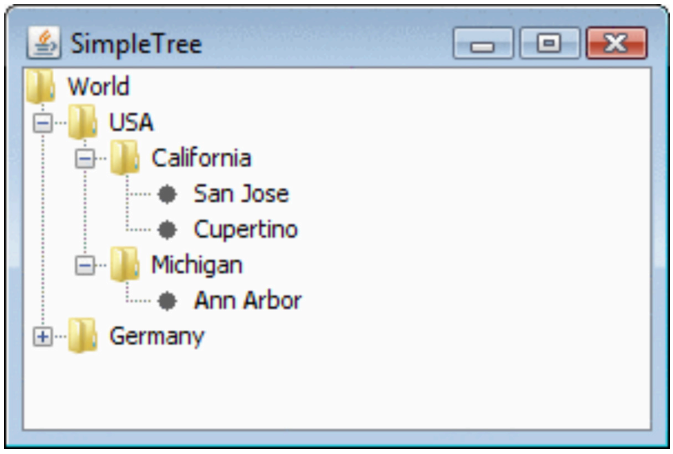


Figure 12.17: A tree with the Windows look-and-feel

You can use the following magic incantation to turn off the lines joining parents and children (see **Figure 12.18**):

```
tree.putClientProperty("JTree.lineStyle", "None");
```



Figure 12.18: A tree with no connecting lines

Conversely, to make sure that the lines are shown, use

```
tree.putClientProperty("JTree.lineStyle", "Angled");
```

Another line style, `"Horizontal"`, is shown in Figure 12.19. The tree is displayed with horizontal lines separating only the children of the root. I'm not quite sure what it is good for.

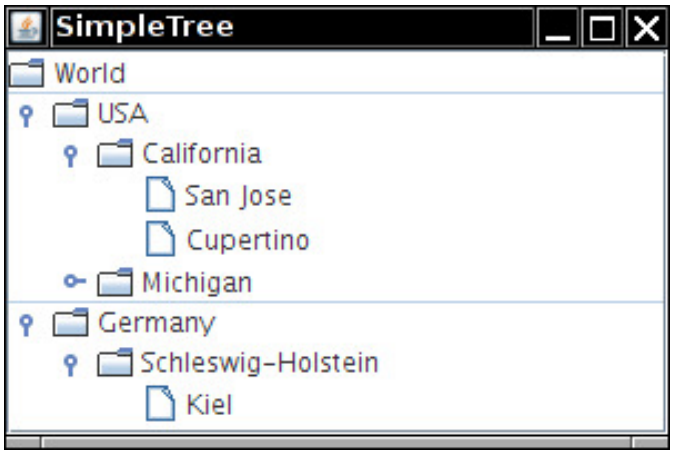


Figure 12.19: A tree with the horizontal line style

By default, there is no handle for collapsing the root of the tree. If you like, you can add one with the call

```
tree.setShowsRootHandles(true);
```

Figure 12.20 shows the result. Now you can collapse the entire tree into the root node.

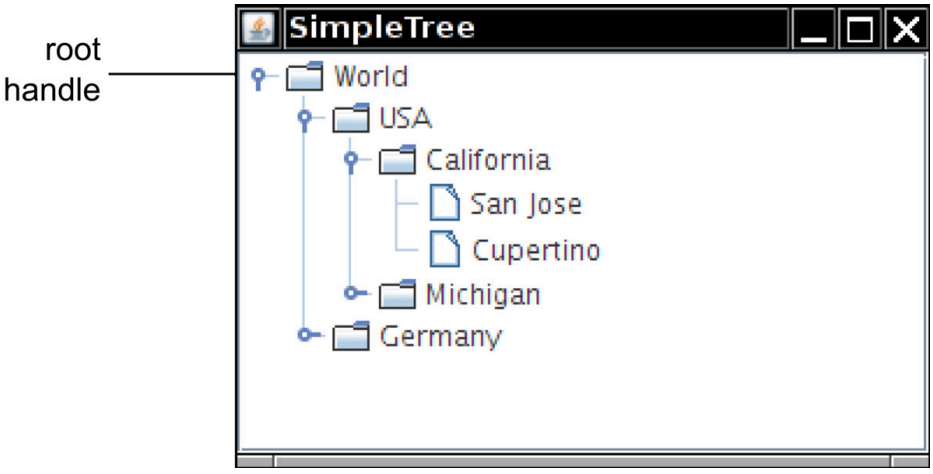


Figure 12.20: A tree with a root handle

Conversely, you can hide the root altogether. You will thus display a *forest*—a set of trees, each with its own root. You still must join all trees in the forest to a common root; then, hide the root with the instruction

```
tree.setRootVisible(false);
```

Look at Figure 12.21. There appear to be two roots, labeled “USA” and “Germany.” The actual root that joins the two is made invisible.



Figure 12.21: A forest

Let's turn from the root to the leaves of the tree. Note that the leaves have an icon different from the other nodes (see **Figure 12.22**).

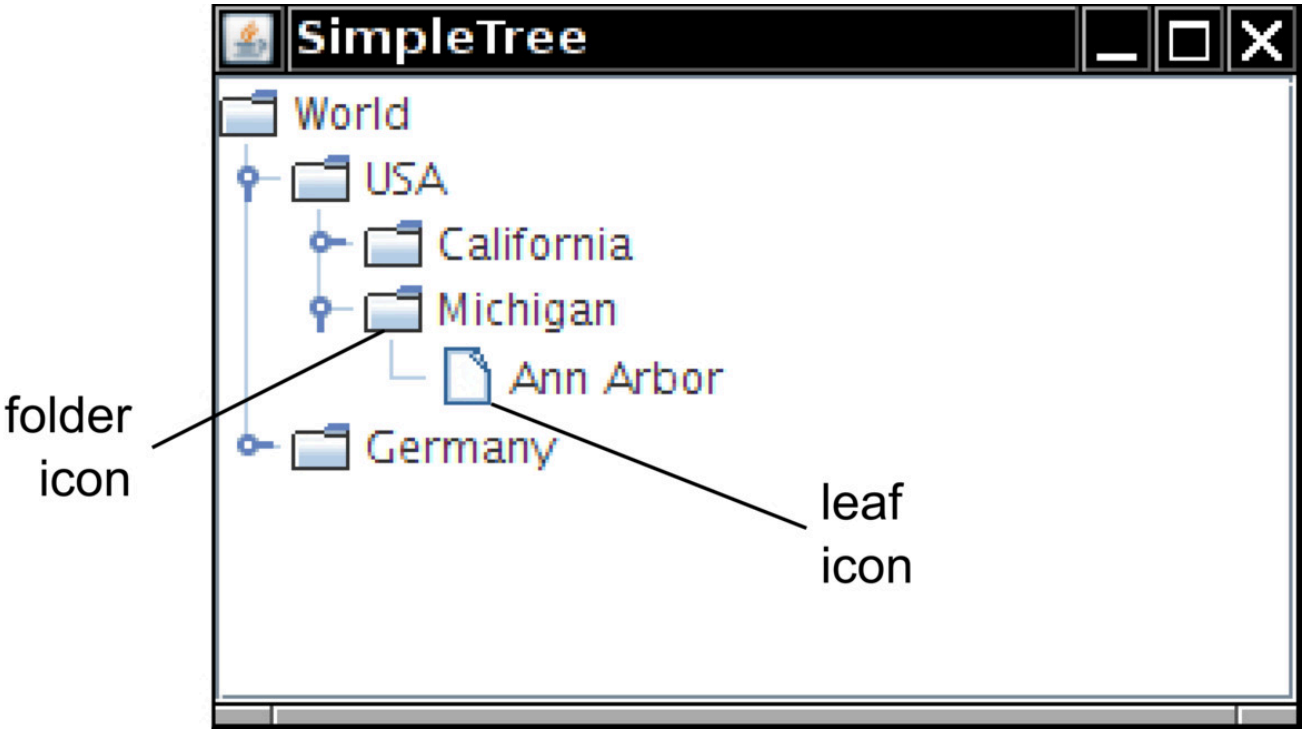


Figure 12.22: Leaf and folder icons

When the tree is displayed, each node is drawn with an icon. There are actually three kinds of icons: a leaf icon, an opened nonleaf icon, and a closed nonleaf icon. For simplicity, we refer to the last two as folder icons.

The node renderer needs to know which icon to use for each node. By default, the decision process works like this: If the `isLeaf` method of a node returns `true`, then the leaf icon is used; otherwise, a folder icon is used.

The `isLeaf` method of the `DefaultMutableTreeNode` class returns `true` if the node has no children. Thus, nodes with children get folder icons, and nodes without children get leaf icons.

Sometimes, that behavior is not appropriate. Suppose we added a node "Montana" to our sample tree, but we're at a loss as to what cities to add. We would not want the state node to get a leaf icon because, conceptually, only the cities are leaves.

The `JTree` class has no idea which nodes should be leaves. It asks the tree model. If a childless node isn't automatically a conceptual leaf, you can ask the tree model to use a different criterion for leafiness—namely, to query the "allows children" node property.

For those nodes that should not have children, call

```
node.setAllowsChildren(false);
```

Then, tell the tree model to ask the value of the “allows children” property to determine whether a node should be displayed with a leaf icon. Use the `setAsksAllowsChildren` method of the `DefaultTreeModel` class to set this behavior:

```
model.setAsksAllowsChildren(true);
```

With this decision criterion, nodes that allow children get folder icons, and nodes that don’t allow children get leaf icons.

Alternatively, if you construct the tree from the root node, supply the setting for the “asks allows children” property in the constructor.

```
var tree = new JTree(root, true); // nodes that don't allow children get leaf icons
```

`javax.swing.JTree` **1.2**

- `JTree(TreeModel model)`
  constructs a tree from a tree model.
- `JTree(TreeNode root)`
- `JTree(TreeNode root, boolean asksAllowChildren)`
  construct a tree with a default tree model that displays the root and its children.
- `void setShowsRootHandles(boolean b)`
  If `b` is `true`, the root node has a handle for collapsing or expanding its children.
- `void setRootVisible(boolean b)`
  If `b` is true, then the root node is displayed. Otherwise, it is hidden.

javax.swing.tree.TreeNode **1.2**

- `boolean isLeaf()`
  returns `true` if this node is conceptually a leaf.
- `boolean getAllowsChildren()`
  returns `true` if this node can have child nodes.

javax.swing.tree.MutableTreeNode **1.2**

- `void setUserObject(Object userObject)`
  sets the “user object” that the tree node uses for rendering.

javax.swing.tree.TreeModel **1.2**

- `boolean isLeaf(Object node)`
  returns `true` if `node` should be displayed as a leaf node.

`javax.swing.tree.DefaultTreeModel` 1.2

- `void setAsksAllowsChildren(boolean b)`
  If `b` is `true`, nodes are displayed as leaves when their `getAllowsChildren` method returns `false`. Otherwise, they are displayed as leaves when their `isLeaf` method returns `true`.

`javax.swing.tree.DefaultMutableTreeNode` 1.2

- `DefaultMutableTreeNode(Object userObject)`
  constructs a mutable tree node with the given user object.
- `void add(MutableTreeNode child)`
  adds a node as the last child of this node.
- `void setAllowsChildren(boolean b)`
  If `b` is `true`, children can be added to this node.

`javax.swing.JComponent` 1.2

- `void putClientProperty(Object key, Object value)`
  adds a key/value pair to a small table that each component manages. This is an "escape hatch" mechanism that some Swing components use for storing properties specific to a look-and-feel.

12.4.2. Editing Trees and Tree Paths

In the next example program, you will see how to edit a tree. **Figure 12.23** shows the user interface. If you click the Add Sibling or Add Child button, the program adds a new node (with title New) to the tree. If you click the Delete button, the program deletes the currently selected node.

Figure 12.23: Editing a tree

To implement this behavior, you need to find out which tree node is currently selected. The `JTree` class has a surprising way of identifying nodes in a tree. It does not deal with tree nodes but with *paths of objects*, called *tree paths*. A tree path starts at the root and consists of a sequence of child nodes (see **Figure 12.24**).

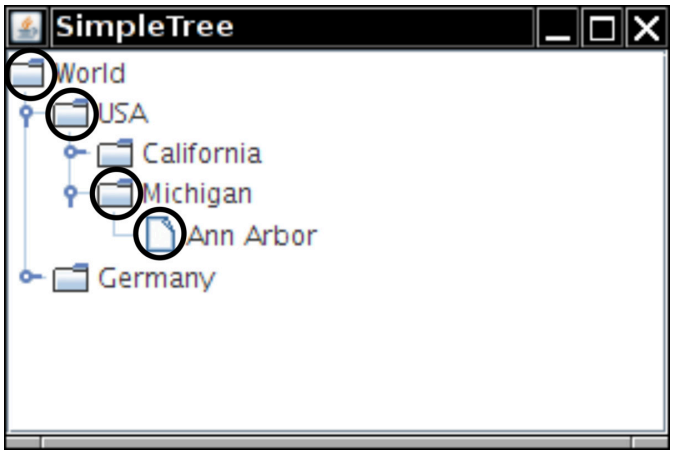


Figure 12.24: A tree path

You might wonder why the `JTree` class needs the whole path. Couldn't it just get a `TreeNode` and keep calling the `getParent` method? In fact, the `JTree` class knows nothing about the `TreeNode` interface. That interface is never used by the `TreeModel` interface; it is only used by the `DefaultTreeModel` implementation. You can have other tree models in which the nodes do not implement the `TreeNode` interface at all. If you use a tree model that manages other types of objects, those objects might not have `getParent` and `getChild` methods. They would of course need to have some other connection to each other. It is the job of the tree model to link nodes together. The `JTree` class itself has no clue about the nature of their linkage. For that reason, the `JTree` class always needs to work with complete paths.

The `TreePath` class manages a sequence of `Object` (not `TreeNode`!) references. A number of `JTree` methods return `TreePath` objects. When you have a tree path, you usually just need to know the terminal node, which you can get with the `getLastPathComponent` method. For example, to find out the currently selected node in a tree, use the `getSelectionPath` method of the `JTree` class. You will get a `TreePath` object back, from which you can retrieve the actual node.

```
TreePath selectionPath = tree.getSelectionPath();
var selectedNode = (DefaultMutableTreeNode) selectionPath.getLastPathComponent();
```

Actually, since this particular query is so common, there is a convenience method that gives the selected node immediately:

```
var selectedNode = (DefaultMutableTreeNode) tree.getLastSelectedPathComponent();
```

This method is not called `getSelectedNode` because the tree does not know that it contains nodes—its tree model deals only with paths of objects.

Note: Tree paths are one of the two ways in which the `JTree` class describes nodes. Quite a few `JTree` methods take or return an integer index—the *row position*. A row position is simply the row number (starting with 0) of the node in the tree display. Only visible nodes have row numbers, and the row number of a node changes if other nodes before it are expanded, collapsed, or modified. For that reason, you should avoid row positions. All `JTree` methods that use row positions have equivalents that use tree paths instead.

Once you have the selected node, you can edit it. However, do not simply add children to a tree node:

```
selectedNode.add(newNode); // No!
```

If you change the structure of the nodes, you change the model but the associated view is not notified. You could send out a notification yourself, but if you use the `insertNodeInto` method of the `DefaultTreeModel` class, the model class takes care of that. For example, the following call appends a new node as the last child of the selected node and notifies the tree view:

```
model.insertNodeInto(newNode, selectedNode, selectedNode.getChildCount());
```

The analogous call `removeNodeFromParent` removes a node and notifies the view:

```
model.removeNodeFromParent(selectedNode);
```

If you keep the node structure in place but change the user object, you should call the following method:

```
model.nodeChanged(changedNode);
```

The automatic notification is a major advantage of using the `DefaultTreeModel`. If you supply your own tree model, you have to implement automatic notification by hand. (See *Core Swing* by Kim Topley for details.)



Caution: The `DefaultTreeModel` class has a `reload` method that reloads the entire model. However, don't call `reload` simply to update the tree after making a few changes. When the tree is regenerated, all nodes beyond the root's children are collapsed again. It will be quite disconcerting to your users if they have to keep expanding the tree after every change.

When the view is notified of a change in the node structure, it updates the display but does not automatically expand a node to show newly added children. In particular, if a user in our sample program adds a new child node to a node for which children are currently collapsed, the new node is silently added to the collapsed subtree. This gives the user no feedback that the command was actually carried out. In such a case, you should make a special effort to expand all parent nodes so that the newly added node becomes visible. Use the `makeVisible` method of the `JTree` class for this purpose. The `makeVisible` method expects a tree path leading to the node that should become visible.

Thus, you need to construct a tree path from the root to the newly inserted node. To get a tree path, first call the `getPathToRoot` method of the `DefaultTreeModel` class. It returns a `TreeNode[]` array of all nodes from a node to the root node. Pass that array to a `TreePath` constructor.

For example, here is how you make the new node visible:

```
TreeNode[] nodes = model.getPathToRoot(newNode);
var path = new TreePath(nodes);
tree.makeVisible(path);
```



Note: It is curious that the `DefaultTreeModel` class feigns almost complete ignorance of the `TreePath` class, even though its job is to communicate with a `JTree`. The `JTree` class uses tree paths a lot, and it never uses arrays of node objects.

Now, suppose your tree is contained inside a scroll pane. After the tree node expansion, the new node might still not be visible because it falls outside the viewport. To overcome that problem, call

```
tree.scrollPathToVisible(path);
```

instead of calling `makeVisible`. This call expands all nodes along the path and tells the ambient scroll pane to scroll the node at the end of the path into view (see **Figure 12.25**).

Figure 12.25: The scroll pane scrolls to display a new node.

By default, tree nodes cannot be edited. However, if you call

```
tree.setEditable(true);
```

the user can edit a node simply by double-clicking, editing the string, and pressing the Enter key. Double-clicking invokes the *default cell editor*, which is implemented by the `DefaultCellEditor` class (see **Figure 12.26**). It is possible to install other cell editors, using the same process that you have seen in our discussion of table cell editors.

Figure 12.26: The default cell editor

Listing 12.9 shows the complete source code of the tree editing program. Run the program, add a few nodes, and edit them by double-clicking. Observe how collapsed nodes expand to show added children and how the scroll pane keeps added nodes in the viewport.

Listing 12.9 v2ch12/treeEdit/TreeEditFrame.java

```
1  package v2ch12.treeEdit;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a tree and buttons to edit the tree.
7   */
8  public class TreeEditFrame extends JFrame {
9      private static final int DEFAULT_WIDTH = 400;
10     private static final int DEFAULT_HEIGHT = 200;
```

```
11
12      private DefaultTreeModel model;
13      private JTree tree;
14
15      public TreeEditFrame() {
16          setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
17
18          // construct tree
19
20          TreeNode root = makeSampleTree();
21          model = new DefaultTreeModel(root);
22          tree = new JTree(model);
23          tree.setEditable(true);
24
25          // add scroll pane with tree
26
27          var scrollPane = new JScrollPane(tree);
28          add(scrollPane, BorderLayout.CENTER);
29
30          makeButtons();
31      }
32
33      public TreeNode makeSampleTree() {
34          var root = new DefaultMutableTreeNode("World");
35          var country = new DefaultMutableTreeNode("USA");
36          root.add(country);
37          var state = new DefaultMutableTreeNode("California");
38          country.add(state);
39          var city = new DefaultMutableTreeNode("San Jose");
40          state.add(city);
41          city = new DefaultMutableTreeNode("San Diego");
42          state.add(city);
43          state = new DefaultMutableTreeNode("Michigan");
44          country.add(state);
45          city = new DefaultMutableTreeNode("Ann Arbor");
46          state.add(city);
47          country = new DefaultMutableTreeNode("Germany");
48          root.add(country);
49          state = new DefaultMutableTreeNode("Schleswig-Holstein");
50          country.add(state);
51          city = new DefaultMutableTreeNode("Kiel");
52          state.add(city);
53          return root;
54      }
55
56      /**
57       * Makes the buttons to add a sibling, add a child, and delete a node.
58       */
59      public void makeButtons() {
60          var panel = new JPanel();
61          var addSiblingButton = new JButton("Add Sibling");
62          addSiblingButton.addActionListener(_ -> {
63              var selectedNode = (DefaultMutableTreeNode) tree.getLastSelectedPathComponent();
64
65              if (selectedNode == null) return;
66
67              var parent = (DefaultMutableTreeNode) selectedNode.getParent();
68
```

```
            if (parent == null) return;

            var newNode = new DefaultMutableTreeNode("New");

            int selectedIndex = parent.getIndex(selectedNode);
            model.insertNodeInto(newNode, parent, selectedIndex + 1);

            // now display new node

            TreeNode[] nodes = model.getPathToRoot(newNode);
            var path = new TreePath(nodes);
            tree.scrollPathToVisible(path);
        });
        panel.add(addSiblingButton);

        var addChildButton = new JButton("Add Child");
        addChildButton.addActionListener(_ -> {
            var selectedNode = (DefaultMutableTreeNode) tree.getLastSelectedPathComponent();

            if (selectedNode == null) return;

            var newNode = new DefaultMutableTreeNode("New");
            model.insertNodeInto(newNode, selectedNode, selectedNode.getChildCount());

            // now display new node

            TreeNode[] nodes = model.getPathToRoot(newNode);
            var path = new TreePath(nodes);
            tree.scrollPathToVisible(path);
        });
        panel.add(addChildButton);

        var deleteButton = new JButton("Delete");
        deleteButton.addActionListener(_ -> {
            var selectedNode = (DefaultMutableTreeNode) tree.getLastSelectedPathComponent();

            if (selectedNode != null && selectedNode.getParent() != null)
                model.removeNodeFromParent(selectedNode);
        });
        panel.add(deleteButton);
        add(panel, BorderLayout.SOUTH);
    }
}
```

`javax.swing.JTree` 1.2

- `TreePath getSelectionPath()`
  gets the path to the currently selected node, or the path to the first selected node if multiple nodes are selected. Returns `null` if no node is selected.
- `Object getLastSelectedPathComponent()`
  gets the node object that represents the currently selected node, or the first node if multiple nodes are selected. Returns `null` if no node is selected.
- `void makeVisible(TreePath path)`
  expands all nodes along the path.

- `void scrollPathToVisible(TreePath path)`
  expands all nodes along the path and, if the tree is contained in a scroll pane, scrolls to ensure that the last node on the path is visible.

`javax.swing.tree.TreePath` 1.2

- `Object getLastPathComponent()`
  gets the last object on this path—that is, the node object that the path represents.

javax.swing.tree.TreeNode 1.2

- `TreeNode getParent()`
  returns the parent node of this node.
- `TreeNode getChildAt(int index)`
  looks up the child node at the given index. The index must be between 0 and `getChildCount()` – 1.
- `int getChildCount()`
  returns the number of children of this node.
- `Enumeration children()`
  returns an enumeration object that iterates through all children of this node.

`javax.swing.tree.DefaultTreeModel` 1.2

- `void insertNodeInto(MutableTreeNode newChild, MutableTreeNode parent, int index)`
  inserts `newChild` as a new child node of `parent` at the given index and notifies the tree model listeners.
- `void removeNodeFromParent(MutableTreeNode node)`
  removes `node` from this model and notifies the tree model listeners.
- `void nodeChanged(TreeNode node)`
  notifies the tree model listeners that `node` has changed.
- `void nodesChanged(TreeNode parent, int[] changedChildIndexes)`
  notifies the tree model listeners that all child nodes of `parent` with the given indexes have changed.
- `void reload()`
  reloads all nodes into the model. This is a drastic operation that you should use only if the nodes have changed completely because of some outside influence.

12.4.3. Node Enumeration

Sometimes you need to find a node in a tree by starting at the root and visiting all children until you have found a match. The `DefaultMutableTreeNode` class has several convenience methods for iterating through nodes.

The `breadthFirstEnumeration` and `depthFirstEnumeration` methods return enumeration objects whose `nextElement` method visits all children of the current node, using either a breadth-first or depth-first traversal. Figure 12.27 shows the traversals for a sample tree—the node labels indicate the order in which the nodes are traversed.

Figure 12.27: Tree traversal orders

Breadth-first enumeration is the easiest to visualize. The tree is traversed in layers. The root is visited first, followed by all of its children, then the grandchildren, and so on.

To visualize depth-first enumeration, imagine a rat trapped in a tree-shaped maze. It rushes along the first path until it comes to a leaf. Then, it backtracks and turns around to the next path, and so on.

Computer scientists also call this *postorder traversal* because the search process visits the children before visiting the parents. The `postOrderEnumeration` method is a synonym for `depthFirstEnumeration`. For completeness, there is also a `preOrderEnumeration`, a depth-first search that enumerates parents before the children.

Here is the typical usage pattern:

```
Enumeration breadthFirst = node.breadthFirstEnumeration();
while (breadthFirst.hasMoreElements())
    do something with breadthFirst.nextElement();
```

Finally, a related method, `pathFromAncestorEnumeration`, finds a path from an ancestor to a given node and enumerates the nodes along that path. That's no big deal—it just keeps calling `getParent` until the ancestor is found and then presents the path in reverse order.

In our next example program, we put node enumeration to work. The program displays inheritance trees of classes. Type the name of a class into the text field on the bottom of the frame. The class and all of its superclasses are added to the tree (see **Figure 12.28**).

Figure 12.28: An inheritance tree

In this example, we take advantage of the fact that the user objects of the tree nodes can be objects of any type. Since our nodes describe classes, we store `Class` objects in the nodes.

We don't want to add the same class object twice, so we need to check whether a class already exists in the tree. The following method finds the node with a given user object if it exists in the tree:

```
public DefaultMutableTreeNode findUserObject(Object obj) {
    Enumeration e = root.breadthFirstEnumeration();
    while (e.hasMoreElements()) {
        DefaultMutableTreeNode node = (DefaultMutableTreeNode) e.nextElement();
        if (node.getUserObject().equals(obj))
            return node;
    }
    return null;
}
```

12.4.4. Rendering Nodes

In your applications, you will often need to change the way a tree component draws the nodes. The most common change is, of course, to choose different icons for nodes and leaves. Other changes might involve changing the font of the node labels or drawing images at the nodes. All these changes are possible by installing a new *tree cell renderer* into the tree. By default, the `JTree` class uses `DefaultTreeCellRenderer` objects to draw each node. The `DefaultTreeCellRenderer` class extends the `JLabel` class. The label contains the node icon and the node label.



Note: The cell renderer does not draw the "handles" for expanding and collapsing subtrees. The handles are part of the look-and-feel, and it is recommended that you do not change them.

You can customize the display in three ways.

- You can change the icons, font, and background color used by a `DefaultTreeCellRenderer`. These settings are used for all nodes in the tree.
- You can install a renderer that extends the `DefaultTreeCellRenderer` class and vary the icons, fonts, and background color for each node.

- You can install a renderer that implements the `TreeCellRenderer` interface to draw a custom image for each node.

Let us look at these possibilities one by one. The easiest customization is to construct a `DefaultTreeCellRenderer` object, change the icons, and install it into the tree:

```
var renderer = new DefaultTreeCellRenderer();
renderer.setLeafIcon(new ImageIcon("blue-ball.gif")); // used for leaf nodes
renderer.setClosedIcon(new ImageIcon("red-ball.gif")); // used for collapsed nodes
renderer.setOpenIcon(new ImageIcon("yellow-ball.gif")); // used for expanded nodes
tree.setCellRenderer(renderer);
```

You can see the effect in **Figure 12.28**. We just use the "ball" icons as placeholders—presumably your user interface designer would supply you with appropriate icons to use for your applications.

We don't recommend that you change the font or background color for an entire tree—that is really the job of the look-and-feel.

However, it can be useful to change the font of some nodes in a tree to highlight them. If you look carefully at **Figure 12.28**, you will notice that the *abstract* classes are set in italics.

To change the appearance of individual nodes, install a tree cell renderer. Tree cell renderers are very similar to the list cell renderers we discussed earlier in this chapter. The `TreeCellRenderer` interface has a single method:

```
Component getTreeCellRendererComponent(JTree tree, Object value, boolean selected,
        boolean expanded, boolean leaf, int row, boolean hasFocus)
```

The `getTreeCellRendererComponent` method of the `DefaultTreeCellRenderer` class returns `this`—in other words, a label. (The `DefaultTreeCellRenderer` class extends the `JLabel` class.) To customize the component, extend the `DefaultTreeCellRenderer` class. Override the `getTreeCellRendererComponent` method as follows: Call the superclass method so it can prepare the label data, customize the label properties, and finally return `this`.

```
class MyTreeCellRenderer extends DefaultTreeCellRenderer {
    public Component getTreeCellRendererComponent(JTree tree, Object value,
            boolean selected, boolean expanded, boolean leaf, int row, boolean hasFocus) {
        Component comp = super.getTreeCellRendererComponent(tree, value, selected,
            expanded, leaf, row, hasFocus);
        DefaultMutableTreeNode node = (DefaultMutableTreeNode) value;
        look at node.getUserObject();
        Font font = appropriate font;
        comp.setFont(font);
        return comp;
    }
}
```

Caution: The `value` parameter of the `getTreeCellRendererComponent` method is the *node* object, *not* the user object! Recall that the user object is a feature of the `DefaultMutableTreeNode`, and that a

`JTree` can contain nodes of an arbitrary type. If your tree uses `DefaultMutableTreeNode` nodes, you must retrieve the user object in a second step, as we did in the preceding code sample.



Caution: The `DefaultTreeCellRenderer` uses the *same* label object for all nodes, only changing the label text for each node. If you change the font for a particular node, you must set it back to its default value when the method is called again. Otherwise, all subsequent nodes will be drawn in the changed font! Look at the code in **Listing 12.10** to see how to restore the font to the default.

The `ClassNameTreeCellRenderer` in **Listing 12.10** sets the class name in either the normal or italic font, depending on the `ABSTRACT` modifier of the `Class` object. We don't want to set a particular font because we don't want to change whatever font the look-and-feel normally uses for labels. For that reason, we use the font from the label and *derive* an italic font from it. Recall that only a single shared `JLabel` object is returned by all calls. We need to hang on to the original font and restore it in the next call to the `getTreeCellRendererComponent` method.

Also, note how we change the node icons in the `ClassTreeFrame` constructor.

`javax.swing.tree.DefaultMutableTreeNode` 1.2

- `Enumeration breadthFirstEnumeration()`
- `Enumeration depthFirstEnumeration()`
- `Enumeration preOrderEnumeration()`
- `Enumeration postOrderEnumeration()`

  return enumeration objects for visiting all nodes of the tree model in a particular order. In breadth-first traversal, children that are closer to the root are visited before those that are farther away. In depth-first traversal, all children of a node are completely enumerated before its siblings are visited. The `postOrderEnumeration` method is a synonym for `depthFirstEnumeration`. The preorder traversal is identical to the postorder traversal except that parents are enumerated before their children.

javax.swing.tree.TreeCellRenderer 1.2

- `Component getTreeCellRendererComponent(JTree tree, Object value, boolean selected, boolean expanded, boolean leaf, int row, boolean hasFocus)`

  returns a component whose `paint` method is invoked to render a tree cell.

`javax.swing.tree.DefaultTreeCellRenderer` 1.2

- `void setLeafIcon(Icon icon)`
- `void setOpenIcon(Icon icon)`
- `void setClosedIcon(Icon icon)`

  set the icon to show for a leaf node, an expanded node, and a collapsed node.

12.4.5. Listening to Tree Events

Most commonly, a tree component is paired with some other component. When the user selects tree nodes, some information shows up in another window. See **Figure 12.29** for an example. When the user selects a class, the instance and static variables of that class are displayed in the text area to the right.

Figure 12.29: A class browser

To obtain this behavior, you need to install a *tree selection listener*. The listener must implement the `TreeSelectionListener` interface—an interface with a single method:

```
void valueChanged(TreeSelectionEvent event)
```

That method is called whenever the user selects or deselects tree nodes.

Add the listener to the tree in the normal way:

```
tree.addTreeSelectionListener(listener);
```

You can specify whether the user is allowed to select a single node, a contiguous range of nodes, or an arbitrary, potentially discontiguous, set of nodes. The `JTree` class uses a `TreeSelectionModel` to manage node selection. You need to retrieve the model to set the selection state to one of `SINGLE_TREE_SELECTION`, `CONTIGUOUS_TREE_SELECTION`, or `DISCONTIGUOUS_TREE_SELECTION`. (Discontiguous selection mode is the default.) For example, in our class browser, we want to allow selection of only a single class:

```
int mode = TreeSelectionModel.SINGLE_TREE_SELECTION;
tree.getSelectionModel().setSelectionMode(mode);
```

Apart from setting the selection mode, you need not worry about the tree selection model.



Note: How the user selects multiple items depends on the look-and-feel. In the Metal look-and-feel, hold down the Ctrl key while clicking an item to add it to the selection, or to remove it if it is currently selected. Hold down the Shift key while clicking an item to select a *range* of items, extending from the previously selected item to the new item.

To find out the current selection, query the tree with the `getSelectionPaths` method:

```
TreePath[] selectedPaths = tree.getSelectionPaths();
```

If you restricted the user to single-item selection, you can use the convenience method `getSelectionPath` which returns the first selected path or `null` if no path was selected.



Caution: The `TreeSelectionEvent` class has a `getPaths` method that returns an array of `TreePath` objects, but that array describes *selection changes*, not the current selection.

Listing 12.10 shows the frame class for the class tree program. The program displays inheritance hierarchies and customizes the display to show abstract classes in italics. (See **Listing 12.11** for the cell renderer.) You can type the name of any class into the text field at the bottom of the frame. Press the Enter key or click the Add button to add the class and its superclasses to the tree. You must enter the full package name, such as `java.util.ArrayList`.

This program is a bit tricky because it uses reflection to construct the class tree. This work is done inside the `addClass` method. (The details are not that important. We use the class tree in this example because inheritance yields a nice supply of trees without laborious coding. When you display trees in your applications, you will have your own source of hierarchical data.) The method uses the breadth-first search algorithm to find whether the current class is already in the tree by calling the `findUserObject` method that we implemented in the preceding section. If the class is not already in the tree, we add the superclasses to the tree, then make the new class node a child and make that node visible.

When you select a tree node, the text area to the right is filled with the fields of the selected class. In the frame constructor, we restrict the user to single-item selection and add a tree selection listener. When the `valueChanged` method is called, we ignore its event parameter and simply ask the tree for the current selection path. As always, we need to get the last node of the path and look up its user object. We then call the `getFieldDescription` method which uses reflection to assemble a string with all fields of the selected class.

Listing 12.10 v2ch12/treeRender/ClassTreeFrame.java

```
1  package v2ch12.treeRender;
2
3  import module java.base;
4  import module java.desktop;
5
6  /**
7   * This frame displays the class tree, a text field, and an "Add" button to add more classes
8   * into the tree.
9   */
10 public class ClassTreeFrame extends JFrame {
11     private static final int DEFAULT_WIDTH = 400;
12     private static final int DEFAULT_HEIGHT = 300;
13
14     private DefaultMutableTreeNode root;
15     private DefaultTreeModel model;
16     private JTree tree;
17     private JTextField textField;
18     private JTextArea textArea;
19
20     public ClassTreeFrame() {
21         setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
22
23         // the root of the class tree is Object
24         root = new DefaultMutableTreeNode(java.lang.Object.class);
25         model = new DefaultTreeModel(root);
26         tree = new JTree(model);
```

```
27
28          // add this class to populate the tree with some data
29          addClass(getClass());
30
31          // set up node icons
32          var renderer = new ClassNameTreeCellRenderer();
33          renderer.setClosedIcon(new ImageIcon("images/red-ball.gif"));
34          renderer.setOpenIcon(new ImageIcon("images/yellow-ball.gif"));
35          renderer.setLeafIcon(new ImageIcon("images/blue-ball.gif"));
36          tree.setCellRenderer(renderer);
37
38          // set up selection mode
39          tree.addTreeSelectionListener(_ -> {
40              // the user selected a different node--update description
41              TreePath path = tree.getSelectionPath();
42              if (path == null) return;
43              var selectedNode = (DefaultMutableTreeNode) path.getLastPathComponent();
44              Class<?> c = (Class<?>) selectedNode.getUserObject();
45              String description = getFieldDescription(c);
46              textArea.setText(description);
47          });
48          int mode = TreeSelectionModel.SINGLE_TREE_SELECTION;
49          tree.getSelectionModel().setSelectionMode(mode);
50
51          // this text area holds the class description
52          textArea = new JTextArea();
53
54          // add tree and text area
55          var panel = new JPanel();
56          panel.setLayout(new GridLayout(1, 2));
57          panel.add(new JScrollPane(tree));
58          panel.add(new JScrollPane(textArea));
59
60          add(panel, BorderLayout.CENTER);
61
62          addTextField();
63      }
64
65      /**
66       * Add the text field and "Add" button to add a new class.
67       */
68      public void addTextField() {
69          var panel = new JPanel();
70
71          ActionListener addListener = _ -> {
72              // add the class whose name is in the text field
73              try {
74                  String text = textField.getText();
75                  addClass(Class.forName(text)); // clear text field to indicate success
76                  textField.setText("");
77              }
78              catch (ClassNotFoundException e) {
79                  JOptionPane.showMessageDialog(null, "Class not found");
80              }
81          };
82
83          // new class names are typed into this text field
84          textField = new JTextField(20);
```

```
        textField.addActionListener(addListener);
        panel.add(textField);

        var addButton = new JButton("Add");
        addButton.addActionListener(addListener);
        panel.add(addButton);

        add(panel, BorderLayout.SOUTH);
    }

    /**
     * Finds an object in the tree.
     * @param obj the object to find
     * @return the node containing the object or null if the object is not present in the tree
     */
    public DefaultMutableTreeNode findUserObject(Object obj) {
        // find the node containing a user object
        var e = (Enumeration<TreeNode>) root.breadthFirstEnumeration();
        while (e.hasMoreElements()) {
            var node = (DefaultMutableTreeNode) e.nextElement();
            if (node.getUserObject().equals(obj)) return node;
        }
        return null;
    }

    /**
     * Adds a new class and any parent classes that aren't yet part of the tree.
     * @param c the class to add
     * @return the newly added node
     */
    public DefaultMutableTreeNode addClass(Class<?> c) {
        // add a new class to the tree

        // skip non-class types
        if (c.isInterface() || c.isPrimitive()) return null;

        // if the class is already in the tree, return its node
        DefaultMutableTreeNode node = findUserObject(c);
        if (node != null) return node;

        // class isn't present--first add class parent recursively

        Class<?> s = c.getSuperclass();

        DefaultMutableTreeNode parent;
        if (s == null)
            parent = root;
        else
            parent = addClass(s);

        // add the class as a child to the parent
        var newNode = new DefaultMutableTreeNode(c);
        model.insertNodeInto(newNode, parent, parent.getChildCount());

        // make node visible
        var path = new TreePath(model.getPathToRoot(newNode));
        tree.makeVisible(path);

```

```
        return newNode;
    }

    /**
     * Returns a description of the fields of a class.
     * @param c the class to be described
     * @return a string containing all field types and names
     */
    public static String getFieldDescription(Class<?> c) {
        // use reflection to find types and names of fields
        var r = new StringBuilder();
        Field[] fields = c.getDeclaredFields();
        for (int i = 0; i < fields.length; i++) {
            Field f = fields[i];
            if (Modifier.isStatic(f.getModifiers())) r.append("static ");
            r.append(f.getType().getName());
            r.append(" ");
            r.append(f.getName());
            r.append("\n");
        }
        return r.toString();
    }
}
```

Listing 12.11 v2ch12/treeRender/ClassNameTreeCellRenderer.java

```
package v2ch12.treeRender;

import module java.base;
import module java.desktop;

/**
 * This class renders a class name either in plain or italic. Abstract classes are italic.
 */
public class ClassNameTreeCellRenderer extends DefaultTreeCellRenderer {
    private Font plainFont = null;
    private Font italicFont = null;

    public Component getTreeCellRendererComponent(JTree tree, Object value, boolean selected,
            boolean expanded, boolean leaf, int row, boolean hasFocus) {
        super.getTreeCellRendererComponent(tree, value, selected, expanded, leaf, row,
                hasFocus);
        // get the user object
        var node = (DefaultMutableTreeNode) value;
        Class<?> c = (Class<?>) node.getUserObject();

        // the first time, derive italic font from plain font
        if (plainFont == null) {
            plainFont = getFont();
            // the tree cell renderer is sometimes called with a label that has a null font
            if (plainFont != null) italicFont = plainFont.deriveFont(Font.ITALIC);
        }

        // set font to italic if the class is abstract, plain otherwise
        if (Modifier.isAbstract(c.getModifiers()))
            setFont(italicFont);
        else
```

```
            setFont(plainFont);
        return this;
    }
}
```

`javax.swing.JTree` 1.2

- `TreePath getSelectionPath()`
- `TreePath[] getSelectionPaths()`
  return the first selected path, or an array of paths to all selected nodes. If no paths are selected, both methods return `null`.

javax.swing.event.TreeSelectionListener 1.2

- `void valueChanged(TreeSelectionEvent event)`
  is called whenever nodes are selected or deselected.

`javax.swing.event.TreeSelectionEvent` 1.2

- `TreePath getPath()`
- `TreePath[] getPaths()`
  get the first path or all paths that have *changed* in this selection event. If you want to know the current selection, not the selection change, call `JTree.getSelectionPaths` instead.

12.4.6. Custom Tree Models

In the final example, we implement a program that inspects the contents of an object, just like a debugger does (see **Figure 12.30**).

Figure 12.30: An object inspection tree

Before going further, compile and run the example program. Each node corresponds to an instance field. If the field is an object, expand it to see *its* instance fields. The program inspects the contents of the frame window. If you poke around a few of the instance fields, you should be able to find some familiar classes. You'll also gain some respect for how complex the Swing user interface components are under the hood.

What's remarkable about the program is that the tree does not use the `DefaultTreeModel`. If you already have data that are hierarchically organized, you might not want to build a duplicate tree and worry about keeping both trees synchronized. That is the situation in our case—the inspected objects are already linked to each other through the object references, so there is no need to replicate the linking structure.

The `TreeModel` interface has only a handful of methods. The first group of methods enables the `JTree` to find the tree nodes by first getting the root, then the children. The `JTree` class calls these methods only when the user actually expands a node.

```
Object getRoot()
int getChildCount(Object parent)
Object getChild(Object parent, int index)
```

This example shows why the `TreeModel` interface, like the `JTree` class itself, does not need an explicit notion of nodes. The root and its children can be any objects. The `TreeModel` is responsible for telling the `JTree` how they are connected.

The next method of the `TreeModel` interface is the reverse of `getChild`:

```
int getIndexOfChild(Object parent, Object child)
```

Actually, this method can be implemented in terms of the first three—see the code in **Listing 12.12**.

The tree model tells the `JTree` which nodes should be displayed as leaves:

```
boolean isLeaf(Object node)
```

If your code changes the tree model, the tree needs to be notified so it can redraw itself. The tree adds itself as a `TreeModelListener` to the model. Thus, the model must support the usual listener management methods:

```
void addTreeModelListener(TreeModelListener l)
void removeTreeModelListener(TreeModelListener l)
```

You can see the implementations for these methods in **Listing 12.13**.

When the model modifies the tree contents, it calls one of the four methods of the `TreeModelListener` interface:

```
void treeNodesChanged(TreeModelEvent e)
void treeNodesInserted(TreeModelEvent e)
void treeNodesRemoved(TreeModelEvent e)
void treeStructureChanged(TreeModelEvent e)
```

The `TreeModelEvent` object describes the location of the change. The details of assembling a tree model event that describes an insertion or removal event are quite technical. You only need to worry about firing these events if your tree can actually have nodes added and removed. In **Listing 12.12**, we show how to fire one event by replacing the root with a new object.



Tip: To simplify the code for event firing, use the `javax.swing.EventListenerList` convenience class that collects listeners. The last three methods of **Listing 12.13** show how to use the class.

Finally, if the user edits a tree node, your model is called with the change:

```
void valueForPathChanged(TreePath path, Object newValue)
```

If you don't allow editing, this method is never called.

If you don't need to support editing, constructing a tree model is easily done. Implement the three methods:

```
Object getRoot()
int getChildCount(Object parent)
Object getChild(Object parent, int index)
```

These methods describe the structure of the tree. Supply routine implementations of the other five methods, as in **Listing 12.12**. You are then ready to display your tree.

Now let's turn to the implementation of the example program. Our tree will contain objects of type `Variable`.



Note: Had we used the `DefaultTreeModel`, our nodes would have been objects of type `DefaultMutableTreeNode` with *user objects* of type `Variable`.

For example, suppose you inspect the variable

```
Employee joe;
```

That variable has a *type* `Employee.class`, a *name* `"joe"`, and a *value*—the value of the object reference `joe`. In **Listing 12.14**, we define a class `Variable` that describes a variable in a program:

```
var v = new Variable(Employee.class, "joe", joe);
```

If the type of the variable is a primitive type, you must use an object wrapper for the value.

```
new Variable(double.class, "salary", Double.valueOf(salary));
```

If the type of the variable is a class, the variable has *fields*. Using reflection, we enumerate all fields and collect them in an `ArrayList`. Since the `getFields` method of the `Class` class does not return the fields of the superclass, we need to call `getFields` on all superclasses as well. You can find the code in the `Variable` constructor. The `getFields` method of our `Variable` class returns the array of fields. Finally, the `toString` method of the `Variable` class formats the node label. The label always contains the variable type and name. If the variable is not a class, the label also contains the value.



Note: If the type is an array, we do not display the elements of the array. This would not be difficult to do; I leave it as the proverbial "exercise for the reader."

Let's move on to the tree model. The first two methods are simple.

```
public Object getRoot() {
    return root;
}

public int getChildCount(Object parent) {
    return ((Variable) parent).getFields().size();
}
```

The `getChild` method returns a new `Variable` object that describes the field with the given index. The `getType` and `getName` methods of the `Field` class yield the field type and name. By using reflection, you can read the field value as `f.get(parentValue)`. That method can throw an `IllegalAccessException`. However, we made all fields accessible in the `Variable` constructor, so this won't happen in practice.

Here is the complete code of the `getChild` method:

```
public Object getChild(Object parent, int index) {
    ArrayList fields = ((Variable) parent).getFields();
    var f = (Field) fields.get(index);
    Object parentValue = ((Variable) parent).getValue();
    try {
        return new Variable(f.getType(), f.getName(), f.get(parentValue));
    }
    catch (IllegalAccessException e) {
        return null;
    }
}
```

These three methods reveal the structure of the object tree to the `JTree` component. The remaining methods are routine—see the source code in **Listing 12.13**.

There is one remarkable fact about this tree model: It actually describes an *infinite* tree. You can verify this by following one of the `WeakReference` objects. Click on the variable named `referent`. It leads you right back to the original object. You get an identical subtree, and you can open its `WeakReference` object again, ad infinitum. Of course, you cannot *store* an infinite set of nodes; the tree model simply generates the nodes on demand as the user expands the parents. **Listing 12.12** shows the frame class of the sample program.

Listing 12.12 v2ch12/treeModel/ObjectInspectorFrame.java

```
1  package v2ch12.treeModel;
2
3  import module java.desktop;
4
5  /**
6   * This frame holds the object tree.
7   */
8  public class ObjectInspectorFrame extends JFrame {
9      private JTree tree;
```

```
    private static final int DEFAULT_WIDTH = 400;
    private static final int DEFAULT_HEIGHT = 300;

    public ObjectInspectorFrame() {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // we inspect this frame object

        var v = new Variable(getClass(), "this", this);
        var model = new ObjectTreeModel();
        model.setRoot(v);

        // construct and show tree

        tree = new JTree(model);
        add(new JScrollPane(tree), BorderLayout.CENTER);
    }
}
```

Listing 12.13 v2ch12/treeModel/ObjectTreeModel.java

```
package v2ch12.treeModel;

import module java.base;
import module java.desktop;

/**
 * This tree model describes the tree structure of a Java object. Children are the objects that
 * are stored in instance variables.
 */
public class ObjectTreeModel implements TreeModel {
    private Variable root;
    private EventListenerList listenerList = new EventListenerList();

    /**
     * Constructs an empty tree.
     */
    public ObjectTreeModel() {
        root = null;
    }

    /**
     * Sets the root to a given variable.
     * @param v the variable that is being described by this tree
     */
    public void setRoot(Variable v) {
        Variable oldRoot = v;
        root = v;
        fireTreeStructureChanged(oldRoot);
    }

    public Object getRoot() {
        return root;
    }

    public int getChildCount(Object parent) {
        return ((Variable) parent).getFields().size();
```

```
    }

    public Object getChild(Object parent, int index) {
        ArrayList<Field> fields = ((Variable) parent).getFields();
        var f = (Field) fields.get(index);
        Object parentValue = ((Variable) parent).getValue();
        try {
            return new Variable(f.getType(), f.getName(), f.get(parentValue));
        }
        catch (IllegalAccessException e) {
            return null;
        }
    }

    public int getIndexOfChild(Object parent, Object child) {
        int n = getChildCount(parent);
        for (int i = 0; i < n; i++)
            if (getChild(parent, i).equals(child)) return i;
        return -1;
    }

    public boolean isLeaf(Object node) {
        return getChildCount(node) == 0;
    }

    public void valueForPathChanged(TreePath path, Object newValue) {
    }

    public void addTreeModelListener(TreeModelListener l) {
        listenerList.add(TreeModelListener.class, l);
    }

    public void removeTreeModelListener(TreeModelListener l) {
        listenerList.remove(TreeModelListener.class, l);
    }

    protected void fireTreeStructureChanged(Object oldRoot) {
        var event = new TreeModelEvent(this, new Object[] { oldRoot });
        for (TreeModelListener l : listenerList.getListeners(TreeModelListener.class))
            l.treeStructureChanged(event);
    }
}
```

Listing 12.14 v2ch12/treeModel/Variable.java

```
package v2ch12.treeModel;

import module java.base;

/**
 * A variable with a type, name, and value.
 */
public class Variable {
    private Class<?> type;
    private String name;
    private Object value;
    private ArrayList<Field> fields;
```

```
13
14      /**
15       * Construct a variable.
16       * @param aType the type
17       * @param aName the name
18       * @param aValue the value
19       */
20      public Variable(Class<?> aType, String aName, Object aValue) {
21          type = aType;
22          name = aName;
23          value = aValue;
24          fields = new ArrayList<>();
25
26          // find all fields if we have a class type except we don't expand strings and
27          // null values
28
29          if (!type.isPrimitive() && !type.isArray() && !type.equals(String.class)
30                  && value != null) {
31              // get fields from the class and all superclasses
32              for (Class<?> c = value.getClass(); c != null; c = c.getSuperclass()) {
33                  Field[] fs = c.getDeclaredFields();
34                  AccessibleObject.setAccessible(fs, true);
35
36                  // get all nonstatic fields
37                  for (Field f : fs)
38                      if (!Modifier.isStatic(f.getModifiers())) fields.add(f);
39              }
40          }
41      }
42
43      /**
44       * Gets the value of this variable.
45       * @return the value
46       */
47      public Object getValue() {
48          return value;
49      }
50
51      /**
52       * Gets all nonstatic fields of this variable.
53       * @return an array list of variables describing the fields
54       */
55      public ArrayList<Field> getFields() {
56          return fields;
57      }
58
59      public String toString() {
60          String r = type + " " + name;
61          if (type.isPrimitive())
62              r += "=" + value;
63          else if (type.equals(String.class))
64              r += "=" + value;
65          else if (value == null) r += "=null";
66          return r;
67      }
68  }
```

javax.swing.tree.TreeModel **1.2**

- `Object getRoot()`
  returns the root node.
- `int getChildCount(Object parent)`
  gets the number of children of the `parent` node.
- `Object getChild(Object parent, int index)`
  gets the child node of the `parent` node at the given index.
- `int getIndexOfChild(Object parent, Object child)`
  gets the index of the `child` node in the `parent` node, or `-1` if `child` is not a child of `parent` in this tree model.
- `boolean isLeaf(Object node)`
  returns `true` if `node` is conceptually a leaf of the tree.
- `void addTreeModelListener(TreeModelListener l)`
- `void removeTreeModelListener(TreeModelListener l)`
  add or remove listeners that are notified when the information in the tree model changes.
- `void valueForPathChanged(TreePath path, Object newValue)`
  is called when a cell editor has modified the value of a node.

javax.swing.event.TreeModelListener **1.2**

- `void treeNodesChanged(TreeModelEvent e)`
- `void treeNodesInserted(TreeModelEvent e)`
- `void treeNodesRemoved(TreeModelEvent e)`
- `void treeStructureChanged(TreeModelEvent e)`
  are called by the tree model when the tree has been modified.

`javax.swing.event.TreeModelEvent` **1.2**

- `TreeModelEvent(Object eventSource, TreePath node)`
  constructs a tree model event.

12.5. Advanced AWT

You can use the methods of the `Graphics` class to create simple drawings. Those methods are sufficient for simple applications, but they fall short when you need to create complex shapes or require complete control over the appearance of the graphics. The Java 2D API is a more sophisticated class library that you can use to produce high-quality drawings. In the following sections, I'll give you an overview of that API.

12.5.1. The Rendering Pipeline

The original JDK 1.0 had a very simple mechanism for drawing shapes. You selected color and paint mode, and called methods of the `Graphics` class such as `drawRect` or `fillOval`. The Java 2D API supports many more options.

- You can easily produce a wide variety of *shapes*.
- You have control over the *stroke*—the pen that traces shape boundaries.
- You can *fill* shapes with solid colors, varying hues, and repeating patterns.
- You can use *transformations* to move, scale, rotate, or stretch shapes.
- You can *clip* shapes to restrict them to arbitrary areas.

- You can select *composition rules* to describe how to combine the pixels of a new shape with existing pixels.

To draw a shape, go through the following steps:

1. Obtain an object of the `Graphics2D` class. This class is a subclass of the `Graphics` class. Ever since Java 1.2, methods such as `paint` and `paintComponent` automatically receive an object of the `Graphics2D` class. Simply use a cast, as follows:

```
public void paintComponent(Graphics g) {
    var g2 = (Graphics2D) g;
    . . .
}
```

2. Use the `setRenderingHints` method to set *rendering hints*—trade-offs between speed and drawing quality.

```
RenderingHints hints = . . .;
g2.setRenderingHints(hints);
```

3. Use the `setStroke` method to set the *stroke*. The stroke draws the outline of the shape. You can select the thickness and choose among solid and dotted lines.

```
Stroke stroke = . . .;
g2.setStroke(stroke);
```

4. Use the `setPaint` method to set the *paint*. The paint fills areas such as the stroke or the interior of a shape. You can create solid color paint, paint with changing hues, or tiled fill patterns.

```
Paint paint = . . .;
g2.setPaint(paint);
```

5. Use the `clip` method to set the *clipping region*.

```
Shape clip = . . .;
g2.clip(clip);
```

6. Use the `transform` method to set a *transformation* from user space to device space. Use transformations if it is easier for you to define your shapes in a custom coordinate system than by using pixel coordinates.

```
AffineTransform transform = . . .;
g2.transform(transform);
```

7. Use the `setComposite` method to set a *composition rule* that describes how to combine the new pixels with the existing pixels.

```
Composite composite = . . .;
g2.setComposite(composite);
```

8. Create a shape. The Java 2D API supplies many shape objects and methods to combine shapes.

```
Shape shape = . . .;
```

9. Draw or fill the shape. If you draw the shape, its outline is stroked. If you fill the shape, the interior is painted.

```
g2.draw(shape);
g2.fill(shape);
```

Of course, in many practical circumstances, you don't need all these steps. There are reasonable defaults for the settings of the 2D graphics context; change the settings only if you want to deviate from the defaults.

In the following sections, you will see how to describe shapes, strokes, paints, transformations, and composition rules.

The various `set` methods simply set the state of the 2D graphics context. They don't cause any drawing. Similarly, when you construct `Shape` objects, no drawing takes place. A shape is only rendered when you call `draw` or `fill`. At that time, the new shape is computed in a *rendering pipeline* (see **Figure 12.31**).

Figure 12.31: The rendering pipeline

In the rendering pipeline, the following steps take place to render a shape:

1. The path of the shape is stroked.
2. The shape is transformed.
3. The shape is clipped. If there is no intersection between the shape and the clipping area, the process stops.
4. The remainder of the shape after clipping is filled.
5. The pixels of the filled shape are composed with the existing pixels. (In **Figure 12.31**, the circle is part of the existing pixels, and the cup shape is superimposed over it.)

In the next section, you will see how to define shapes. Then, we will turn to the 2D graphics context settings.

`java.awt.Graphics2D` 1.2

- `void draw(Shape s)`
  draws the outline of the given shape with the current paint.
- `void fill(Shape s)`
  fills the interior of the given shape with the current paint.

12.5.2. The `Shape` Class Hierarchy

Here are some of the methods in the `Graphics` class to draw shapes:

```
drawLine
drawRectangle
drawRoundRect
draw3DRect
drawPolygon
drawPolyline
drawOval
drawArc
```

There are also corresponding `fill` methods. These methods have been in the `Graphics` class ever since JDK 1.0. The Java 2D API uses a completely different, object-oriented approach. Instead of methods, there are classes:

```
Line2D
Rectangle2D
RoundRectangle2D
Ellipse2D
Arc2D
QuadCurve2D
CubicCurve2D
GeneralPath
```

These classes all implement the `Shape` interface, which we will examine in the following sections.

The `Line2D`, `Rectangle2D`, `RoundRectangle2D`, `Ellipse2D`, and `Arc2D` classes correspond to the `drawLine`, `drawRectangle`, `drawRoundRect`, `drawOval`, and `drawArc` methods. (The concept of a "3D rectangle" has died the death it so richly deserved—there is no analog to the `draw3DRect` method.) The Java 2D API supplies two additional classes, quadratic and cubic curves, that we will discuss in this section. There is no `Polygon2D` class; instead, the `GeneralPath` class describes paths made up from lines, quadratic and cubic curves. You can use a `GeneralPath` to describe a polygon; we'll show you how later in this section.

To draw a shape, first create an object of a class that implements the `Shape` interface and then call the `draw` method of the `Graphics2D` class.

The classes

```
Rectangle2D
RoundRectangle2D
Ellipse2D
Arc2D
```

all inherit from a common superclass `RectangularShape`. Admittedly, ellipses and arcs are not rectangular, but they have a *bounding rectangle* (see **Figure 12.32**).

Figure 12.32: The bounding rectangle of an ellipse and an arc

Each of the classes with a name ending in "2D" has two subclasses for specifying coordinates as `float` or `double` quantities. In Volume I, you already encountered `Rectangle2D.Float` and `Rectangle2D.Double`.

The same scheme is used for the other classes, such as `Arc2D.Float` and `Arc2D.Double`.

Internally, all graphics classes use `float` coordinates because `float` numbers use less storage space but have sufficient precision for geometric computations. However, the Java programming language makes it a bit more tedious to manipulate `float` numbers. For that reason, most methods of the graphics classes use `double` parameters and return values. Only when constructing a 2D object you need to choose between the constructors with `float` and `double` coordinates. For example,

```
var floatRect = new Rectangle2D.Float(5F, 10F, 7.5F, 15F);
var doubleRect = new Rectangle2D.Double(5, 10, 7.5, 15);
```

The `Xxx2D.Float` and `Xxx2D.Double` classes are subclasses of the `Xxx2D` classes. After object construction, essentially no benefit accrues from remembering the subclass, and you can just store the constructed object in a superclass variable as in the code example above.

As you can see from the curious names, the `Xxx2D.Float` and `Xxx2D.Double` classes are also inner classes of the `Xxx2D` classes. That is just a minor syntactical convenience to avoid inflation of outer class names.

Finally, the `Point2D` class describes a point with an *x* and a *y* coordinate. Points are used to define shapes, but they aren't themselves shapes.

Figure 12.33 shows the relationships between the shape classes. However, the `Double` and `Float` subclasses are omitted. Legacy classes from the pre-2D library are marked with a gray fill.

Figure 12.33: Relationships between the shape classes

12.5.3. Constructing Shapes

You already saw how to use the `Rectangle2D`, `Ellipse2D`, and `Line2D` classes in **Chapter 10**. In this section, you will learn how to work with the remaining 2D shapes.

For the `RoundRectangle2D` shape, specify the top left corner, width, height, and the *x* and *y* dimensions of the corner area that should be rounded (see **Figure 12.34**). For example, the call

```
var r = new RoundRectangle2D.Double(150, 200, 100, 50, 20, 20);
```

produces a rounded rectangle with circles of radius 20 at each of the corners.

Figure 12.34: Constructing a `RoundRectangle2D`

To construct an arc, specify the bounding box, the start angle, the angle swept out by the arc (see **Figure 12.35**), and the closure type—one of `Arc2D.OPEN`, `Arc2D.PIE`, or `Arc2D.CHORD`.

```
var a = new Arc2D(x, y, width, height, startAngle, arcAngle, closureType);
```

Figure 12.35: Constructing an elliptical arc

Figure 12.36 illustrates the arc types.

Figure 12.36: Arc types



Caution: If the arc is elliptical, the computation of the arc angles is not at all straightforward. The API documentation states: “The angles are specified relative to the nonsquare framing rectangle such that 45 degrees always falls on the line from the center of the ellipse to the upper right corner of the framing rectangle. As a result, if the framing rectangle is noticeably longer along one axis than the other, the angles to the start and end of the arc segment will be skewed farther along the longer axis of the frame.” Unfortunately, the documentation is silent on how to compute this “skew.” Here are the details:

Suppose the center of the arc is the origin and the point `(x, y)` lies on the arc. You can get a skewed angle with the following formula:

```
skewedAngle = Math.toDegrees(Math.atan2(-y * height, x * width));
```

The result is a value between `-180` and `180`. Compute the skewed start and end angles in this way. Then, compute the difference between the two skewed angles. If the start angle or the difference is negative, add `360` to the start angle. Then, supply the start angle and the difference to the arc constructor.

If you run the example program at the end of this section, you can visually check that this calculation yields the correct values for the arc constructor (see **Figure 12.39**).

The Java 2D API supports *quadratic* and *cubic* curves. In this chapter, we do not get into the mathematics of these curves. You can get a feel for how the curves look by running the program in **Listing 12.15**. Quadratic curves are specified by two *end points* and a *control point* (see **Figure 12.37**). Cubic curves (**Figure 12.38**) have two control points. Moving the control points changes the shape of the curves.

Figure 12.37: A quadratic curve

Figure 12.38: A cubic curve

To construct quadratic and cubic curves, give the coordinates of the end points and the control points. For example,

```
var q = new QuadCurve2D.Double(startX, startY, controlX, controlY, endX, endY);
var c = new CubicCurve2D.Double(startX, startY, control1X, control1Y,
    control2X, control2Y, endX, endY);
```

Quadratic curves are not very flexible, and they are not commonly used in practice. Cubic curves (such as the Bézier curves drawn by the `CubicCurve2D` class) are, however, very common. By combining many cubic curves

so that the slopes at the connection points match, you can create complex, smooth-looking curved shapes. For more information, refer to *Computer Graphics: Principles and Practice, Third Edition*, by John F. Hughes et al.

You can build arbitrary sequences of line segments, quadratic curves, and cubic curves, and store them in a `GeneralPath` object. Specify the first coordinate of the path with the `moveTo` method, for example:

```
var path = new GeneralPath();
path.moveTo(10, 20);
```

You can then extend the path by calling one of the methods `lineTo`, `quadTo`, or `curveTo`. These methods extend the path by a line, a quadratic curve, or a cubic curve. To call `lineTo`, supply the end point. For the two curve methods, supply the control points, then the end point. For example,

```
path.lineTo(20, 30);
path.curveTo(control1X, control1Y, control2X, control2Y, endX, endY);
```

Close the path by calling the `closePath` method. It draws a straight line back to the starting point of the path.

To make a polygon, simply call `moveTo` to go to the first corner point, followed by repeated calls to `lineTo` to visit the other corner points. Finally, call `closePath` to close the polygon. The program in **Listing 12.15** shows this in more detail.

A general path does not have to be connected. You can call `moveTo` at any time to start a new path segment.

Finally, you can use the `append` method to add arbitrary `Shape` objects to a general path. The outline of the shape is added to the end to the path. The second parameter of the `append` method is `true` if the new shape should be connected to the last point on the path, `false` otherwise. For example, the call

```
Rectangle2D r = . . .;
path.append(r, false);
```

appends the outline of a rectangle to the path without connecting it to the existing path. But

```
path.append(r, true);
```

adds a straight line from the end point of the path to the starting point of the rectangle, and then adds the rectangle outline to the path.

The program in **Listing 12.15** lets you create sample paths. Pick a shape maker from the combo box. The program contains shape makers for

- Straight lines
- Rectangles, rounded rectangles, and ellipses
- Arcs (showing lines for the bounding rectangle and the start and end angles, in addition to the arc itself)
- Polygons (using a `GeneralPath`)
- Quadratic and cubic curves

Use the mouse to adjust the control points. As you move them, the shape continuously repaints itself.

The program is a bit complex because it handles multiple shapes and supports dragging of the control points.

An abstract superclass `ShapeMaker` encapsulates the commonality of the shape maker classes. Each shape has a fixed number of control points that the user can move around. The `getPointCount` method returns that value. The abstract method

```
Shape makeShape(Point2D[] points)
```

computes the actual shape, given the current positions of the control points. The `toString` method returns the class name so that the `ShapeMaker` objects can simply be dumped into a `JComboBox`.

To enable dragging of the control points, the `ShapePanel` class handles both mouse and mouse motion events. If the mouse is pressed on top of a rectangle, subsequent mouse drags move the rectangle.

The majority of the shape maker classes are simple—their `makeShape` methods just construct and return the requested shapes. However, the `ArcMaker` class needs to compute the distorted start and end angles. Furthermore, to demonstrate that the computation is indeed correct, the returned shape is a `GeneralPath` containing the arc itself, the bounding rectangle, and the lines from the center of the arc to the angle control points (see **Figure 12.39**).

Figure 12.39: The `ShapeDemo` program

Listing 12.15 v2ch12/shape/ShapeDemo.java

```
1  package v2ch12.shape;
2
3  import module java.base;
4  import module java.desktop;
5  import java.awt.event.MouseEvent;
6
7  /**
8   * This program demonstrates the various 2D shapes.
9   */
10 class ShapeDemo {
11     void main() {
```

```
        EventQueue.invokeLater(() -> {
            var frame = new ShapeTestFrame();
            frame.setTitle("ShapeTest");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
        });
    }
}

/**
 * This frame contains a combo box to select a shape and a component to draw it.
 */
class ShapeTestFrame extends JFrame {
    public ShapeTestFrame() {
        var comp = new ShapeComponent();
        add(comp, BorderLayout.CENTER);
        var comboBox = new JComboBox<ShapeMaker>();
        comboBox.addItem(new LineMaker());
        comboBox.addItem(new RectangleMaker());
        comboBox.addItem(new RoundRectangleMaker());
        comboBox.addItem(new EllipseMaker());
        comboBox.addItem(new ArcMaker());
        comboBox.addItem(new PolygonMaker());
        comboBox.addItem(new QuadCurveMaker());
        comboBox.addItem(new CubicCurveMaker());
        comboBox.addActionListener(_ -> {
            ShapeMaker shapeMaker = comboBox.getItemAt(comboBox.getSelectedIndex());
            comp.setShapeMaker(shapeMaker);
        });
        add(comboBox, BorderLayout.NORTH);
        comp.setShapeMaker((ShapeMaker) comboBox.getItemAt(0));
        pack();
    }
}

/**
 * This component draws a shape and allows the user to move the points that define it.
 */
class ShapeComponent extends JComponent {
    private static final Dimension PREFERRED_SIZE = new Dimension(300, 200);
    private Point2D[] points;
    private static RandomGenerator generator = RandomGenerator.getDefault();
    private static int SIZE = 10;
    private int current;
    private ShapeMaker shapeMaker;

    public ShapeComponent() {
        addMouseListener(new MouseAdapter() {
            public void mousePressed(MouseEvent event) {
                Point p = event.getPoint();
                for (int i = 0; i < points.length; i++) {
                    double x = points[i].getX() - SIZE / 2;
                    double y = points[i].getY() - SIZE / 2;
                    var r = new Rectangle2D.Double(x, y, SIZE, SIZE);
                    if (r.contains(p)) {
                        current = i;
                        return;
                    }
```

```
                }
            }

            public void mouseReleased(MouseEvent event) {
                current = -1;
            }
        });
        addMouseMotionListener(new MouseMotionAdapter() {
            public void mouseDragged(MouseEvent event) {
                if (current == -1) return;
                points[current] = event.getPoint();
                repaint();
            }
        });
        current = -1;
    }

    /**
     * Set a shape maker and initialize it with a random point set.
     * @param aShapeMaker a shape maker that defines a shape from a point set
     */
    public void setShapeMaker(ShapeMaker aShapeMaker) {
        shapeMaker = aShapeMaker;
        int n = shapeMaker.getPointCount();
        points = new Point2D[n];
        for (int i = 0; i < n; i++) {
            double x = generator.nextDouble() * getWidth();
            double y = generator.nextDouble() * getHeight();
            points[i] = new Point2D.Double(x, y);
        }
        repaint();
    }

    public void paintComponent(Graphics g) {
        if (points == null) return;
        var g2 = (Graphics2D) g;
        for (int i = 0; i < points.length; i++) {
            double x = points[i].getX() - SIZE / 2;
            double y = points[i].getY() - SIZE / 2;
            g2.fill(new Rectangle2D.Double(x, y, SIZE, SIZE));
        }

        g2.draw(shapeMaker.makeShape(points));
    }

    public Dimension getPreferredSize() {
        return PREFERRED_SIZE;
    }
}

/**
 * A shape maker can make a shape from a point set. Concrete subclasses must return a shape in
 * the makeShape method.
 */
abstract class ShapeMaker {
    private int pointCount;

    /**
```

```
     * Constructs a shape maker.
     * @param pointCount the number of points needed to define this shape
     */
    public ShapeMaker(int pointCount) {
        this.pointCount = pointCount;
    }

    /**
     * Gets the number of points needed to define this shape.
     * @return the point count
     */
    public int getPointCount() {
        return pointCount;
    }

    /**
     * Makes a shape out of the given point set.
     * @param p the points that define the shape
     * @return the shape defined by the points
     */
    public abstract Shape makeShape(Point2D[] p);

    public String toString() {
        return getClass().getName();
    }
}

/**
 * Makes a line that joins two given points.
 */
class LineMaker extends ShapeMaker {
    public LineMaker() {
        super(2);
    }

    public Shape makeShape(Point2D[] p) {
        return new Line2D.Double(p[0], p[1]);
    }
}

/**
 * Makes a rectangle that joins two given corner points.
 */
class RectangleMaker extends ShapeMaker {
    public RectangleMaker() {
        super(2);
    }

    public Shape makeShape(Point2D[] p) {
        var s = new Rectangle2D.Double();
        s.setFrameFromDiagonal(p[0], p[1]);
        return s;
    }
}

/**
 * Makes a round rectangle that joins two given corner points.
 */
```

```
class RoundRectangleMaker extends ShapeMaker {
    public RoundRectangleMaker() {
        super(2);
    }

    public Shape makeShape(Point2D[] p) {
        var s = new RoundRectangle2D.Double(0, 0, 0, 0, 20, 20);
        s.setFrameFromDiagonal(p[0], p[1]);
        return s;
    }
}

/**
 * Makes an ellipse contained in a bounding box with two given corner points.
 */
class EllipseMaker extends ShapeMaker {
    public EllipseMaker() {
        super(2);
    }

    public Shape makeShape(Point2D[] p) {
        var s = new Ellipse2D.Double();
        s.setFrameFromDiagonal(p[0], p[1]);
        return s;
    }
}

/**
 * Makes an arc contained in a bounding box with two given corner points, and with starting and
 * ending angles given by lines emanating from the center of the bounding box and ending in two
 * given points. To show the correctness of the angle computation, the returned shape contains
 * the arc, the bounding box, and the lines.
 */
class ArcMaker extends ShapeMaker {
    public ArcMaker() {
        super(4);
    }

    public Shape makeShape(Point2D[] p) {
        double centerX = (p[0].getX() + p[1].getX()) / 2;
        double centerY = (p[0].getY() + p[1].getY()) / 2;
        double width = Math.abs(p[1].getX() - p[0].getX());
        double height = Math.abs(p[1].getY() - p[0].getY());

        double skewedStartAngle = Math.toDegrees(
            Math.atan2(-(p[2].getY() - centerY) * width, (p[2].getX() - centerX) * height));
        double skewedEndAngle = Math.toDegrees(
            Math.atan2(-(p[3].getY() - centerY) * width, (p[3].getX() - centerX) * height));
        double skewedAngleDifference = skewedEndAngle - skewedStartAngle;
        if (skewedStartAngle < 0) skewedStartAngle += 360;
        if (skewedAngleDifference < 0) skewedAngleDifference += 360;

        var s = new Arc2D.Double(0, 0, 0, 0, skewedStartAngle, skewedAngleDifference,
                Arc2D.OPEN);
        s.setFrameFromDiagonal(p[0], p[1]);

        var g = new GeneralPath();
        g.append(s, false);
```

```
        var r = new Rectangle2D.Double();
        r.setFrameFromDiagonal(p[0], p[1]);
        g.append(r, false);
        var center = new Point2D.Double(centerX, centerY);
        g.append(new Line2D.Double(center, p[2]), false);
        g.append(new Line2D.Double(center, p[3]), false);
        return g;
    }
}

/**
 * Makes a polygon defined by six corner points.
 */
class PolygonMaker extends ShapeMaker {
    public PolygonMaker() {
        super(6);
    }

    public Shape makeShape(Point2D[] p) {
        var s = new GeneralPath();
        s.moveTo((float) p[0].getX(), (float) p[0].getY());
        for (int i = 1; i < p.length; i++)
            s.lineTo((float) p[i].getX(), (float) p[i].getY());
        s.closePath();
        return s;
    }
}

/**
 * Makes a quad curve defined by two end points and a control point.
 */
class QuadCurveMaker extends ShapeMaker {
    public QuadCurveMaker() {
        super(3);
    }

    public Shape makeShape(Point2D[] p) {
        return new QuadCurve2D.Double(p[0].getX(), p[0].getY(), p[1].getX(), p[1].getY(),
                p[2].getX(), p[2].getY());
    }
}

/**
 * Makes a cubic curve defined by two end points and two control points.
 */
class CubicCurveMaker extends ShapeMaker {
    public CubicCurveMaker() {
        super(4);
    }

    public Shape makeShape(Point2D[] p) {
        return new CubicCurve2D.Double(p[0].getX(), p[0].getY(), p[1].getX(), p[1].getY(),
                p[2].getX(), p[2].getY(), p[3].getX(), p[3].getY());
    }
}
```

`java.awt.geom.RoundRectangle2D.Double` 1.2

- `RoundRectangle2D.Double(double x, double y, double width, double height, double arcWidth, double arcHeight)`
  constructs a rounded rectangle with the given bounding rectangle and arc dimensions. See for an explanation of the `arcWidth` and `arcHeight` parameters.

`java.awt.geom.Arc2D.Double` 1.2

- `Arc2D.Double(double x, double y, double w, double h, double startAngle, double arcAngle, int type)`
  constructs an arc with the given bounding rectangle, start and arc angle, and arc type. The `startAngle` and `arcAngle` are shown in . The type is one of `Arc2D.OPEN`, `Arc2D.PIE`, and `Arc2D.CHORD`.

`java.awt.geom.QuadCurve2D.Double` 1.2

- `QuadCurve2D.Double(double x1, double y1, double ctrlx, double ctrly, double x2, double y2)`
  constructs a quadratic curve from a start point, a control point, and an end point.

`java.awt.geom.CubicCurve2D.Double` 1.2

- `CubicCurve2D.Double(double x1, double y1, double ctrlx1, double ctrly1, double ctrlx2, double ctrly2, double x2, double y2)`
  constructs a cubic curve from a start point, two control points, and an end point.

`java.awt.geom.GeneralPath` 1.2

- `GeneralPath()`
  constructs an empty general path.

`java.awt.geom.Path2D.Float` 6

- `void moveTo(float x, float y)`
  makes (`x`, `y`) the *current point*—that is, the starting point of the next segment.
- `void lineTo(float x, float y)`
- `void quadTo(float ctrlx, float ctrly, float x, float y)`
- `void curveTo(float ctrl1x, float ctrl1y, float ctrl2x, float ctrl2y, float x, float y)`
  draw a line, quadratic curve, or cubic curve from the current point to the end point (`x`, `y`), and make that end point the current point.

`java.awt.geom.Path2D` 6

- `void append(Shape s, boolean connect)`
  adds the outline of the given shape to the general path. If `connect` is `true`, the current point of the general path is connected to the starting point of the added shape by a straight line.
- `void closePath()`
  closes the path by drawing a straight line from the current point to the first point in the path.

12.5.4. Areas

In the preceding section, you saw how you can specify complex shapes by constructing general paths composed of lines and curves. By using a sufficient number of lines and curves, you can draw essentially any shape. For example, the shapes of characters in the fonts that you see on the screen and on your printouts are made up of lines and quadratic or cubic curves.

Occasionally, it is easier to describe a shape by composing it from *areas*, such as rectangles, polygons, or ellipses. The Java 2D API supports four *constructive area geometry* operations that combine two areas into a new area.

- `add`: The combined area contains all points that are in the first or the second area.
- `subtract`: The combined area contains all points that are in the first but not the second area.
- `intersect`: The combined area contains all points that are in the first and the second area.
- `exclusiveOr`: The combined area contains all points that are in either the first or the second area, but not in both.

Figure 12.40 shows these operations.

Figure 12.40: Constructive area geometry operations

To construct a complex area, start with a default area object.

```
var a = new Area();
```

Then, combine the area with any shape.

```
a.add(new Rectangle2D.Double(. . .));
a.subtract(path);
. . .
```

The `Area` class implements the `Shape` interface. You can stroke the boundary of the area with the `draw` method or paint the interior with the `fill` method of the `Graphics2D` class.

`java.awt.geom.Area`

- `void add(Area other)`
- `void subtract(Area other)`
- `void intersect(Area other)`
- `void exclusiveOr(Area other)`
  carry out the constructive area geometry operation with this area and the other area and set this area to the result.

12.5.5. Strokes

The `draw` operation of the `Graphics2D` class draws the boundary of a shape by using the currently selected *stroke*. By default, the stroke is a solid line that is 1 pixel wide. You can select a different stroke by calling the `setStroke` method and supplying an object of a class that implements the `Stroke` interface. The Java 2D API defines only one such class, called `BasicStroke`. In this section, we'll look at the capabilities of the `BasicStroke` class.

You can construct strokes of arbitrary thickness. For example, here is how to draw lines that are 10 pixels wide:

```
g2.setStroke(new BasicStroke(10.0F));
g2.draw(new Line2D.Double(. . .));
```

When a stroke is more than a pixel thick, the *end* of the stroke can have different styles. **Figure 12.41** shows these so-called end cap styles. You have three choices:

- A *butt cap* simply ends the stroke at its end point.
- A *round cap* adds a half-circle to the end of the stroke.
- A *square cap* adds a half-square to the end of the stroke.

Figure 12.41: End cap styles

When two thick strokes meet, there are three choices for the *join style* (see **Figure 12.42**).

- A *bevel join* joins the strokes with a straight line that is perpendicular to the bisector of the angle between the two strokes.
- A *round join* extends each stroke to have a round cap.
- A *miter join* extends both strokes by adding a “spike.”

Figure 12.42: Join styles

If two lines come together in a miter join at a very small angle, a bevel join is used instead, preventing extremely long spikes. The *miter limit* controls this transition. Technically, this is the ratio of the distance of the inner and outer corners of the spike divided by the stroke width. The default miter limit of 10 corresponds to an angle of about 11 degrees.

You can specify these choices in the `BasicStroke` constructor, for example:

```
g2.setStroke(new BasicStroke(10.0F, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
g2.setStroke(new BasicStroke(10.0F, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER,
    15.0F /* miter limit */));
```

Finally, you can create dashed lines by setting a *dash pattern.* In the program in **Listing 12.16**, you can select a dash pattern that spells out SOS in Morse code. The dash pattern is a `float[]` array that contains the lengths of the "on" and "off" intervals (see **Figure 12.43**).

Figure 12.43: A dash pattern

You can specify the dash pattern and a *dash phase* when constructing the `BasicStroke`. The dash phase indicates where in the dash pattern each line should start. A segment of this length, preceding the starting point of the stroke, is assumed to have the dash pattern already applied. Normally, you set this value to `0`.

```
float[] dashPattern = { 10, 10, 10, 10, 10, 10, 30, 10, 30, . . . };
g2.setStroke(new BasicStroke(10.0F, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER,
    10.0F /* miter limit */, dashPattern, 0 /* dash phase */));
```



Note: End cap styles are applied to the ends of *each dash* in a dash pattern.

The program in **Listing 12.16** lets you specify end cap styles, join styles, and dashed lines (see **Figure 12.44**). You can move the ends of the line segments to test the miter limit: Select the miter join, then move the line segment to form a very acute angle. You will see the miter join turn into a bevel join.

Figure 12.44: The `StrokeDemo` program

The program is similar to the program in **Listing 12.15**. The mouse listener remembers your click on the end point of a line segment, and the mouse motion listener monitors the dragging of the end point. A set of radio buttons signal the user choices for the end cap style, join style, and solid or dashed line. The `paintComponent` method of the `StrokePanel` class constructs a `GeneralPath` consisting of the two line segments that join the three points which the user can move with the mouse. It then constructs a `BasicStroke`, according to the selections the user made, and finally draws the path.

Listing 12.16 v2ch12/stroke/StrokeDemo.java

```
package v2ch12.stroke;

import module java.desktop;
import java.awt.event.MouseEvent;

/**
 * This program demonstrates different stroke types.
 */
class StrokeDemo {
    void main() {
        EventQueue.invokeLater(() -> {
            var frame = new StrokeTestFrame();
            frame.setTitle("StrokeTest");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
        });
    }
}

/**
 * This frame lets the user choose the cap, join, and line style, and shows the resulting
 * stroke.
 */
class StrokeTestFrame extends JFrame {
    private StrokeComponent canvas;
    private JPanel buttonPanel;

    public StrokeTestFrame() {
        canvas = new StrokeComponent();
        add(canvas, BorderLayout.CENTER);

        buttonPanel = new JPanel();
        buttonPanel.setLayout(new GridLayout(3, 3));
        add(buttonPanel, BorderLayout.NORTH);

        var group1 = new ButtonGroup();
        makeCapButton("Butt Cap", BasicStroke.CAP_BUTT, group1);
        makeCapButton("Round Cap", BasicStroke.CAP_ROUND, group1);
        makeCapButton("Square Cap", BasicStroke.CAP_SQUARE, group1);

        var group2 = new ButtonGroup();
        makeJoinButton("Miter Join", BasicStroke.JOIN_MITER, group2);
        makeJoinButton("Bevel Join", BasicStroke.JOIN_BEVEL, group2);
        makeJoinButton("Round Join", BasicStroke.JOIN_ROUND, group2);

        var group3 = new ButtonGroup();
        makeDashButton("Solid Line", false, group3);
        makeDashButton("Dashed Line", true, group3);
    }

    /**
     * Makes a radio button to change the cap style.
     * @param label the button label
     * @param style the cap style
     * @param group the radio button group
     */
```

```
    private void makeCapButton(String label, final int style, ButtonGroup group) {
        // select first button in group
        boolean selected = group.getButtonCount() == 0;
        var button = new JRadioButton(label, selected);
        buttonPanel.add(button);
        group.add(button);
        button.addActionListener(_ -> canvas.setCap(style));
        pack();
    }

    /**
     * Makes a radio button to change the join style.
     * @param label the button label
     * @param style the join style
     * @param group the radio button group
     */
    private void makeJoinButton(String label, final int style, ButtonGroup group) {
        // select first button in group
        boolean selected = group.getButtonCount() == 0;
        var button = new JRadioButton(label, selected);
        buttonPanel.add(button);
        group.add(button);
        button.addActionListener(_ -> canvas.setJoin(style));
    }

    /**
     * Makes a radio button to set solid or dashed lines.
     * @param label the button label
     * @param style false for solid, true for dashed lines
     * @param group the radio button group
     */
    private void makeDashButton(String label, final boolean style, ButtonGroup group) {
        // select first button in group
        boolean selected = group.getButtonCount() == 0;
        var button = new JRadioButton(label, selected);
        buttonPanel.add(button);
        group.add(button);
        button.addActionListener(_ -> canvas.setDash(style));
    }
}

/**
 * This component draws two joined lines, using different stroke objects, and allows the
 * user to drag the three points defining the lines.
 */
class StrokeComponent extends JComponent {
    private static final Dimension PREFERRED_SIZE = new Dimension(400, 400);
    private static int SIZE = 10;

    private Point2D[] points;
    private int current;
    private float width;
    private int cap;
    private int join;
    private boolean dash;

    public StrokeComponent() {
        addMouseListener(new MouseAdapter() {
```

```
            public void mousePressed(MouseEvent event) {
                Point p = event.getPoint();
                for (int i = 0; i < points.length; i++) {
                    double x = points[i].getX() - SIZE / 2;
                    double y = points[i].getY() - SIZE / 2;
                    var r = new Rectangle2D.Double(x, y, SIZE, SIZE);
                    if (r.contains(p)) {
                        current = i;
                        return;
                    }
                }
            }

            public void mouseReleased(MouseEvent event) {
                current = -1;
            }
        });

        addMouseMotionListener(new MouseMotionAdapter() {
            public void mouseDragged(MouseEvent event) {
                if (current == -1) return;
                points[current] = event.getPoint();
                repaint();
            }
        });

        points = new Point2D[3];
        points[0] = new Point2D.Double(200, 100);
        points[1] = new Point2D.Double(100, 200);
        points[2] = new Point2D.Double(200, 200);
        current = -1;
        width = 8.0F;
    }

    public void paintComponent(Graphics g) {
        var g2 = (Graphics2D) g;
        var path = new GeneralPath();
        path.moveTo((float) points[0].getX(), (float) points[0].getY());
        for (int i = 1; i < points.length; i++)
            path.lineTo((float) points[i].getX(), (float) points[i].getY());
        BasicStroke stroke;
        if (dash) {
            float miterLimit = 10.0F;
            float[] dashPattern = { 10F, 10F, 10F, 10F, 10F, 10F, 30F, 10F, 30F, 10F, 30F, 10F,
                    10F, 10F, 10F, 10F, 10F, 30F };
            float dashPhase = 0;
            stroke = new BasicStroke(width, cap, join, miterLimit, dashPattern, dashPhase);
        }
        else
            stroke = new BasicStroke(width, cap, join);
        g2.setStroke(stroke);
        g2.draw(path);
    }

    /**
     * Sets the join style.
     * @param j the join style
     */
```

```
    public void setJoin(int j) {
        join = j;
        repaint();
    }

    /**
     * Sets the cap style.
     * @param c the cap style
     */
    public void setCap(int c) {
        cap = c;
        repaint();
    }

    /**
     * Sets solid or dashed lines.
     * @param d false for solid, true for dashed lines
     */
    public void setDash(boolean d) {
        dash = d;
        repaint();
    }

    public Dimension getPreferredSize() {
        return PREFERRED_SIZE;
    }
}
```

`java.awt.Graphics2D` 1.2

- `void setStroke(Stroke s)`
  sets the stroke of this graphics context to the given object that implements the `Stroke` interface.

`java.awt.BasicStroke` 1.2

- `BasicStroke(float width)`
- `BasicStroke(float width, int cap, int join)`
- `BasicStroke(float width, int cap, int join, float miterlimit)`
- `BasicStroke(float width, int cap, int join, float miterlimit, float[] dash, float dashPhase)`
  construct a stroke object with the given attributes.
  The end cap style is one of `CAP_BUTT`, `CAP_ROUND`, and `CAP_SQUARE`.
  The join style is one of `JOIN_BEVEL`, `JOIN_MITER`, and `JOIN_ROUND`.

12.5.6. Paint

When you fill a shape, its inside is covered with *paint*. Use the `setPaint` method to set the paint style to an object whose class implements the `Paint` interface. The Java 2D API provides three such classes:

- The `Color` class implements the `Paint` interface. To fill shapes with a solid color, simply call `setPaint` with a `Color` object, such as

```
g2.setPaint(Color.red);
```

- The `GradientPaint` class varies colors by interpolating between two given color values (see **Figure 12.45**).

Figure 12.45: Gradient paint

- The `TexturePaint` class fills an area with repetitions of an image (see **Figure 12.46**).

Figure 12.46: Texture paint

You can construct a `GradientPaint` object by specifying two points and the colors that you want at these two points.

```
g2.setPaint(new GradientPaint(p1, Color.RED, p2, Color.YELLOW));
```

Colors are interpolated along the line joining the two points. Colors are constant along lines perpendicular to that joining line. Points beyond an end point of the line are given the color at the end point.

Alternatively, if you call the `GradientPaint` constructor with `true` for the `cyclic` parameter:

```
g2.setPaint(new GradientPaint(p1, Color.RED, p2, Color.YELLOW, true));
```

Then, the color variation *cycles* and keeps varying beyond the end points.

To construct a `TexturePaint` object, specify a `BufferedImage` and an *anchor* rectangle.

```
g2.setPaint(new TexturePaint(bufferedImage, anchorRectangle));
```

I will introduce the `BufferedImage` class later in this chapter when images are discussed in detail. The simplest way of obtaining a buffered image is to read an image file:

```
bufferedImage = ImageIO.read(new File("blue-ball.gif"));
```

The anchor rectangle is extended indefinitely in *x* and *y* directions to tile the entire coordinate plane. The image is scaled to fit into the anchor and then replicated into each tile.

`java.awt.Graphics2D` 1.2

- `void setPaint(Paint s)`
  sets the paint of this graphics context to the given object that implements the `Paint` interface.

`java.awt.GradientPaint` 1.2

- `GradientPaint(float x1, float y1, Color color1, float x2, float y2, Color color2)`
- `GradientPaint(float x1, float y1, Color color1, float x2, float y2, Color color2, boolean cyclic)`
- `GradientPaint(Point2D p1, Color color1, Point2D p2, Color color2)`
- `GradientPaint(Point2D p1, Color color1, Point2D p2, Color color2, boolean cyclic)`
  construct a gradient paint object that fills shapes with color such that the start point is colored with `color1`, the end point is colored with `color2`, and the colors in between are linearly interpolated. Colors are constant along lines perpendicular to the line joining the start and the end point. By default, the gradient paint is not cyclic—that is, points beyond the start and end points are colored with the same color as the start and end point. If the gradient paint is *cyclic,* then colors continue to be interpolated, first returning to the starting point color and then repeating indefinitely in both directions.

`java.awt.TexturePaint` 1.2

- `TexturePaint(BufferedImage texture, Rectangle2D anchor)`
  creates a texture paint object. The anchor rectangle defines the tiling of the space to be painted; it is repeated indefinitely in *x* and *y* directions, and the texture image is scaled to fill each tile.

12.5.7. Coordinate Transformations

Suppose you need to draw an object, such as an automobile. You know, from the manufacturer's specifications, the height, wheelbase, and total length. You could, of course, figure out all pixel positions, assuming some number of pixels per meter. However, there is an easier way: You can ask the graphics context to carry out the conversion for you.

```
g2.scale(pixelsPerMeter, pixelsPerMeter);
g2.draw(new Line2D.Double(coordinates in meters)); // converts to pixels and
                                                   // draws scaled line
```

The `scale` method of the `Graphics2D` class sets the *coordinate transformation* of the graphics context to a scaling transformation. That transformation changes *user coordinates* (user-specified units) to *device coordinates* (pixels). **Figure 12.47** shows how the transformation works.

Figure 12.47: User and device coordinates

Coordinate transformations are very useful in practice. They allow you to work with convenient coordinate values. The graphics context takes care of the dirty work of transforming them to pixels.

There are four fundamental transformations.

- Scaling: blowing up, or shrinking, all distances from a fixed point
- Rotation: rotating all points around a fixed center
- Translation: moving all points by a fixed amount
- Shear: leaving one line fixed and "sliding" the lines parallel to it by an amount that is proportional to the distance from the fixed line

Figure 12.48 shows how these four fundamental transformations act on a unit square.

Figure 12.48: The fundamental transformations

The `scale`, `rotate`, `translate`, and `shear` methods of the `Graphics2D` class set the coordinate transformation of the graphics context to one of these fundamental transformations.

You can *compose* the transformations. For example, you might want to rotate shapes *and* double their size; supply both a rotation and a scaling transformation:

```
g2.rotate(angle);
g2.scale(2, 2);
g2.draw(. . .);
```

In this case, it does not matter in which order you supply the transformations. However, with most transformations, order does matter. For example, if you want to rotate and shear, then it makes a difference which of the transformations you supply first. You need to figure out what your intention is. The graphics context will apply the transformations in the order opposite to that in which you supplied them—that is, the last transformation you supply is applied first.

You can supply as many transformations as you like. For example, consider the following sequence of transformations:

```
g2.translate(x, y);
g2.rotate(a);
g2.translate(-x, -y);
```

The last transformation (which is applied first) moves the point (`x` , `y`) to the origin. The second transformation rotates with an angle `a` around the origin. The final transformation moves the origin back to (`x` , `y`). The overall effect is a rotation with center point (`x` , `y`)—see **Figure 12.49**. Since rotating about a point other than the origin is such a common operation, there is a shortcut:

```
g2.rotate(a, x, y);
```

Figure 12.49: Composing transformations

If you know some matrix theory, you are probably aware that all rotations, translations, scalings, shears, and their compositions can be expressed by transformation matrices of the form:

$$\begin{bmatrix} x_{\text{new}} \\ y_{\text{new}} \\ 1 \end{bmatrix} = \begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Such a transformation is called an *affine transformation.* In the Java 2D API, the `AffineTransform` class describes such a transformation. If you know the components of a particular transformation matrix, you can construct it directly as

```
var t = new AffineTransform(a, b, c, d, e, f);
```

Additionally, the factory methods `getRotateInstance`, `getScaleInstance`, `getTranslateInstance`, and `getShearInstance` construct the matrices that represent these transformation types. For example, the call

```
t = AffineTransform.getScaleInstance(2.0F, 0.5F);
```

returns a transformation that corresponds to the matrix

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Finally, the instance methods `setToRotation`, `setToScale`, `setToTranslation`, and `setToShear` set a transformation object to a new type. Here is an example:

```
t.setToRotation(angle); // sets t to a rotation
```

You can set the coordinate transformation of the graphics context to an `AffineTransform` object.

```
g2.setTransform(t); // replaces current transformation
```

However, in practice, you shouldn't call the `setTransform` operation, as it replaces any existing transformation that the graphics context may have. For example, a graphics context for printing in landscape mode already contains a 90-degree rotation transformation. If you call `setTransform`, you obliterate that rotation. Instead, call the `transform` method.

```
g2.transform(t); // composes current transformation with t
```

It composes the existing transformation with the new `AffineTransform` object.

If you just want to apply a transformation temporarily, first get the old transformation, compose it with your new transformation, and finally restore the old transformation when you are done.

```
AffineTransform oldTransform = g2.getTransform(); // save old transform
g2.transform(t); // apply temporary transform
draw on g2
g2.setTransform(oldTransform); // restore old transform
```

`java.awt.geom.AffineTransform` 1.2

- `AffineTransform(double a, double b, double c, double d, double e, double f)`
- `AffineTransform(float a, float b, float c, float d, float e, float f)`

  construct the affine transform with matrix

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix}$$

- `AffineTransform(double[] m)`
- `AffineTransform(float[] m)`
  construct the affine transform with matrix
  $$\begin{bmatrix} \texttt{m[0]} & \texttt{m[2]} & \texttt{m[4]} \\ \texttt{m[1]} & \texttt{m[3]} & \texttt{m[5]} \\ 0 & 0 & 1 \end{bmatrix}$$
- `static AffineTransform getRotateInstance(double a)`
  creates a rotation around the origin by the angle `a` (in radians). The transformation matrix is
  $$\begin{bmatrix} \cos(a) & -\sin(a) & 0 \\ \sin(a) & \cos(a) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
  If `a` is between 0 and π / 2, the rotation moves the positive *x* axis toward the positive *y* axis.
- `static AffineTransform getRotateInstance(double a, double x, double y)`
  creates a rotation around the point `(x,y)` by the angle `a` (in radians).
- `static AffineTransform getScaleInstance(double sx, double sy)`
  creates a scaling transformation that scales the *x* axis by `sx` and the *y* axis by `sy`. The transformation matrix is
  $$\begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
- `static AffineTransform getShearInstance(double shx, double shy)`
  creates a shear transformation that shears the *x* axis by `shx` and the *y* axis by `shy`. The transformation matrix is
  $$\begin{bmatrix} 1 & shx & 0 \\ shy & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
- `static AffineTransform getTranslateInstance(double tx, double ty)`
  creates a translation that moves the *x* axis by `tx` and the *y* axis by `ty`. The transformation matrix is

$$\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix}$$

- `void setToRotation(double a)`
- `void setToRotation(double a, double x, double y)`
- `void setToScale(double sx, double sy)`
- `void setToShear(double sx, double sy)`
- `void setToTranslation(double tx, double ty)`
  set this affine transformation to a basic transformation with the given parameters. See the `getXxxInstance` methods for an explanation of the basic transformations and their parameters.

`java.awt.Graphics2D` 1.2

- `void setTransform(AffineTransform t)`
  replaces the existing coordinate transformation of this graphics context with `t`.
- `void transform(AffineTransform t)`
  composes the existing coordinate transformation of this graphics context with `t`.
- `void rotate(double a)`
- `void rotate(double a, double x, double y)`
- `void scale(double sx, double sy)`
- `void shear(double sx, double sy)`
- `void translate(double tx, double ty)`
  compose the existing coordinate transformation of this graphics context with a basic transformation with the given parameters. See the `AffineTransform.getXxxInstance` methods for an explanation of the basic transformations and their parameters.

12.5.8. Clipping

By setting a *clipping shape* in the graphics context, you constrain all drawing operations to the interior of that clipping shape.

```
g2.setClip(clipShape); // but see below
g2.draw(shape); // draws only the part that falls inside the clipping shape
```

However, in practice, you don't want to call the `setClip` operation because it replaces any existing clipping shape that the graphics context might have. For example, as you will see later in this chapter, a graphics context for printing comes with a clip rectangle that ensures that you don't draw on the margins. Instead, call the `clip` method.

```
g2.clip(clipShape); // better
```

The `clip` method intersects the existing clipping shape with the new one that you supply.

If you just want to apply a clipping area temporarily, you should first get the old clip, add your new clip, and finally restore the old clip when you are done:

```
Shape oldClip = g2.getClip(); // save old clip
g2.clip(clipShape); // apply temporary clip
draw on g2
g2.setClip(oldClip); // restore old clip
```

In **Figure 12.50**, we show off the clipping capability with a rather dramatic drawing of a line pattern clipped by a complex shape—namely, the outline of a set of letters.

Figure 12.50: Using letter shapes to clip a line pattern

To obtain the character outlines, you need a *font render context*. Use the `getFontRenderContext` method of the `Graphics2D` class.

```
FontRenderContext context = g2.getFontRenderContext();
```

Next, using a string, a font, and the font render context, create a `TextLayout` object:

```
var layout = new TextLayout("Hello", font, context);
```

This text layout object describes the layout of a sequence of characters, as rendered by a particular font render context. The layout depends on the font render context—the same characters will look different on a screen and on a printer.

More important for our application, the `getOutline` method returns a `Shape` object that describes the shape of the outline of the characters in the text layout. The outline shape starts at the origin (0, 0), which might not be what you want. In that case, supply an affine transform to the `getOutline` operation to specify where you would like the outline to appear.

```
AffineTransform transform = AffineTransform.getTranslateInstance(0, 100);
Shape outline = layout.getOutline(transform);
```

Then, append the outline to the clipping shape.

```
var clipShape = new GeneralPath();
clipShape.append(outline, false);
```

Finally, set the clipping shape and draw a set of lines. The lines appear only inside the character boundaries.

```
g2.setClip(clipShape);
var p = new Point2D.Double(0, 0);
for (int i = 0; i < NLINES; i++) {
    double x = . . .;
    double y = . . .;
    var q = new Point2D.Double(x, y);
    g2.draw(new Line2D.Double(p, q)); // lines are clipped
}
```

`java.awt.Graphics` 1.0

- `void setClip(Shape s)` 1.2
  sets the current clipping shape to the shape `s`.
- `Shape getClip()` 1.2
  returns the current clipping shape.

`java.awt.Graphics2D` 1.2

- `void clip(Shape s)`
  intersects the current clipping shape with the shape `s`.
- `FontRenderContext getFontRenderContext()`
  returns a font render context that is necessary for constructing `TextLayout` objects.

`java.awt.font.TextLayout` 1.2

- `TextLayout(String s, Font f, FontRenderContext context)`
  constructs a text layout object from a given string and font, using the font render context to obtain font properties for a particular device.
- `float getAdvance()`
  returns the width of this text layout.
- `float getAscent()`
- `float getDescent()`
  return the height of this text layout above and below the baseline.
- `float getLeading()`
  returns the distance between successive lines in the font used by this text layout.

12.5.9. Transparency and Composition

In the standard RGB color model, every color is described by its red, green, and blue components. However, it is also convenient to describe areas of an image that are *transparent* or partially transparent. When you superimpose an image onto an existing drawing, the transparent pixels do not obscure the pixels under them at all, whereas partially transparent pixels are mixed with the pixels under them. **Figure 12.51** shows the effect of overlaying a partially transparent rectangle on an image. You can still see the details of the image shine through from under the rectangle.

Figure 12.51: Overlaying a partially transparent rectangle on an image

In the Java 2D API, transparency is described by an *alpha channel.* Each pixel has, in addition to its red, green, and blue color components, an alpha value between 0 (fully transparent) and 1 (fully opaque). For example, the rectangle in **Figure 12.51** was filled with a pale yellow color with 50% transparency:

```
new Color(0.7F, 0.7F, 0.0F, 0.5F);
```

Now let us look at what happens if you superimpose two shapes. You need to blend or *compose* the colors and alpha values of the source and destination pixels. Porter and Duff, two researchers in the field of computer graphics, have formulated 12 possible *composition rules* for this blending process. The Java 2D API implements all of these rules. Before going any further, I'd like to point out that only two of these rules have practical significance. If you find the rules arcane or confusing, just use the `SRC_OVER` rule. It is the default rule for a `Graphics2D` object, and it gives the most intuitive results.

Here is the theory behind the rules. Suppose you have a *source pixel* with alpha value a_S. In the image, there is already a *destination pixel* with alpha value a_D. You want to compose the two. The diagram in **Figure 12.52** shows how to design a composition rule.

Figure 12.52: Designing a composition rule

Porter and Duff consider the alpha value as the probability that the pixel color should be used. From the perspective of the source, there is a probability a_S that it wants to use the source color and a probability of 1 – a_S that it doesn't care. The same holds for the destination. When composing the colors, let us assume that the probabilities are independent. Then there are four cases, as shown in **Figure 12.52**. If the source wants to use the source color

and the destination doesn't care, then it seems reasonable to let the source have its way. That's why the upper right corner of the diagram is labeled "S". The probability for that event is $a_S \cdot (1 - a_D)$. Similarly, the lower left corner is labeled "D". What should one do if both destination and source would like to select their color? That's where the Porter–Duff rules come in. If we decide that the source is more important, we label the lower right corner with an "S" as well. That rule is called `SRC_OVER`. In that rule, you combine the source colors with a weight of a_S and the destination colors with a weight of $(1 - a_S) \cdot a_D$.

The visual effect is a blending of the source and destination, with preference given to the source. In particular, if a_S is 1, then the destination color is not taken into account at all. If a_S is 0, then the source pixel is completely transparent and the destination color is unchanged.

The other rules depend on what letters you put in the boxes of the probability diagram. **Table 12.3** and **Figure 12.53** show all rules that are supported by the Java 2D API. The images in the figure show the results of the rules when a rectangular source region with an alpha of 0.75 is combined with an elliptical destination region with an alpha of 1.0.

Table 12.3: The Porter–Duff Composition Rules

Rule	Explanation
`CLEAR`	Source clears destination.
`SRC`	Source overwrites destination and empty pixels.
`DST`	Source does not affect destination.
`SRC_OVER`	Source blends with destination and overwrites empty pixels.
`DST_OVER`	Source does not affect destination and overwrites empty pixels.
`SRC_IN`	Source overwrites destination.
`SRC_OUT`	Source clears destination and overwrites empty pixels.
`DST_IN`	Source alpha modifies destination.
`DST_OUT`	Source alpha complement modifies destination.
`SRC_ATOP`	Source blends with destination.
`DST_ATOP`	Source alpha modifies destination. Source overwrites empty pixels.
`XOR`	Source alpha complement modifies destination. Source overwrites empty pixels.

Figure 12.53: Porter–Duff composition rules

As you can see, most of the rules aren't very useful. Consider, as an extreme case, the `DST_IN` rule. It doesn't take the source color into account at all, but uses the alpha of the source to affect the destination. The `SRC` rule is potentially useful—it forces the source color to be used, turning off blending with the destination.

Use the `setComposite` method of the `Graphics2D` class to install an object of a class that implements the `Composite` interface. The Java 2D API supplies one such class, `AlphaComposite`, that implements all the Porter–Duff rules in **Figure 12.53**.

The factory method `getInstance` of the `AlphaComposite` class yields an `AlphaComposite` object. You supply the rule and the alpha value to be used for source pixels. For example, consider the following code:

```
int rule = AlphaComposite.SRC_OVER;
float alpha = 0.5f;
g2.setComposite(AlphaComposite.getInstance(rule, alpha));
g2.setPaint(Color.blue);
g2.fill(rectangle);
```

The rectangle is then painted with blue color and an alpha value of 0.5. Since the composition rule is `SRC_OVER`, it is transparently overlaid on the existing image.

The program in **Listing 12.17** lets you explore these composition rules. Pick a rule from the combo box and use the slider to set the alpha value of the `AlphaComposite` object.

Furthermore, the program displays a verbal description of each rule. Note that the descriptions are computed from the composition rule diagrams. For example, a `"DS"` in the second row stands for "blends with destination."

The program has one important twist. There is no guarantee that the graphics context that corresponds to the screen has an alpha channel. (In fact, it generally does not.) When pixels are deposited to a destination without an alpha channel, the pixel colors are multiplied with the alpha value and the alpha value is discarded. Now, several of the Porter–Duff rules use the alpha values of the destination, which means a destination alpha channel is important. For that reason, we use a buffered image with the ARGB color model to compose the shapes. After the images have been composed, we draw the resulting image to the screen.

```
var image = new BufferedImage(getWidth(), getHeight(), BufferedImage.TYPE_INT_ARGB);
Graphics2D gImage = image.createGraphics();
// now draw to gImage
g2.drawImage(image, null, 0, 0);
```

Listing 12.17 shows the frame class. The component class is in **Listing 12.18**. The `Rule` class in **Listing 12.19** provides a brief explanation for each rule—see **Figure 12.54**. As you run the program, move the alpha slider from left to right to see the effect on the composed shapes. In particular, note that the only difference between the `DST_IN` and `DST_OUT` rules is how the destination (!) color changes when you change the source alpha.

Figure 12.54: The `CompositeDemo` program

Listing 12.17 v2ch12/composite/CompositeDemoFrame.java

```
1  package v2ch12.composite;
2
3  import module java.desktop;
4
5  /**
6   * This frame contains a combo box to choose a composition rule, a slider to change the source
7   * alpha channel, and a component that shows the composition.
8   */
9  class CompositeDemoFrame extends JFrame {
10     private static final int DEFAULT_WIDTH = 400;
11     private static final int DEFAULT_HEIGHT = 400;
```

```
12
13      private CompositeComponent canvas;
14      private JComboBox<Rule> ruleCombo;
15      private JSlider alphaSlider;
16      private JTextField explanation;
17
18      public CompositeDemoFrame() {
19          setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
20
21          canvas = new CompositeComponent();
22          add(canvas, BorderLayout.CENTER);
23
24          ruleCombo = new JComboBox<>(
25                  new Rule[] { new Rule("CLEAR", "  ", "  "), new Rule("SRC", " S", " S"),
26                          new Rule("DST", "  ", "DD"), new Rule("SRC_OVER", " S", "DS"),
27                          new Rule("DST_OVER", " S", "DD"), new Rule("SRC_IN", "  ", " S"),
28                          new Rule("SRC_OUT", " S", "  "), new Rule("DST_IN", "  ", " D"),
29                          new Rule("DST_OUT", "  ", "D "), new Rule("SRC_ATOP", "  ", "DS"),
30                          new Rule("DST_ATOP", " S", " D"), new Rule("XOR", " S", "D "), });
31          ruleCombo.addActionListener(_ -> {
32              var r = (Rule) ruleCombo.getSelectedItem();
33              canvas.setRule(r.getValue());
34              explanation.setText(r.getExplanation());
35          });
36
37          alphaSlider = new JSlider(0, 100, 75);
38          alphaSlider.addChangeListener(_ -> canvas.setAlpha(alphaSlider.getValue()));
39          var panel = new JPanel();
40          panel.add(ruleCombo);
41          panel.add(new JLabel("Alpha"));
42          panel.add(alphaSlider);
43          add(panel, BorderLayout.NORTH);
44
45          explanation = new JTextField();
46          add(explanation, BorderLayout.SOUTH);
47
48          canvas.setAlpha(alphaSlider.getValue());
49          Rule r = ruleCombo.getItemAt(ruleCombo.getSelectedIndex());
50          canvas.setRule(r.getValue());
51          explanation.setText(r.getExplanation());
52      }
53  }
```

Listing 12.18 v2ch12/composite/CompositeComponent.java

```
1   package v2ch12.composite;
2
3   import module java.desktop;
4
5   /**
6    * This component draws two shapes, composed with a composition rule.
7    */
8   class CompositeComponent extends JComponent {
9       private int rule;
10      private Shape shape1;
11      private Shape shape2;
12      private float alpha;
```

```
13
14      public CompositeComponent() {
15          shape1 = new Ellipse2D.Double(100, 100, 150, 100);
16          shape2 = new Rectangle2D.Double(150, 150, 150, 100);
17      }
18
19      public void paintComponent(Graphics g) {
20          var g2 = (Graphics2D) g;
21
22          var image = new BufferedImage(getWidth(), getHeight(), BufferedImage.TYPE_INT_ARGB);
23          Graphics2D gImage = image.createGraphics();
24          gImage.setPaint(Color.red);
25          gImage.fill(shape1);
26          AlphaComposite composite = AlphaComposite.getInstance(rule, alpha);
27          gImage.setComposite(composite);
28          gImage.setPaint(Color.blue);
29          gImage.fill(shape2);
30          g2.drawImage(image, null, 0, 0);
31      }
32
33      /**
34       * Sets the composition rule.
35       * @param r the rule (as an AlphaComposite constant)
36       */
37      public void setRule(int r) {
38          rule = r;
39          repaint();
40      }
41
42      /**
43       * Sets the alpha of the source.
44       * @param a the alpha value between 0 and 100
45       */
46      public void setAlpha(int a) {
47          alpha = (float) a / 100.0F;
48          repaint();
49      }
50  }
```

Listing 12.19 v2ch12/composite/Rule.java

```
1   package v2ch12.composite;
2
3   import module java.desktop;
4
5   /**
6    * This class describes a Porter-Duff rule.
7    */
8   public class Rule {
9       private String name;
10      private String porterDuff1;
11      private String porterDuff2;
12
13      /**
14       * Constructs a Porter-Duff rule.
15       * @param n the rule name
16       * @param pd1 the first row of the Porter-Duff square
```

```
     * @param pd2 the second row of the Porter-Duff square
     */
    public Rule(String n, String pd1, String pd2) {
        name = n;
        porterDuff1 = pd1;
        porterDuff2 = pd2;
    }

    /**
     * Gets an explanation of the behavior of this rule.
     * @return the explanation
     */
    public String getExplanation() {
        var r = new StringBuilder("Source ");
        if (porterDuff2.equals("  ")) r.append("clears");
        if (porterDuff2.equals(" S")) r.append("overwrites");
        if (porterDuff2.equals("DS")) r.append("blends with");
        if (porterDuff2.equals(" D")) r.append("alpha modifies");
        if (porterDuff2.equals("D ")) r.append("alpha complement modifies");
        if (porterDuff2.equals("DD")) r.append("does not affect");
        r.append(" destination");
        if (porterDuff1.equals(" S")) r.append(" and overwrites empty pixels");
        r.append(".");
        return r.toString();
    }

    public String toString() {
        return name;
    }

    /**
     * Gets the value of this rule in the AlphaComposite class.
     * @return the AlphaComposite constant value, or -1 if there is no matching constant
     */
    public int getValue() {
        try {
            return (Integer) AlphaComposite.class.getField(name).get(null);
        }
        catch (Exception e) {
            return -1;
        }
    }
}
```

`java.awt.Graphics2D` 1.2

- `void setComposite(Composite s)`
  sets the composite of this graphics context to the given object that implements the `Composite` interface.

`java.awt.AlphaComposite` 1.2

- `static AlphaComposite getInstance(int rule)`
- `static AlphaComposite getInstance(int rule, float sourceAlpha)`
  construct an alpha composite object. The rule is one of `CLEAR`, `SRC`, `SRC_OVER`, `DST_OVER`, `SRC_IN`, `SRC_OUT`, `DST_IN`, `DST_OUT`, `DST`, `DST_ATOP`, `SRC_ATOP`, `XOR`.

12.6. Raster Images

The Java2D API lets you create drawings that are made up of lines, curves, and areas. It is a “vector” API because you specify the mathematical properties of the shapes. However, for processing images that are made up of pixels, you want to work with a “raster” of color data. The following sections show you how to process raster images in Java.

12.6.1. Readers and Writers for Images

The `javax.imageio` package contains out-of-the-box support for reading and writing several common file formats, as well as a framework that enables third parties to add readers and writers for other formats. The GIF, JPEG, PNG, BMP (Windows bitmap), and WBMP (wireless bitmap) file formats are supported.

The basics of the library are extremely straightforward. To load an image, use the static `read` method of the `ImageIO` class:

```
File f = . . .;
BufferedImage image = ImageIO.read(f);
```

The `ImageIO` class picks an appropriate reader, based on the file type. It may consult the file extension and the “magic number” at the beginning of the file for that purpose. If no suitable reader can be found or the reader can’t decode the file contents, the `read` method returns `null`.

Writing an image to a file is just as simple:

```
File f = . . .;
String format = . . .;
ImageIO.write(image, format, f);
```

Here the format string is a string identifying the image format, such as `"JPEG"` or `"PNG"`. The `ImageIO` class picks an appropriate writer and saves the file.

12.6.2. Obtaining Readers and Writers for Image File Types

For more advanced image reading and writing operations that go beyond the static `read` and `write` methods of the `ImageIO` class, you first need to get the appropriate `ImageReader` and `ImageWriter` objects. The `ImageIO` class enumerates readers and writers that match one of the following:

- An image format (such as `"JPEG"`)
- A file suffix (such as `"jpg"`)
- A MIME type (such as `"image/jpeg"`)



Note: MIME is the Multipurpose Internet Mail Extensions standard. The MIME standard defines common data formats such as `image/jpeg` and `application/pdf`.

For example, you can obtain a reader that reads JPEG files as follows:

```
ImageReader reader = null;
Iterator<ImageReader> iter = ImageIO.getImageReadersByFormatName("JPEG");
if (iter.hasNext()) reader = iter.next();
```

The `getImageReadersBySuffix` and `getImageReadersByMIMEType` methods enumerate readers that match a file extension or MIME type.

It is possible that the `ImageIO` class can locate multiple readers that can all read a particular image type. In that case, you have to pick one of them, but it isn't clear how you can decide which one is the best. To find out more information about a reader, obtain its *service provider interface*:

```
ImageReaderSpi spi = reader.getOriginatingProvider();
```

Then you can get the vendor name and version number:

```
String vendor = spi.getVendor();
String version = spi.getVersion();
```

Perhaps that information can help you decide among the choices—or you might just present a list of readers to your program users and let them choose. For now, we assume that the first enumerated reader is adequate.

In the sample program in **Listing 12.20**, we want to find all file suffixes of all available readers so that we can use them in a file filter. Use the static `ImageIO.getReaderFileSuffixes` method for this purpose:

```
String[] extensions = ImageIO.getWriterFileSuffixes();
chooser.setFileFilter(new FileNameExtensionFilter("Image files", extensions));
```

For saving files, we have to work harder. We'd like to present the user with a menu of all supported image types. Unfortunately, the `getWriterFormatNames` of the `ImageIO` class returns a rather curious list with redundant names, such as

```
jpg, BMP, bmp, JPG, jpeg, wbmp, png, JPEG, PNG, WBMP, GIF, gif, TIFF, tif, tiff
```

That's not something one would want to present in a menu. What is needed is a list of "preferred" format names. The helper method `getWriterFormats` in **Listing 12.20** looks up the first writer associated with each format name, and asks it what its format names are, in the hope that it will list the most popular one first. Indeed, for the JPEG writer, this works fine—it lists `"JPEG"` before the other options. The PNG writer, on the other hand, lists `"png"` in lower case before `"PNG"`. Therefore, for consistency, the helper method forces lowercase names to upper case.

12.6.3. Reading and Writing Files with Multiple Images

Some files—in particular, animated GIF files—contain multiple images. The `read` method of the `ImageIO` class reads a single image. To read multiple images, turn the input source (for example, an input stream or file) into an `ImageInputStream`.

```
InputStream in = . . .;
ImageInputStream imageIn = ImageIO.createImageInputStream(in);
```

Then, attach the image input stream to the reader:

```
reader.setInput(imageIn, true);
```

The second argument indicates that the input is in “seek forward only” mode. Otherwise, random access is used, either by buffering stream input as it is read or by using random file access. Random access is required for certain operations. For example, to find out the number of images in a GIF file, you need to read the entire file. If you then want to fetch an image, the input must be read again.

This consideration is only important if you read from a stream, if the input contains multiple images, and if the image format doesn’t have the information that you request (such as the image count) in the header. If you read from a file, simply use

```
File f = . . .;
ImageInputStream imageIn = ImageIO.createImageInputStream(f);
reader.setInput(imageIn);
```

Once you have a reader, you can read the images in the input by calling

```
BufferedImage image = reader.read(index);
```

where `index` is the image index, starting with `0`.

If the input is in the “seek forward only” mode, you keep reading images until the `read` method throws an `IndexOutOfBoundsException`. Otherwise, you can call the `getNumImages` method:

```
int n = reader.getNumImages(true);
```

Here, the argument indicates that you allow a search of the input to determine the number of images. That method throws an `IllegalStateException` if the input is in the “seek forward only” mode. Alternatively, you can set the “allow search” parameter to `false`. Then the `getNumImages` method returns `-1` if it can’t determine the number of images without a search. In that case, you’ll have to switch to Plan B and keep reading images until you get an `IndexOutOfBoundsException`.

Some files contain thumbnails—smaller versions of an image for preview purposes. You can get the number of thumbnails of an image with the call

```
int count = reader.getNumThumbnails(index);
```

Then you get a particular index as

```
BufferedImage thumbnail = reader.getThumbnail(index, thumbnailIndex);
```

Sometimes you may want to get the image size before actually getting the image—in particular, if the image is huge or comes from a slow network connection. Use the calls

```
int width = reader.getWidth(index);
int height = reader.getHeight(index);
```

to get the dimensions of an image with a given index.

To write a file with multiple images, you first need an `ImageWriter`. The `ImageIO` class can enumerate the writers capable of writing a particular image format:

```
String format = . . .;
ImageWriter writer = null;
Iterator<ImageWriter> iter = ImageIO.getImageWritersByFormatName(format);
if (iter.hasNext()) writer = iter.next();
```

Next, turn an output stream or file into an `ImageOutputStream` and attach it to the writer. For example,

```
File f = . . .;
ImageOutputStream imageOut = ImageIO.createImageOutputStream(f);
writer.setOutput(imageOut);
```

You must wrap each image into an `IIOImage` object. You can optionally supply a list of thumbnails and image metadata (such as compression algorithms and color information). In this example, we just use `null` for both; see the API documentation for additional information.

```
var iioImage = new IIOImage(images[i], null, null);
```

To write out the *first* image, use the `write` method:

```
writer.write(new IIOImage(images[0], null, null));
```

For subsequent images, use

```
if (writer.canInsertImage(i))
    writer.writeInsert(i, iioImage, null);
```

The third argument can contain an `ImageWriteParam` object to set image writing details such as tiling and compression; use `null` for default values.

Not all file formats can handle multiple images. In that case, the `canInsertImage` method returns `false` for `i` > 0, and only a single image is saved.

The program in **Listing 12.20** lets you load and save files in the formats for which the Java library supplies readers and writers. The program displays multiple images (see **Figure 12.55**), but not thumbnails.

Figure 12.55: An animated GIF image

Listing 12.20 v2ch12/imageIO/ImageIOFrame.java

```
package v2ch12.imageIO;

import module java.base;
import module java.desktop;
import java.util.List;

/**
 * This frame displays the loaded images. The menu has items for loading and saving files.
 */
public class ImageIOFrame extends JFrame {
    private static final int DEFAULT_WIDTH = 400;
    private static final int DEFAULT_HEIGHT = 400;

    private static Set<String> writerFormats = getWriterFormats();

    private BufferedImage[] images;

    public ImageIOFrame() {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        var fileMenu = new JMenu("File");
        var openItem = new JMenuItem("Open");
        openItem.addActionListener(_ -> openFile());
        fileMenu.add(openItem);

        var saveMenu = new JMenu("Save");
        fileMenu.add(saveMenu);
        Iterator<String> iter = writerFormats.iterator();
        while (iter.hasNext()) {
            final String formatName = iter.next();
            var formatItem = new JMenuItem(formatName);
            saveMenu.add(formatItem);
```

```
            formatItem.addActionListener(_ -> saveFile(formatName));
        }

        var exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(_ -> System.exit(0));
        fileMenu.add(exitItem);

        var menuBar = new JMenuBar();
        menuBar.add(fileMenu);
        setJMenuBar(menuBar);
    }

    /**
     * Open a file and load the images.
     */
    public void openFile() {
        var chooser = new JFileChooser();
        chooser.setCurrentDirectory(new File("."));
        String[] extensions = ImageIO.getReaderFileSuffixes();
        chooser.setFileFilter(new FileNameExtensionFilter("Image files", extensions));
        int r = chooser.showOpenDialog(this);
        if (r != JFileChooser.APPROVE_OPTION) return;
        File f = chooser.getSelectedFile();
        Box box = Box.createVerticalBox();
        try {
            String name = f.getName();
            String suffix = name.substring(name.lastIndexOf(".") + 1);
            Iterator<ImageReader> iter = ImageIO.getImageReadersBySuffix(suffix);
            ImageReader reader = iter.next();
            ImageInputStream imageIn = ImageIO.createImageInputStream(f);
            reader.setInput(imageIn);
            int count = reader.getNumImages(true);
            images = new BufferedImage[count];
            for (int i = 0; i < count; i++) {
                images[i] = reader.read(i);
                box.add(new JLabel(new ImageIcon(images[i])));
            }
        }
        catch (IOException e) {
            JOptionPane.showMessageDialog(this, e);
        }
        setContentPane(new JScrollPane(box));
        validate();
    }

    /**
     * Save the current image in a file.
     * @param formatName the file format
     */
    public void saveFile(final String formatName) {
        if (images == null) return;
        Iterator<ImageWriter> iter = ImageIO.getImageWritersByFormatName(formatName);
        ImageWriter writer = iter.next();
        var chooser = new JFileChooser();
        chooser.setCurrentDirectory(new File("."));
        String[] extensions = writer.getOriginatingProvider().getFileSuffixes();
        chooser.setFileFilter(new FileNameExtensionFilter("Image files", extensions));

```

```
91          int r = chooser.showSaveDialog(this);
92          if (r != JFileChooser.APPROVE_OPTION) return;
93          File f = chooser.getSelectedFile();
94          try {
95              ImageOutputStream imageOut = ImageIO.createImageOutputStream(f);
96              writer.setOutput(imageOut);
97
98              writer.write(new IIOImage(images[0], null, null));
99              for (int i = 1; i < images.length; i++) {
100                 var iioImage = new IIOImage(images[i], null, null);
101                 if (writer.canInsertImage(i)) writer.writeInsert(i, iioImage, null);
102             }
103         }
104         catch (IOException e) {
105             JOptionPane.showMessageDialog(this, e);
106         }
107     }
108
109     /**
110      * Gets a set of "preferred" format names of all image writers. The preferred
111      * format name is the first format name that a writer specifies.
112      * @return the format name set
113      */
114     public static Set<String> getWriterFormats() {
115         var writerFormats = new TreeSet<String>();
116         var formatNames = List.of(ImageIO.getWriterFormatNames());
117         while (formatNames.size() > 0) {
118             String name = formatNames.iterator().next();
119             Iterator<ImageWriter> iter = ImageIO.getImageWritersByFormatName(name);
120             ImageWriter writer = iter.next();
121             String[] names = writer.getOriginatingProvider().getFormatNames();
122             String format = names[0];
123             if (format.equals(format.toLowerCase())) format = format.toUpperCase();
124             writerFormats.add(format);
125             formatNames.removeAll(List.of(names));
126         }
127         return writerFormats;
128     }
129 }
```

`javax.imageio.ImageIO` **1.4**

- `static BufferedImage read(File input)`
- `static BufferedImage read(InputStream input)`
- `static BufferedImage read(URL input)`

  read an image from `input`.
- `static boolean write(RenderedImage image, String formatName, File output)`
- `static boolean write(RenderedImage image, String formatName, OutputStream output)`

  write an image in the given format to `output`. Return `false` if no appropriate writer was found.

- `static Iterator<ImageReader> getImageReadersByFormatName(String formatName)`
- `static Iterator<ImageReader> getImageReadersBySuffix(String fileSuffix)`
- `static Iterator<ImageReader> getImageReadersByMIMEType(String mimeType)`
- `static Iterator<ImageWriter> getImageWritersByFormatName(String formatName)`
- `static Iterator<ImageWriter> getImageWritersBySuffix(String fileSuffix)`
- `static Iterator<ImageWriter> getImageWritersByMIMEType(String mimeType)`
  get all readers and writers that are able to handle the given format (e.g., `"JPEG"`), file suffix (e.g., `"jpg"`), or MIME type (e.g., `"image/jpeg"`).
- `static String[] getReaderFormatNames()`
- `static String[] getReaderMIMETypes()`
- `static String[] getWriterFormatNames()`
- `static String[] getWriterMIMETypes()`
- `static String[] getReaderFileSuffixes()` 6
- `static String[] getWriterFileSuffixes()` 6
  get all format names, MIME type names, and file suffixes supported by readers and writers.
- `ImageInputStream createImageInputStream(Object input)`
- `ImageOutputStream createImageOutputStream(Object output)`
  create an image input or image output stream from the given object. The object can be a file, a stream, a `RandomAccessFile`, or another object for which a service provider exists. Return `null` if no registered service provider can handle the object.

`javax.imageio.ImageReader` 1.4

- `void setInput(Object input)`
- `void setInput(Object input, boolean seekForwardOnly)`
  set the input source of the reader. The `input` parameter is an `ImageInputStream` object or another object that this reader can accept. The `seekForwardOnly` parameter is `true` if the reader should read forward only. By default, the reader uses random access and, if necessary, buffers image data.
- `BufferedImage read(int index)`
  reads the image with the given image index (starting at `0`). Throws an `IndexOutOfBoundsException` if no such image is available.
- `int getNumImages(boolean allowSearch)`
  gets the number of images in this reader. If `allowSearch` is `false` and the number of images cannot be determined without reading forward, then `-1` is returned. If `allowSearch` is `true` and the reader is in the "seek forward only" mode, then an `IllegalStateException` is thrown.
- `int getNumThumbnails(int index)`
  gets the number of thumbnails of the image with the given `index`.
- `BufferedImage readThumbnail(int index, int thumbnailIndex)`
  gets the thumbnail with index `thumbnailIndex` of the image with the given `index`.
- `int getWidth(int index)`
- `int getHeight(int index)`
  get the image width and height. Throw an `IndexOutOfBoundsException` if no such image is available.
- `ImageReaderSpi getOriginatingProvider()`
  gets the service provider that constructed this reader.

`javax.imageio.spi.IIOServiceProvider` 1.4

- `String getVendorName()`
- `String getVersion()`

get the vendor name and version of this service provider.

`javax.imageio.spi.ImageReaderWriterSpi` 1.4

- `String[] getFormatNames()`
- `String[] getFileSuffixes()`
- `String[] getMIMETypes()`
  get the format names, file suffixes, and MIME types supported by the readers or writers that this service provider creates.

`javax.imageio.ImageWriter` 1.4

- `void setOutput(Object output)`
  sets the output target of this writer. The `output` parameter is an `ImageOutputStream` object or another object that this writer can accept.
- `void write(IIOImage image)`
- `void write(RenderedImage image)`
  write a single image to the output.
- `void writeInsert(int index, IIOImage image, ImageWriteParam param)`
  writes an image into a multi-image file.
- `boolean canInsertImage(int index)`
  returns `true` if it is possible to insert an image at the given index.
- `ImageWriterSpi getOriginatingProvider()`
  gets the service provider that constructed this writer.

`javax.imageio.IIOImage` 1.4

- `IIOImage(RenderedImage image, List thumbnails, IIOMetadata metadata)`
  constructs an `IIOImage` from an image, optional thumbnails, and optional metadata.

12.6.4. Image Manipulation

Suppose you have an image and you would like to improve its appearance. You then need to access the individual pixels of the image and replace them with other pixels. Or perhaps you want to compute the pixels of an image from scratch—for example, to show the result of physical measurements or a mathematical computation. The `BufferedImage` class gives you control over the pixels in an image, and the classes that implement the `BufferedImageOp` interface let you transform images.



Note: JDK 1.0 had a completely different, and far more complex, imaging framework that was optimized for *incremental rendering* of images downloaded from the Web, a scan line at a time. However, it was difficult to manipulate those images. We do not discuss that framework in this book.

Most of the images that you manipulate are simply read in from an image file—they were either produced by a device such as a digital camera or scanner, or constructed by a drawing program. In this section, we'll show you a different technique for constructing an image—namely, building it up a pixel at a time.

To create an image, construct a `BufferedImage` object in the usual way.

```
image = new BufferedImage(width, height, BufferedImage.TYPE_INT_ARGB);
```

Now, call the `getRaster` method to obtain an object of type `WritableRaster`. You will use this object to access and modify the pixels of the image.

```
WritableRaster raster = image.getRaster();
```

The `setPixel` method lets you set an individual pixel. The complexity here is that you can't simply set the pixel to a `Color` value. You must know how the buffered image specifies color values. That depends on the *type* of the image. If your image has a type of `TYPE_INT_ARGB`, then each pixel is described by four values—red, green, blue, and alpha, each between `0` and `255`. You have to supply them in an array of four integers:

```
int[] black = { 0, 0, 0, 255 };
raster.setPixel(i, j, black);
```

In the lingo of the Java 2D API, these values are called the *sample values* of the pixel.



Caution: There are also `setPixel` methods with parameters of types `float[]` and `double[]`. However, the values that you need to place into these arrays are *not* normalized color values between `0.0` and `1.0`.

```
float[] red = { 1.0F, 0.0F, 0.0F, 1.0F };
raster.setPixel(i, j, red); // ERROR
```

You need to supply values between `0` and `255`, no matter what the type of the array is.

You can supply a batch of pixels with the `setPixels` method. Specify the starting pixel position and the width and height of the rectangle that you want to set. Then, supply an array that contains the sample values for all pixels. For example, if your buffered image has a type of `TYPE_INT_ARGB`, supply the red, green, blue, and alpha values of the first pixel, then the red, green, blue, and alpha values for the second pixel, and so on.

```
var pixels = new int[4 * width * height];
pixels[0] = . . .; // red value for first pixel
pixels[1] = . . .; // green value for first pixel
pixels[2] = . . .; // blue value for first pixel
pixels[3] = . . .; // alpha value for first pixel
. . .
raster.setPixels(x, y, width, height, pixels);
```

Conversely, to read a pixel, use the `getPixel` method. Supply an array of four integers to hold the sample values.

```
var sample = new int[4];
raster.getPixel(x, y, sample);
var color = new Color(sample[0], sample[1], sample[2], sample[3]);
```

You can read multiple pixels with the `getPixels` method.

```
raster.getPixels(x, y, width, height, samples);
```

If you use an image type other than `TYPE_INT_ARGB` and you know how that type represents pixel values, you can still use the `getPixel/setPixel` methods. However, you have to know the encoding of the sample values in the particular image type.

If you need to manipulate an image with an arbitrary, unknown image type, then you have to work a bit harder. Every image type has a *color model* that can translate between sample value arrays and the standard RGB color model.



Note: The RGB color model isn't as standard as you might think. The exact look of a color value depends on the characteristics of the imaging device. Digital cameras, scanners, monitors, and LCD displays all have their own idiosyncrasies. As a result, the same RGB value can look quite different on different devices. The International Color Consortium (**https://www.color.org**) recommends that all color data be accompanied by an *ICC profile* that specifies how the colors map to a standard form such as the 1931 CIE XYZ color specification. That specification was designed by the Commission Internationale de l'Eclairage, or CIE (**https://www.cie.co.at**), the international organization in charge of providing technical guidance in all matters of illumination and color. The specification is a standard method for representing any color that the human eye can perceive as a triplet of coordinates called X, Y, Z. (See, for example, *Computer Graphics: Principles and Practice, Third Edition*, by John F. Hughes et al., Chapter 28, for more information on the 1931 CIE XYZ specification.)

ICC profiles are complex, however. A simpler proposed standard, called sRGB (**https://www.w3.org/Graphics/Color/sRGB.html**), specifies an exact mapping between RGB values and the 1931 CIE XYZ values that was designed to work well with typical color monitors. The Java 2D API uses that mapping when converting between RGB and other color spaces.

The `getColorModel` method returns the color model:

```
ColorModel model = image.getColorModel();
```

To find the color value of a pixel, call the `getDataElements` method of the `Raster` class. That call returns an `Object` that contains a color-model-specific description of the color value.

```
Object data = raster.getDataElements(x, y, null);
```



Note: The object that is returned by the `getDataElements` method is actually an array of sample values. You don't need to know this to process the object, but it explains why the method is called `getDataElements`.

The color model can translate the object to standard ARGB values. The `getRGB` method returns an `int` value that has the alpha, red, green, and blue values packed in four blocks of eight bits each. You can construct a `Color` value out of that integer with the `Color(int argb, boolean hasAlpha)` constructor:

```
int argb = model.getRGB(data);
var color = new Color(argb, true);
```

To set a pixel to a particular color, reverse these steps. The `getRGB` method of the `Color` class yields an `int` value with the alpha, red, green, and blue values. Supply that value to the `getDataElements` method of the

`ColorModel` class. The return value is an `Object` that contains the color-model-specific description of the color value. Pass the object to the `setDataElements` method of the `WritableRaster` class.

```
int argb = color.getRGB();
Object data = model.getDataElements(argb, null);
raster.setDataElements(x, y, data);
```

To illustrate how to use these methods to build an image from individual pixels, we bow to tradition and draw a Mandelbrot set, as shown in **Figure 12.56**.

Figure 12.56: A Mandelbrot set

The idea of the Mandelbrot set is that each point of the plane is associated with a sequence of numbers. If that sequence stays bounded, you color the point. If it "escapes to infinity," you leave it transparent.

Here is how you can construct the simplest Mandelbrot set. For each point (a, b), look at sequences that start with $(x, y) = (0, 0)$ and iterate:

$$x_{\text{new}} = x^2 - y^2 + a$$

$$y_{\text{new}} = 2 \cdot x \cdot y + b$$

It turns out that if x or y ever gets larger than 2, then the sequence escapes to infinity. Only the pixels that correspond to points (a, b) leading to a bounded sequence are colored. (The formulas for the number sequences come ultimately from the mathematics of complex numbers; we'll just take them for granted.)

Listing 12.21 shows the code. This program demonstrates how to use the `ColorModel` class for translating `Color` values into pixel data. That process is independent of the image type. Just for fun, change the color type of the buffered image to `TYPE_BYTE_GRAY`. You don't need to change any other code—the color model of the image automatically takes care of the conversion from colors to sample values.

Listing 12.21 v2ch12/rasterImage/RasterImageFrame.java

```
package v2ch12.rasterImage;

import module java.desktop;

/**
 * This frame shows an image with a Mandelbrot set.
 */
public class RasterImageFrame extends JFrame {
    private static final double XMIN = -2;
    private static final double XMAX = 2;
    private static final double YMIN = -2;
    private static final double YMAX = 2;
    private static final int MAX_ITERATIONS = 16;
    private static final int IMAGE_WIDTH = 400;
    private static final int IMAGE_HEIGHT = 400;

    public RasterImageFrame() {
        BufferedImage image = makeMandelbrot(IMAGE_WIDTH, IMAGE_HEIGHT);
        add(new JLabel(new ImageIcon(image)));
        pack();
    }

    /**
     * Makes the Mandelbrot image.
     * @param width the width
     * @parah height the height
     * @return the image
     */
    public BufferedImage makeMandelbrot(int width, int height) {
        var image = new BufferedImage(width, height, BufferedImage.TYPE_INT_ARGB);
        WritableRaster raster = image.getRaster();
        ColorModel model = image.getColorModel();

        Color fractalColor = Color.RED;
        int argb = fractalColor.getRGB();
        Object colorData = model.getDataElements(argb, null);

        for (int i = 0; i < width; i++)
            for (int j = 0; j < height; j++) {
                double a = XMIN + i * (XMAX - XMIN) / width;
                double b = YMIN + j * (YMAX - YMIN) / height;
                if (!escapesToInfinity(a, b)) raster.setDataElements(i, j, colorData);
            }
        return image;
    }

    private boolean escapesToInfinity(double a, double b) {
        double x = 0.0;
        double y = 0.0;
        int iterations = 0;
        while (x <= 2 && y <= 2 && iterations < MAX_ITERATIONS) {
            double xnew = x * x - y * y + a;
            double ynew = 2 * x * y + b;
            x = xnew;
            y = ynew;
            iterations++;
```

```
57          }
58          return x > 2 || y > 2;
59      }
60  }
```

`java.awt.image.BufferedImage` 1.2

- `BufferedImage(int width, int height, int imageType)`
  constructs a buffered image object.
  The most common image types are `TYPE_INT_RGB`, `TYPE_INT_ARGB`, `TYPE_BYTE_GRAY`, and `TYPE_BYTE_INDEXED`.
- `ColorModel getColorModel()`
  returns the color model of this buffered image.
- `WritableRaster getRaster()`
  gets the raster for accessing and modifying pixels of this buffered image.

`java.awt.image.Raster` 1.2

- `Object getDataElements(int x, int y, Object data)`
  returns the sample data for a raster point, in an array whose element type and length depend on the color model. If `data` is not `null`, it is assumed to be an array that is appropriate for holding sample data, and it is filled. If `data` is `null`, a new array is allocated. Its element type and length depend on the color model.
- `int[] getPixel(int x, int y, int[] sampleValues)`
- `float[] getPixel(int x, int y, float[] sampleValues)`
- `double[] getPixel(int x, int y, double[] sampleValues)`
- `int[] getPixels(int x, int y, int width, int height, int[] sampleValues)`
- `float[] getPixels(int x, int y, int width, int height, float[] sampleValues)`
- `double[] getPixels(int x, int y, int width, int height, double[] sampleValues)`
  return the sample values for a raster point, or a rectangle of raster points, in an array whose length depends on the color model. If `sampleValues` is not `null`, it is assumed to be sufficiently long for holding the sample values, and it is filled. If `sampleValues` is `null`, a new array is allocated. These methods are only useful if you know the meaning of the sample values for a color model.

`java.awt.image.WritableRaster` 1.2

- `void setDataElements(int x, int y, Object data)`
  sets the sample data for a raster point. `data` is an array filled with the sample data for a pixel. Its element type and length depend on the color model.
- `void setPixel(int x, int y, int[] sampleValues)`
- `void setPixel(int x, int y, float[] sampleValues)`
- `void setPixel(int x, int y, double[] sampleValues)`
- `void setPixels(int x, int y, int width, int height, int[] sampleValues)`
- `void setPixels(int x, int y, int width, int height, float[] sampleValues)`
- `void setPixels(int x, int y, int width, int height, double[] sampleValues)`
  set the sample values for a raster point or a rectangle of raster points. These methods are only useful if you know the encoding of the sample values for a color model.

`java.awt.image.ColorModel` 1.2

- `int getRGB(Object data)`
  returns the ARGB value that corresponds to the sample data passed in the `data` array. Its element type and length depend on the color model.
- `Object getDataElements(int argb, Object data);`
  returns the sample data for a color value. If `data` is not `null`, it is assumed to be an array that is appropriate for holding sample data, and it is filled. If `data` is `null`, a new array is allocated. `data` is an array filled with the sample data for a pixel. Its element type and length depend on the color model.

`java.awt.Color` 1.0

- `Color(int argb, boolean hasAlpha)` 1.2
  creates a color with the specified combined ARGB value if `hasAlpha` is `true`, or the specified RGB value if `hasAlpha` is `false`.
- `int getRGB()`
  returns the ARGB color value corresponding to this color.

12.6.5. Filtering Images

In the preceding section, you saw how to build up an image from scratch. However, often you want to access image data for a different reason: You already have an image and you want to improve it in some way.

Of course, you can use the `getPixel/getDataElements` methods that you saw in the preceding section to read the image data, manipulate them, and write them back. Fortunately, the Java 2D API already supplies a number of *filters* that carry out common image processing operations for you.

The image manipulations all implement the `BufferedImageOp` interface. After you construct the operation, simply call the `filter` method to transform an image into another.

```
BufferedImageOp op = . . .;
BufferedImage filteredImage
    = new BufferedImage(image.getWidth(), image.getHeight(), image.getType());
op.filter(image, filteredImage);
```

Some operations can transform an image in place (`op.filter(image, image)`), but most can't.

Five classes implement the `BufferedImageOp` interface:

```
AffineTransformOp
RescaleOp
LookupOp
ColorConvertOp
ConvolveOp
```

The `AffineTransformOp` carries out an affine transformation on the pixels. For example, here is how you can rotate an image about its center:

```
AffineTransform transform = AffineTransform.getRotateInstance(Math.toRadians(angle),
    image.getWidth() / 2, image.getHeight() / 2);
var op = new AffineTransformOp(transform, interpolation);
op.filter(image, filteredImage);
```

The `AffineTransformOp` constructor requires an affine transform and an *interpolation* strategy. Interpolation is necessary to determine the target image pixels if the source pixels are transformed somewhere between target pixels. For example, if you rotate source pixels, they will generally not fall exactly onto target pixels. There are three interpolation strategies: `AffineTransformOp.TYPE_BICUBIC`, `AffineTransformOp.TYPE_BILINEAR`, and `AffineTransformOp.TYPE_NEAREST_NEIGHBOR`. Bicubic interpolation takes a bit longer but looks better than the other two.

The program in **Listing 12.22** lets you rotate an image by 5 degrees (see **Figure 12.57**).

Figure 12.57: A rotated image

The `RescaleOp` carries out a rescaling operation

$x_{\mathbf{new}} = a \cdot x + b$

for each of the color components in the image. (Alpha components are not affected.) The effect of rescaling with $a > 1$ is to brighten the image. Construct the `RescaleOp` by specifying the scaling parameters and optional rendering hints. In **Listing 12.22**, we use:

```
float a = 1.1f;
float b = 20.0f;
var op = new RescaleOp(a, b, null);
```

You can also supply separate scaling values for each color component—see the API notes.

The `LookupOp` operation lets you specify an arbitrary mapping of sample values. Supply a table that specifies how each value should be mapped. In the example program, we compute the *negative* of all colors, changing the color c to $255 - c$.

The `LookupOp` constructor requires an object of type `LookupTable` and a map of optional hints. The `LookupTable` class is abstract, with two concrete subclasses: `ByteLookupTable` and `ShortLookupTable`. Since RGB color values are bytes, a `ByteLookupTable` should suffice. However, because of the bug described in

https://bugs.java.com/bugdatabase/view_bug.do?bug_id=6183251, we will use a `ShortLookupTable` instead. Here is how we construct the `LookupOp` for the example program:

```
var negative = new short[256];
for (int i = 0; i < 256; i++) negative[i] = (short) (255 - i);
var table = new ShortLookupTable(0, negative);
var op = new LookupOp(table, null);
```

The lookup is applied to each color component separately, but not to the alpha component. You can also supply different lookup tables for each color component—see the API notes.



Note: You cannot apply a `LookupOp` to an image with an indexed color model. (In those images, each sample value is an offset into a color palette.)

The `ColorConvertOp` is useful for color space conversions. We do not discuss it here.

The most powerful of the transformations is the `ConvolveOp`, which carries out a mathematical *convolution*. Without getting too deeply into the mathematical details, the basic idea is simple. Consider, for example, the *blur filter* (see Figure 12.58).

Figure 12.58: Blurring an image

The blurring is achieved by replacing each pixel with the *average* value from the pixel and its eight neighbors. Intuitively, it makes sense why this operation would blur out the picture. Mathematically, the averaging can be expressed as a convolution operation with the following *kernel:*

$$\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

The kernel of a convolution is a matrix that tells what weights should be applied to the neighboring values. The kernel above produces a blurred image. A different kernel carries out *edge detection,* locating the areas of color changes:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Edge detection is an important technique for analyzing photographic images (see **Figure 12.59**).

Figure 12.59: Edge detection and inversion

To construct a convolution operation, first set up an array of the values for the kernel and construct a `Kernel` object. Then, construct a `ConvolveOp` object from the kernel and use it for filtering.

```
float[] elements = {
    0.0f, -1.0f, 0.0f,
    -1.0f,  4.f, -1.0f,
    0.0f, -1.0f, 0.0f
};
var kernel = new Kernel(3, 3, elements);
var op = new ConvolveOp(kernel);
op.filter(image, filteredImage);
```

The program in **Listing 12.22** allows a user to load in a GIF or JPEG image and carry out the image manipulations that we discussed. Thanks to the power of the operations provided by Java 2D API, the program is very simple.

Listing 12.22 v2ch12/imageProcessing/ImageProcessingFrame.java

```
1  package v2ch12.imageProcessing;
2
3  import module java.base;
4  import module java.desktop;
5
6  /**
7   * This frame has a menu to load an image and to specify various transformations, and a
8   * component to show the resulting image.
9   */
10 public class ImageProcessingFrame extends JFrame {
11     private static final int DEFAULT_WIDTH = 400;
12     private static final int DEFAULT_HEIGHT = 400;
13
```

```
    private BufferedImage image;

    public ImageProcessingFrame() {
        setTitle("ImageProcessingTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        add(new JComponent() {
            public void paintComponent(Graphics g) {
                if (image != null) g.drawImage(image, 0, 0, null);
            }
        });

        var fileMenu = new JMenu("File");
        var openItem = new JMenuItem("Open");
        openItem.addActionListener(_ -> openFile());
        fileMenu.add(openItem);

        var exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(_ -> System.exit(0));
        fileMenu.add(exitItem);

        var editMenu = new JMenu("Edit");
        var blurItem = new JMenuItem("Blur");
        blurItem.addActionListener(_ -> {
            float weight = 1.0f / 9.0f;
            float[] elements = new float[9];
            for (int i = 0; i < 9; i++)
                elements[i] = weight;
            convolve(elements);
        });
        editMenu.add(blurItem);

        var sharpenItem = new JMenuItem("Sharpen");
        sharpenItem.addActionListener(_ -> {
            float[] elements = { 0.0f, -1.0f, 0.0f, -1.0f, 5.f, -1.0f, 0.0f, -1.0f, 0.0f };
            convolve(elements);
        });
        editMenu.add(sharpenItem);

        var brightenItem = new JMenuItem("Brighten");
        brightenItem.addActionListener(_ -> {
            float a = 1.1f;
            float b = 20.0f;
            var op = new RescaleOp(a, b, null);
            filter(op);
        });
        editMenu.add(brightenItem);

        var edgeDetectItem = new JMenuItem("Edge detect");
        edgeDetectItem.addActionListener(_ -> {
            float[] elements = { 0.0f, -1.0f, 0.0f, -1.0f, 4.f, -1.0f, 0.0f, -1.0f, 0.0f };
            convolve(elements);
        });
        editMenu.add(edgeDetectItem);

        var negativeItem = new JMenuItem("Negative");
        negativeItem.addActionListener(_ -> {
            short[] negative = new short[256 * 1];
```

```
            for (int i = 0; i < 256; i++)
                negative[i] = (short) (255 - i);
            var table = new ShortLookupTable(0, negative);
            var op = new LookupOp(table, null);
            filter(op);
        });
        editMenu.add(negativeItem);

        var rotateItem = new JMenuItem("Rotate");
        rotateItem.addActionListener(_ -> {
            if (image == null) return;
            var transform = AffineTransform.getRotateInstance(Math.toRadians(5),
                    image.getWidth() / 2, image.getHeight() / 2);
            var op = new AffineTransformOp(transform, AffineTransformOp.TYPE_BICUBIC);
            filter(op);
        });
        editMenu.add(rotateItem);

        var menuBar = new JMenuBar();
        menuBar.add(fileMenu);
        menuBar.add(editMenu);
        setJMenuBar(menuBar);
    }

    /**
     * Open a file and load the image.
     */
    public void openFile() {
        var chooser = new JFileChooser("images");
        chooser.setCurrentDirectory(new File(getClass().getPackage().getName()));
        String[] extensions = ImageIO.getReaderFileSuffixes();
        chooser.setFileFilter(new FileNameExtensionFilter("Image files", extensions));
        int r = chooser.showOpenDialog(this);
        if (r != JFileChooser.APPROVE_OPTION) return;

        try {
            Image img = ImageIO.read(chooser.getSelectedFile());
            image = new BufferedImage(img.getWidth(null), img.getHeight(null),
                    BufferedImage.TYPE_INT_RGB);
            image.getGraphics().drawImage(img, 0, 0, null);
        }
        catch (IOException e) {
            JOptionPane.showMessageDialog(this, e);
        }
        repaint();
    }

    /**
     * Apply a filter and repaint.
     * @param op the image operation to apply
     */
    private void filter(BufferedImageOp op) {
        if (image == null) return;
        image = op.filter(image, null);
        repaint();
    }

    /**
```

```
130       * Apply a convolution and repaint.
131       * @param elements the convolution kernel (an array of 9 matrix elements)
132       */
133      private void convolve(float[] elements) {
134          var kernel = new Kernel(3, 3, elements);
135          var op = new ConvolveOp(kernel);
136          filter(op);
137      }
138  }
```

java.awt.image.BufferedImageOp 1.2

- `BufferedImage filter(BufferedImage source, BufferedImage dest)`
  applies the image operation to the source image and stores the result in the destination image. If `dest` is `null`, a new destination image is created. The destination image is returned.

`java.awt.image.AffineTransformOp` 1.2

- `AffineTransformOp(AffineTransform t, int interpolationType)`
  constructs an affine transform operator. The interpolation type is one of `TYPE_BILINEAR`, `TYPE_BICUBIC`, or `TYPE_NEAREST_NEIGHBOR`.

`java.awt.image.RescaleOp` 1.2

- `RescaleOp(float a, float b, RenderingHints hints)`
- `RescaleOp(float[] as, float[] bs, RenderingHints hints)`
  construct a rescale operator that carries out the scaling operation $x_{\mathbf{new}} = a \cdot x + b$. When using the first constructor, all color components (but not the alpha component) are scaled with the same coefficients. When using the second constructor, you supply either the values for each color component, in which case the alpha component is unaffected, or the values for both alpha and color components.

`java.awt.image.LookupOp` 1.2

- `LookupOp(LookupTable table, RenderingHints hints)`
  constructs a lookup operator for the given lookup table.

`java.awt.image.ByteLookupTable` 1.2

- `ByteLookupTable(int offset, byte[] data)`
- `ByteLookupTable(int offset, byte[][] data)`
  construct a lookup table for converting `byte` values. The offset is subtracted from the input before the lookup. The values in the first constructor are applied to all color components but not the alpha component. When using the second constructor, supply either the values for each color component, in which case the alpha component is unaffected, or the values for both alpha and color components.

`java.awt.image.ShortLookupTable` 1.2

- `ShortLookupTable(int offset, short[] data)`
- `ShortLookupTable(int offset, short[][] data)`
  construct a lookup table for converting `short` values. The offset is subtracted from the input before the lookup. The values in the first constructor are applied to all color components but not the alpha component. When using the second constructor, supply either the values for each color component, in which case the alpha component is unaffected, or the values for both alpha and color components.

`java.awt.image.ConvolveOp` 1.2

- `ConvolveOp(Kernel kernel)`
- `ConvolveOp(Kernel kernel, int edgeCondition, RenderingHints hints)`
  construct a convolution operator. The edge condition specified is one of `EDGE_NO_OP` and `EDGE_ZERO_FILL`. Edge values need to be treated specially because they don't have sufficient neighboring values to compute the convolution. The default is `EDGE_ZERO_FILL`.

`java.awt.image.Kernel` 1.2

- `Kernel(int width, int height, float[] matrixElements)`
  constructs a kernel for the given matrix.

12.7. Printing

In the following sections, I'll show you how you can easily print a drawing on a single sheet of paper, how you can manage a multipage printout, and how you can save a printout as a PostScript file.

12.7.1. Graphics Printing

In this section, we will tackle what is probably the most common printing situation: printing a 2D graphic. Of course, the graphic can contain text in various fonts or even consist entirely of text.

To generate a printout, you have to take care of these two tasks:

- Supply an object that implements the `Printable` interface
- Start a print job

The `Printable` interface has a single method:

```
int print(Graphics g, PageFormat format, int page)
```

That method is called whenever the print engine needs to have a page formatted for printing. Your code draws the text and the images to be printed onto the graphics context. The page format tells you the paper size and the print margins. The page number tells you which page to render.

To start a print job, use the `PrinterJob` class. First, call the static `getPrinterJob` method to get a print job object. Then set the `Printable` object that you want to print.

```
Printable canvas = . . .;
PrinterJob job = PrinterJob.getPrinterJob();
job.setPrintable(canvas);
```



Caution: The class `PrintJob` handles JDK 1.1-style printing. That class is now obsolete. Do not confuse it with the `PrinterJob` class.

Before starting the print job, you should call the `printDialog` method to display a print dialog box (see **Figure 12.60**). That dialog box gives the user a chance to select the printer to be used (in case multiple printers are available), the page range that should be printed, and various printer settings.

Figure 12.60: A cross-platform print dialog box

Collect printer settings in an object of a class that implements the `PrintRequestAttributeSet` interface, such as the `HashPrintRequestAttributeSet` class.

```
var attributes = new HashPrintRequestAttributeSet();
```

Add attribute settings and pass the `attributes` object to the `printDialog` method.

The `printDialog` method returns `true` if the user clicked OK and `false` if the user canceled the dialog box. If the user accepted, call the `print` method of the `PrinterJob` class to start the printing process. The `print` method might throw a `PrinterException`. Here is the outline of the printing code:

```
if (job.printDialog(attributes)) {
    try {
        job.print(attributes);
    }
    catch (PrinterException e) {
```

```
         . . .
      }
   }
```



Note: Prior to JDK 1.4, the printing system used the native print and page setup dialog boxes of the host platform. To show a native print dialog box, call the `printDialog` method with no parameters. (There is no way to collect user settings in an attribute set.)

During printing, the `print` method of the `PrinterJob` class makes repeated calls to the `print` method of the `Printable` object associated with the job.

Since the job does not know how many pages you want to print, it simply keeps calling the `print` method. As long as the `print` method returns the value `Printable.PAGE_EXISTS`, the print job keeps producing pages. When the `print` method returns `Printable.NO_SUCH_PAGE`, the print job stops.



Caution: The page numbers that the print job passes to the `print` method start with page 0.

Therefore, the print job doesn't have an accurate page count until after the printout is complete. For that reason, the print dialog box can't display the correct page range—instead, it displays "Pages 1 to 1." You will see in the next section how to avoid this blemish by supplying a `Book` object to the print job.

During the printing process, the print job repeatedly calls the `print` method of the `Printable` object. The print job is allowed to make multiple calls *for the same page.* You should therefore not count pages inside the `print` method but always rely on the page number parameter. There is a good reason why the print job might call the `print` method repeatedly for the same page. Some printers, in particular dot-matrix and inkjet printers, use *banding.* They print one band at a time, advance the paper, and then print the next band. The print job might use banding even for laser printers that print a full page at a time—it gives the print job a way of managing the size of the spool file.

If the print job needs the `Printable` object to print a band, it sets the clip area of the graphics context to the requested band and calls the `print` method. Its drawing operations are clipped against the band rectangle, and only those drawing elements that show up in the band are rendered. Your `print` method need not be aware of that process, with one caveat: It should *not* interfere with the clip area.



Caution: The `Graphics` object that your `print` method gets is also clipped against the page margins. If you replace the clip area, you can draw outside the margins. Especially in a printer graphics context, the clipping area must be respected. Call `clip`, not `setClip`, to further restrict the clipping area. If you must remove a clip area, make sure to call `getClip` at the beginning of your `print` method and restore that clip area.

The `PageFormat` parameter of the `print` method contains information about the printed page. The methods `getWidth` and `getHeight` return the paper size, measured in *points.* (One point is 1/72 of an inch; an inch equals 25.4 millimeters.) For example, A4 paper is approximately 595 × 842 points, and US Letter paper is 612 × 792 points.

Points are a common measurement in the printing trade in the United States. Much to the chagrin of the rest of the world, the printing package uses point units. There are two purposes for that: paper sizes and paper margins are measured in points, and points are the default unit for all print graphics contexts. You can verify that in the example program at the end of this section. The program prints two lines of text that are 72 units apart. Run the example program and measure the distance between the baselines; they are exactly 1 inch or 25.4 millimeters apart.

The `getWidth` and `getHeight` methods of the `PageFormat` class give you the complete paper size. Not all of the paper area is printable. Users typically select margins, and even if they don't, printers need to somehow grip the sheets of paper on which they print and therefore have a small unprintable area around the edges.

The methods `getImageableWidth` and `getImageableHeight` tell you the dimensions of the area that you can actually fill. However, the margins need not be symmetrical, so you must also know the top left corner of the imageable area (see **Figure 12.61**), which you obtain by the methods `getImageableX` and `getImageableY`.

Figure 12.61: Page format measurements



Tip: The graphics context that you receive in the `print` method is clipped to exclude the margins, but the origin of the coordinate system is nevertheless the top left corner of the paper. It makes sense to translate the coordinate system to the top left corner of the imageable area. Simply start your `print` method with

```
g.translate(pageFormat.getImageableX(), pageFormat.getImageableY());
```

If you want your users to choose the settings for the page margins or to switch between portrait and landscape orientation without setting other printing attributes, call the `pageDialog` method of the `PrinterJob` class:

```
PageFormat format = job.pageDialog(attributes);
```



Note: One of the tabs of the print dialog box contains the page setup dialog (see **Figure 12.62**). You might still want to give users an option to set the page format before printing, especially if your program presents a "what you see is what you get" display of the pages to be printed. The `pageDialog` method returns a `PageFormat` object with the user settings.

Figure 12.62: A cross-platform page setup dialog

The program at the end of this section shows how to render the same set of shapes on the screen and on the printed page. A subclass of `JPanel` implements the `Printable` interface. Both the `paintComponent` and the `print` methods call the same method to carry out the actual drawing.

```
class PrintPanel extends JPanel implements Printable {
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        var g2 = (Graphics2D) g;
        drawPage(g2);
    }

    public int print(Graphics g, PageFormat pf, int page) throws PrinterException {
        if (page >= 1) return Printable.NO_SUCH_PAGE;
        var g2 = (Graphics2D) g;
        g2.translate(pf.getImageableX(), pf.getImageableY());
        drawPage(g2);
        return Printable.PAGE_EXISTS;
    }

    public void drawPage(Graphics2D g2) {
        // shared drawing code goes here
        . . .
```

```
    }
    . . .
}
```

This example displays and prints the image shown in **Figure 12.50**—namely, the outline of the message “Hello, World” used as a clipping area for a pattern of lines.

Click the Print button to start printing, or click the Page setup button to open the page setup dialog box. **Listing 12.23** shows the code.



Note: To show a native page setup dialog box, pass a default `PageFormat` object to the `pageDialog` method. The method clones that object, modifies it according to the user selections in the dialog box, and returns the cloned object.

```
PageFormat defaultFormat = printJob.defaultPage();
PageFormat selectedFormat = printJob.pageDialog(defaultFormat);
```

Listing 12.23 v2ch12/print/PrintDemoFrame.java

```
1  package v2ch12.print;
2
3  import module java.desktop;
4
5  /**
6   * This frame shows a panel with 2D graphics and buttons to print the graphics and to set up
7   * the page format.
8   */
9  public class PrintDemoFrame extends JFrame {
10     private PrintComponent canvas;
11     private PrintRequestAttributeSet attributes;
12
13     public PrintDemoFrame() {
14         canvas = new PrintComponent();
15         add(canvas, BorderLayout.CENTER);
16
17         attributes = new HashPrintRequestAttributeSet();
18
19         var buttonPanel = new JPanel();
20         var printButton = new JButton("Print");
21         buttonPanel.add(printButton);
22         printButton.addActionListener(_ -> {
23             try {
24                 PrinterJob job = PrinterJob.getPrinterJob();
25                 job.setPrintable(canvas);
26                 if (job.printDialog(attributes)) job.print(attributes);
27             }
28             catch (PrinterException e) {
29                 JOptionPane.showMessageDialog(PrintDemoFrame.this, e);
30             }
31         });
32
33         var pageSetupButton = new JButton("Page setup");
34         buttonPanel.add(pageSetupButton);
35         pageSetupButton.addActionListener(_ -> {
```

```
            PrinterJob job = PrinterJob.getPrinterJob();
            job.pageDialog(attributes);
        });

        add(buttonPanel, BorderLayout.NORTH);
        pack();
    }
}
```

Listing 12.24 v2ch12/print/PrintComponent.java

```
package v2ch12.print;

import module java.desktop;

/**
 * This component generates a 2D graphics image for screen display and printing.
 */
public class PrintComponent extends JComponent implements Printable {
    private static final Dimension PREFERRED_SIZE = new Dimension(300, 300);

    public void paintComponent(Graphics g) {
        var g2 = (Graphics2D) g;
        drawPage(g2);
    }

    public int print(Graphics g, PageFormat pf, int page) throws PrinterException {
        if (page >= 1) return Printable.NO_SUCH_PAGE;
        var g2 = (Graphics2D) g;
        g2.translate(pf.getImageableX(), pf.getImageableY());
        g2.draw(new Rectangle2D.Double(0, 0, pf.getImageableWidth(), pf.getImageableHeight()));

        drawPage(g2);
        return Printable.PAGE_EXISTS;
    }

    /**
     * This method draws the page both on the screen and the printer graphics context.
     * @param g2 the graphics context
     */
    public void drawPage(Graphics2D g2) {
        FontRenderContext context = g2.getFontRenderContext();
        var f = new Font("Serif", Font.PLAIN, 72);
        var clipShape = new GeneralPath();

        var layout = new TextLayout("Hello", f, context);
        AffineTransform transform = AffineTransform.getTranslateInstance(0, 72);
        Shape outline = layout.getOutline(transform);
        clipShape.append(outline, false);

        layout = new TextLayout("World", f, context);
        transform = AffineTransform.getTranslateInstance(0, 144);
        outline = layout.getOutline(transform);
        clipShape.append(outline, false);

        g2.draw(clipShape);
        g2.clip(clipShape);
```

```
47
48          final int NLINES = 50;
49          var p = new Point2D.Double(0, 0);
50          for (int i = 0; i < NLINES; i++) {
51              double x = (2 * getWidth() * i) / NLINES;
52              double y = (2 * getHeight() * (NLINES - 1 - i)) / NLINES;
53              var q = new Point2D.Double(x, y);
54              g2.draw(new Line2D.Double(p, q));
55          }
56      }
57
58      public Dimension getPreferredSize() {
59          return PREFERRED_SIZE;
60      }
61  }
```

java.awt.print.Printable **1.2**

- `int print(Graphics g, PageFormat format, int pageNumber)`
  renders a page and returns `PAGE_EXISTS`, or returns `NO_SUCH_PAGE`.

`java.awt.print.PrinterJob` **1.2**

- `static PrinterJob getPrinterJob()`
  returns a printer job object.
- `PageFormat defaultPage()`
  returns the default page format for this printer.
- `boolean printDialog(PrintRequestAttributeSet attributes)`
- `boolean printDialog()`
  open a print dialog box to allow a user to select the pages to be printed and to change print settings. The first method displays a cross-platform dialog box, the second a native dialog box. The first method modifies the `attributes` object to reflect the user settings. Both methods return `true` if the user accepts the dialog box.
- `PageFormat pageDialog(PrintRequestAttributeSet attributes)`
- `PageFormat pageDialog(PageFormat defaults)`
  display a page setup dialog box. The first method displays a cross-platform dialog box, the second a native dialog box. Both methods return a `PageFormat` object with the format that the user requested in the dialog box. The first method modifies the `attributes` object to reflect the user settings. The second method does not modify the `defaults` object.
- `void setPrintable(Printable p)`
- `void setPrintable(Printable p, PageFormat format)`
  set the `Printable` of this print job and an optional page format.
- `void print()`
- `void print(PrintRequestAttributeSet attributes)`
  print the current `Printable` by repeatedly calling its `print` method and sending the rendered pages to the printer, until no more pages are available.

`java.awt.print.PageFormat` 1.2

- `double getWidth()`
- `double getHeight()`
  return the width and height of the page.
- `double getImageableWidth()`
- `double getImageableHeight()`
  return the width and height of the imageable area of the page.
- `double getImageableX()`
- `double getImageableY()`
  return the position of the top left corner of the imageable area.
- `int getOrientation()`
  returns one of `PORTRAIT`, `LANDSCAPE`, or `REVERSE_LANDSCAPE`. Page orientation is transparent to programmers because the page format and graphics context settings automatically reflect the page orientation.

12.7.2. Multiple-Page Printing

In practice, you usually don't pass a raw `Printable` object to a print job. Instead, you should obtain an object of a class that implements the `Pageable` interface. The Java platform supplies one such class, called `Book`. A book is made up of sections, each of which is a `Printable` object. To make a book, add `Printable` objects and their page counts.

```
var book = new Book();
Printable coverPage = . . .;
Printable bodyPages = . . .;
book.append(coverPage, pageFormat); // append 1 page
book.append(bodyPages, pageFormat, pageCount);
```

Then, use the `setPageable` method to pass the `Book` object to the print job.

```
printJob.setPageable(book);
```

Now the print job knows exactly how many pages to print, so the print dialog box displays an accurate page range and the user can select the entire range or subranges.



Caution: When the print job calls the `print` methods of the `Printable` sections, it passes the current page number of the *book*, and not of each *section*, as the current page number. That is a huge pain—each section must know the page counts of the preceding sections to make sense of the page number parameter.

From your perspective as a programmer, the biggest challenge of using the `Book` class is that you must know how many pages each section will have when you print it. Your `Printable` class needs a *layout algorithm* that computes the layout of the material on the printed pages. Before printing starts, invoke that algorithm to compute the page breaks and the page count. You can retain the layout information so you have it handy during the printing process.

You must guard against the possibility that the user has changed the page format. If that happens, you must recompute the layout, even if the information that you want to print has not changed.

Listing 12.26 shows how to produce a multipage printout. This program prints a message in very large characters on a number of pages (see Figure 12.63). You can then trim the margins and tape the pages together to form a banner.

Figure 12.63: A banner

The `layoutPages` method of the `Banner` class computes the layout. We first lay out the message string in a 72-point font. We then compute the height of the resulting string and compare it with the imageable height of the page. We derive a scale factor from these two measurements. When printing the string, we magnify it by that scale factor.



Caution: To lay out your information precisely, you usually need access to the printer graphics context. Unfortunately, there is no way to obtain that graphics context before printing actually starts. In our example program, we make do with the screen graphics context and hope that the font metrics of the screen and printer match.

The `getPageCount` method of the `Banner` class first calls the layout method. Then it scales up the width of the string and divides it by the imageable width of each page. The quotient, rounded up to the next integer, is the page count.

It sounds like it might be difficult to print the banner because characters can be broken across multiple pages. However, thanks to the power of the Java 2D API, this turns out not to be a problem at all. When a particular page is requested, we simply use the `translate` method of the `Graphics2D` class to shift the top left corner of the string to the left. Then, we set a clip rectangle that equals the current page (see Figure 12.64). Finally, we scale the graphics context with the scale factor that the layout method computed.

Figure 12.64: Printing a page of a banner

This example shows the power of transformations. The drawing code is kept simple, and the transformation does all the work of placing the drawing in the appropriate place. Finally, the clip cuts away the part of the image that falls outside the page. This program shows another compelling use of transformations—to display a print preview.

Listing 12.25 v2ch12/book/BookDemoFrame.java

```
package v2ch12.book;

import module java.desktop;

/**
 * This frame has a text field for the banner text and buttons for printing, page setup, and
 * print preview.
 */
public class BookDemoFrame extends JFrame {
    private JTextField text;
    private PageFormat pageFormat;
    private PrintRequestAttributeSet attributes;

    public BookDemoFrame() {
        text = new JTextField();
        add(text, BorderLayout.NORTH);

        attributes = new HashPrintRequestAttributeSet();

        var buttonPanel = new JPanel();

        var printButton = new JButton("Print");
        buttonPanel.add(printButton);
        printButton.addActionListener(_ -> {
            try {
                PrinterJob job = PrinterJob.getPrinterJob();
                job.setPageable(makeBook());
                if (job.printDialog(attributes)) {
                    job.print(attributes);
                }
            }
            catch (PrinterException e) {
                JOptionPane.showMessageDialog(BookDemoFrame.this, e);
            }
        });

        var pageSetupButton = new JButton("Page setup");
        buttonPanel.add(pageSetupButton);
        pageSetupButton.addActionListener(_ -> {
            PrinterJob job = PrinterJob.getPrinterJob();
            pageFormat = job.pageDialog(attributes);
        });

        var printPreviewButton = new JButton("Print preview");
        buttonPanel.add(printPreviewButton);
        printPreviewButton.addActionListener(_ -> {
            var dialog = new PrintPreviewDialog(makeBook());
            dialog.setVisible(true);
        });

```

```
        add(buttonPanel, BorderLayout.SOUTH);
        pack();
    }

    /**
     * Makes a book that contains a cover page and the pages for the banner.
     */
    public Book makeBook() {
        if (pageFormat == null) {
            PrinterJob job = PrinterJob.getPrinterJob();
            pageFormat = job.defaultPage();
        }
        var book = new Book();
        String message = text.getText();
        var banner = new Banner(message);
        int pageCount = banner.getPageCount((Graphics2D) getGraphics(), pageFormat);
        book.append(new CoverPage(message + " (" + pageCount + " pages)"), pageFormat);
        book.append(banner, pageFormat, pageCount);
        return book;
    }
}
```

Listing 12.26 v2ch12/book/Banner.java

```
package v2ch12.book;

import module java.desktop;

/**
 * A banner that prints a text string on multiple pages.
 */
public class Banner implements Printable {
    private String message;
    private double scale;

    /**
     * Constructs a banner.
     * @param m the message string
     */
    public Banner(String m) {
        message = m;
    }

    /**
     * Gets the page count of this section.
     * @param g2 the graphics context
     * @param pf the page format
     * @return the number of pages needed
     */
    public int getPageCount(Graphics2D g2, PageFormat pf) {
        if (message.equals("")) return 0;
        FontRenderContext context = g2.getFontRenderContext();
        var f = new Font("Serif", Font.PLAIN, 72);
        Rectangle2D bounds = f.getStringBounds(message, context);
        scale = pf.getImageableHeight() / bounds.getHeight();
        double width = scale * bounds.getWidth();
        int pages = (int) Math.ceil(width / pf.getImageableWidth());
```

```
        return pages;
    }

    public int print(Graphics g, PageFormat pf, int page) throws PrinterException {
        var g2 = (Graphics2D) g;
        if (page > getPageCount(g2, pf)) return Printable.NO_SUCH_PAGE;
        g2.translate(pf.getImageableX(), pf.getImageableY());

        drawPage(g2, pf, page);
        return Printable.PAGE_EXISTS;
    }

    public void drawPage(Graphics2D g2, PageFormat pf, int page) {
        if (message.equals("")) return;
        page--; // account for cover page

        drawCropMarks(g2, pf);
        g2.clip(new Rectangle2D.Double(0, 0, pf.getImageableWidth(), pf.getImageableHeight()));
        g2.translate(-page * pf.getImageableWidth(), 0);
        g2.scale(scale, scale);
        FontRenderContext context = g2.getFontRenderContext();
        var f = new Font("Serif", Font.PLAIN, 72);
        var layout = new TextLayout(message, f, context);
        AffineTransform transform
            = AffineTransform.getTranslateInstance(0, layout.getAscent());
        Shape outline = layout.getOutline(transform);
        g2.draw(outline);
    }

    /**
     * Draws 1/2" crop marks in the corners of the page.
     * @param g2 the graphics context
     * @param pf the page format
     */
    public void drawCropMarks(Graphics2D g2, PageFormat pf) {
        final double C = 36; // crop mark length = 1/2 inch
        double w = pf.getImageableWidth();
        double h = pf.getImageableHeight();
        g2.draw(new Line2D.Double(0, 0, 0, C));
        g2.draw(new Line2D.Double(0, 0, C, 0));
        g2.draw(new Line2D.Double(w, 0, w, C));
        g2.draw(new Line2D.Double(w, 0, w - C, 0));
        g2.draw(new Line2D.Double(0, h, 0, h - C));
        g2.draw(new Line2D.Double(0, h, C, h));
        g2.draw(new Line2D.Double(w, h, w, h - C));
        g2.draw(new Line2D.Double(w, h, w - C, h));
    }
}

/**
 * This class prints a cover page with a title.
 */
class CoverPage implements Printable {
    private String title;

    /**
     * Constructs a cover page.
     * @param t the title
```

```
     */
    public CoverPage(String t) {
        title = t;
    }

    public int print(Graphics g, PageFormat pf, int page) throws PrinterException {
        if (page >= 1) return Printable.NO_SUCH_PAGE;
        var g2 = (Graphics2D) g;
        g2.setPaint(Color.black);
        g2.translate(pf.getImageableX(), pf.getImageableY());
        FontRenderContext context = g2.getFontRenderContext();
        Font f = g2.getFont();
        var layout = new TextLayout(title, f, context);
        float ascent = layout.getAscent();
        g2.drawString(title, 0, ascent);
        return Printable.PAGE_EXISTS;
    }
}
```

Listing 12.27 v2ch12/book/PrintPreviewDialog.java

```
package v2ch12.book;

import module java.desktop;

/**
 * This class implements a generic print preview dialog.
 */
public class PrintPreviewDialog extends JDialog {
    private static final int DEFAULT_WIDTH = 300;
    private static final int DEFAULT_HEIGHT = 300;

    private PrintPreviewCanvas canvas;

    /**
     * Constructs a print preview dialog.
     * @param p a Printable
     * @param pf the page format
     * @param pages the number of pages in p
     */
    public PrintPreviewDialog(Printable p, PageFormat pf, int pages) {
        var book = new Book();
        book.append(p, pf, pages);
        layoutUI(book);
    }

    /**
     * Constructs a print preview dialog.
     * @param b a Book
     */
    public PrintPreviewDialog(Book b) {
        layoutUI(b);
    }

    /**
     * Lays out the UI of the dialog.
     * @param book the book to be previewed
```

```
     */
    public void layoutUI(Book book) {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        canvas = new PrintPreviewCanvas(book);
        add(canvas, BorderLayout.CENTER);

        var buttonPanel = new JPanel();

        var nextButton = new JButton("Next");
        buttonPanel.add(nextButton);
        nextButton.addActionListener(_ -> canvas.flipPage(1));

        var previousButton = new JButton("Previous");
        buttonPanel.add(previousButton);
        previousButton.addActionListener(_ -> canvas.flipPage(-1));

        var closeButton = new JButton("Close");
        buttonPanel.add(closeButton);
        closeButton.addActionListener(_ -> setVisible(false));

        add(buttonPanel, BorderLayout.SOUTH);
    }
}
```

Listing 12.28 v2ch12/book/PrintPreviewCanvas.java

```
package v2ch12.book;

import module java.desktop;

/**
 * The canvas for displaying the print preview.
 */
public class PrintPreviewCanvas extends JComponent {
    private Book book;
    private int currentPage;

    /**
     * Constructs a print preview canvas.
     * @param b the book to be previewed
     */
    public PrintPreviewCanvas(Book b) {
        book = b;
        currentPage = 0;
    }

    public void paintComponent(Graphics g) {
        var g2 = (Graphics2D) g;
        PageFormat pageFormat = book.getPageFormat(currentPage);

        double xoff; // x offset of page start in window
        double yoff; // y offset of page start in window
        double scale; // scale factor to fit page in window
        double px = pageFormat.getWidth();
        double py = pageFormat.getHeight();
        double sx = getWidth() - 1;
```

```
        double sy = getHeight() - 1;
        if (px / py < sx / sy) // center horizontally
        {
            scale = sy / py;
            xoff = 0.5 * (sx - scale * px);
            yoff = 0;
        }
        else
        // center vertically
        {
            scale = sx / px;
            xoff = 0;
            yoff = 0.5 * (sy - scale * py);
        }
        g2.translate((float) xoff, (float) yoff);
        g2.scale((float) scale, (float) scale);

        // draw page outline (ignoring margins)
        var page = new Rectangle2D.Double(0, 0, px, py);
        g2.setPaint(Color.white);
        g2.fill(page);
        g2.setPaint(Color.black);
        g2.draw(page);

        Printable printable = book.getPrintable(currentPage);
        try {
            printable.print(g2, pageFormat, currentPage);
        }
        catch (PrinterException e) {
            g2.draw(new Line2D.Double(0, 0, px, py));
            g2.draw(new Line2D.Double(px, 0, 0, py));
        }
    }

    /**
     * Flip the book by the given number of pages.
     * @param by the number of pages to flip by. Negative values flip backwards.
     */
    public void flipPage(int by) {
        int newPage = currentPage + by;
        if (0 <= newPage && newPage < book.getNumberOfPages()) {
            currentPage = newPage;
            repaint();
        }
    }
}
```

12.7.3. Print Services

So far, you have seen how to print 2D graphics. However, the printing API affords far greater flexibility. The API defines a number of data types and lets you find print services that are able to print them. Among the data types are

- Images in GIF, JPEG, or PNG format
- Documents in text, HTML, PostScript, or PDF format
- Raw printer code data

- Objects of a class that implements `Printable`, `Pageable`, or `RenderableImage`

The data themselves can be stored in a source of bytes or characters such as an input stream, a URL, or an array. A *document flavor* describes the combination of a data source and a data type. The `DocFlavor` class defines a number of inner classes for the various data sources. Each of the inner classes defines constants to specify the flavors. For example, the constant

```
DocFlavor.INPUT_STREAM.GIF
```

describes a GIF image that is read from an input stream. **Table 12.4** lists the combinations.

Table 12.4: Document Flavors for Print Services

Data Source	Data Type	MIME Type
`INPUT_STREAM`	`GIF`	`image/gif`
`URL`	`JPEG`	`image/jpeg`
`BYTE_ARRAY`	`PNG`	`image/png`
	`POSTSCRIPT`	`application/postscript`
	`PDF`	`application/pdf`
	`TEXT_HTML_HOST`	`text/html` (using host encoding)
	`TEXT_HTML_US_ASCII`	`text/html; charset=us-ascii`
	`TEXT_HTML_UTF_8`	`text/html; charset=utf-8`
	`TEXT_HTML_UTF_16`	`text/html; charset=utf-16`
	`TEXT_HTML_UTF_16LE`	`text/html; charset=utf-16le` (little-endian)
	`TEXT_HTML_UTF_16BE`	`text/html; charset=utf-16be` (big-endian)
	`TEXT_PLAIN_HOST`	`text/plain` (using host encoding)
	`TEXT_PLAIN_US_ASCII`	`text/plain; charset=us-ascii`
	`TEXT_PLAIN_UTF_8`	`text/plain; charset=utf-8`
	`TEXT_PLAIN_UTF_16`	`text/plain; charset=utf-16`
	`TEXT_PLAIN_UTF_16LE`	`text/plain; charset=utf-16le` (little-endian)
	`TEXT_PLAIN_UTF_16BE`	`text/plain; charset=utf-16be` (big-endian)

Data Source	Data Type	MIME Type
	`PCL`	`application/vnd.hp-PCL` (Hewlett Packard Printer Control Language)
	`AUTOSENSE`	`application/octet-stream` (raw printer data)
`READER`	`TEXT_HTML`	`text/html; charset=utf-16`
`STRING`	`TEXT_PLAIN`	`text/plain; charset=utf-16`
`CHAR_ARRAY`		
`SERVICE_FORMATTED`	`PRINTABLE`	N/A
	`PAGEABLE`	N/A
	`RENDERABLE_IMAGE`	N/A

Suppose you want to print a GIF image located in a file. First, find out whether there is a *print service* that is capable of handling the task. The static `lookupPrintServices` method of the `PrintServiceLookup` class returns an array of `PrintService` objects that can handle the given document flavor.

```
DocFlavor flavor = DocFlavor.INPUT_STREAM.GIF;
PrintService[] services = PrintServiceLookup.lookupPrintServices(flavor, null);
```

The second argument of the `lookupPrintServices` method is `null` to indicate that we don't want to constrain the search by specifying printer attributes. We'll cover attributes in the next section.

If the lookup yields an array with more than one element, select from the listed print services. You can call the `getName` method of the `PrintService` class to get the printer names and let the user choose.

Next, get a document print job from the service:

```
DocPrintJob job = services[i].createPrintJob();
```

For printing, you need an object that implements the `Doc` interface. The Java library supplies a class `SimpleDoc` for that purpose. The `SimpleDoc` constructor requires the data source object, the document flavor, and an optional attribute set. For example,

```
var in = new FileInputStream(fileName);
var doc = new SimpleDoc(in, flavor, null);
```

Finally, you are ready to print:

```
job.print(doc, null);
```

As before, the `null` argument can be replaced by an attribute set.

Note that this printing process is quite different from that of the preceding section. There is no user interaction through print dialog boxes. For example, you can implement a server-side printing mechanism in which users submit print jobs through a web form.

`javax.print.PrintServiceLookup` 1.4

- `PrintService[] lookupPrintServices(DocFlavor flavor, AttributeSet attributes)`
  looks up the print services that can handle the given document flavor and attributes. The `attributes` parameter can be `null` if there are no attributes to consider.

javax.print.PrintService 1.4

- `DocPrintJob createPrintJob()`
  creates a print job for printing an object of a class that implements the `Doc` interface, such as a `SimpleDoc`.

javax.print.DocPrintJob 1.4

- `void print(Doc doc, PrintRequestAttributeSet attributes)`
  prints the given document with the given attributes. The `attributes` parameter can be `null` if no printing attributes are required.

`javax.print.SimpleDoc` 1.4

- `SimpleDoc(Object data, DocFlavor flavor, DocAttributeSet attributes)`
  constructs a `SimpleDoc` object that can be printed with a `DocPrintJob`. The `data` parameter is the object with the print data, such as an input stream or a `Printable`. The `attributes` parameter can be `null` if attributes are not required.

12.7.4. Stream Print Services

A print service sends print data to a printer. A stream print service generates the same print data but instead sends them to a stream, perhaps for delayed printing or because the print data format can be interpreted by other programs. In particular, if the print data format is PostScript, it may be useful to save the print data to a file because many programs can process PostScript files. The Java platform includes a stream print service that can produce PostScript output from images and 2D graphics. You can use that service on all systems, even if there are no local printers.

Enumerating stream print services is a bit more tedious than locating regular print services. You need both the `DocFlavor` of the object to be printed and the MIME type of the stream output. You then get a `StreamPrintServiceFactory` array of factories.

```
DocFlavor flavor = DocFlavor.SERVICE_FORMATTED.PRINTABLE;
String mimeType = "application/postscript";
StreamPrintServiceFactory[] factories
    = StreamPrintServiceFactory.lookupStreamPrintServiceFactories(flavor, mimeType);
```

The `StreamPrintServiceFactory` class has no methods that would help us distinguish any one factory from another, so we just take `factories[0]`. We call the `getPrintService` method with an output stream parameter to get a `StreamPrintService` object.

```
var out = new FileOutputStream(fileName);
StreamPrintService service = factories[0].getPrintService(out);
```

The `StreamPrintService` class is a subclass of `PrintService`. To produce a printout, simply follow the steps of the preceding section.

`javax.print.StreamPrintServiceFactory` 1.4

- `StreamPrintServiceFactory[] lookupStreamPrintServiceFactories(DocFlavor flavor, String mimeType)`
  looks up the stream print service factories that can print the given document flavor and produce an output stream of the given MIME type.
- `StreamPrintService getPrintService(OutputStream out)`
  gets a print service that sends the printing output to the given output stream.

The program in **Listing 12.29** demonstrates how to use a stream print service to print Java 2D shapes to a PostScript file. You can replace the sample drawing code with code that generates any Java 2D shapes and have the shapes converted to PostScript. Then you can easily convert the result to PDF or EPS, using an external tool. (Unfortunately, Java does not support printing to PDF directly.)



Note: In this example, we call a `draw` method that draws Java 2D shapes onto a `Graphics2D` object. If you want to draw the surface of a component (for example, a table or tree), use the following code:

```
private static int IMAGE_WIDTH = component.getWidth();
private static int IMAGE_HEIGHT = component.getHeight();
public static void draw(Graphics2D g2) { component.paint(g2); }
```

Listing 12.29 v2ch12/printService/PrintServiceDemo.java

```
1  package v2ch12.printService;
2
3  import module java.base;
4  import module java.desktop;
5
6  /**
7   * This program demonstrates the use of stream print services. The program prints Java 2D
8   * shapes to a PostScript file. If you don't supply a file name on the command line, the output
9   * is saved to out.ps.
10  */
11 class PrintServiceDemo {
12     // Set your image dimensions here
13     int IMAGE_WIDTH = 300;
14     int IMAGE_HEIGHT = 300;
15
16     void main(String[] args) throws Exception {
17         String fileName = args.length > 0 ? args[0] : "out.ps";
18         DocFlavor flavor = DocFlavor.SERVICE_FORMATTED.PRINTABLE;
```

```
        String mimeType = "application/postscript";
        StreamPrintServiceFactory[] factories = StreamPrintServiceFactory
                .lookupStreamPrintServiceFactories(flavor, mimeType);
        var out = new FileOutputStream(fileName);
        if (factories.length > 0) {
            PrintService service = factories[0].getPrintService(out);
            var doc = new SimpleDoc(new Printable() {
                public int print(Graphics g, PageFormat pf, int page) {
                    if (page >= 1)
                        return Printable.NO_SUCH_PAGE;
                    else {
                        double sf1 = pf.getImageableWidth() / (IMAGE_WIDTH + 1);
                        double sf2 = pf.getImageableHeight() / (IMAGE_HEIGHT + 1);
                        double s = Math.min(sf1, sf2);
                        var g2 = (Graphics2D) g;
                        g2.translate((pf.getWidth() - pf.getImageableWidth()) / 2,
                                (pf.getHeight() - pf.getImageableHeight()) / 2);
                        g2.scale(s, s);

                        draw(g2);
                        return Printable.PAGE_EXISTS;
                    }
                }
            }, flavor, null);
            DocPrintJob job = service.createPrintJob();
            var attributes = new HashPrintRequestAttributeSet();
            job.print(doc, attributes);
        }
        else
            IO.println("No factories for " + mimeType);
    }
    void draw(Graphics2D g2) {
        // Your drawing instructions go here
        FontRenderContext context = g2.getFontRenderContext();
        var f = new Font("Serif", Font.PLAIN, 72);
        var clipShape = new GeneralPath();

        var layout = new TextLayout("Hello", f, context);
        AffineTransform transform = AffineTransform.getTranslateInstance(0, 72);
        Shape outline = layout.getOutline(transform);
        clipShape.append(outline, false);

        layout = new TextLayout("World", f, context);
        transform = AffineTransform.getTranslateInstance(0, 144);
        outline = layout.getOutline(transform);
        clipShape.append(outline, false);

        g2.draw(clipShape);
        g2.clip(clipShape);

        final int NLINES = 50;
        var p = new Point2D.Double(0, 0);
        for (int i = 0; i < NLINES; i++) {
            double x = (2 * IMAGE_WIDTH * i) / NLINES;
            double y = (2 * IMAGE_HEIGHT * (NLINES - 1 - i)) / NLINES;
            var q = new Point2D.Double(x, y);
            g2.draw(new Line2D.Double(p, q));
```

```
76          }
77      }
78  }
```

12.7.5. Printing Attributes

The print service API contains a complex set of interfaces and classes to specify various kinds of attributes. There are four important groups of attributes. The first two specify requests to the printer.

- *Print request attributes* request particular features for all `doc` objects in a print job, such as two-sided printing or the paper size.
- *Doc attributes* are request attributes that apply only to a single `doc` object.

The other two attributes contain information about the printer and job status.

- *Print service attributes* give information about the print service, such as the printer make and model or whether the printer is currently accepting jobs.
- *Print job attributes* give information about the status of a particular print job, such as whether the job is already completed.

To describe the various attributes, there is an interface `Attribute` with subinterfaces:

```
PrintRequestAttribute
DocAttribute
PrintServiceAttribute
PrintJobAttribute
SupportedValuesAttribute
```

Individual attribute classes implement one or more of these interfaces. For example, objects of the `Copies` class describe the number of copies of a printout. That class implements both the `PrintRequestAttribute` and the `PrintJobAttribute` interfaces. Clearly, a print request can contain a request for multiple copies. Conversely, an attribute of the print job might be how many of these copies were actually printed. That number might be lower, perhaps because of printer limitations or because the printer ran out of paper.

The `SupportedValuesAttribute` interface indicates that an attribute value does not reflect actual request or status data but rather the capability of a service. For example, the `CopiesSupported` class implements the `SupportedValuesAttribute` interface. An object of that class might describe that a printer supports 1 through 99 copies of a printout.

Figure 12.65 shows a class diagram of the attribute hierarchy.

Figure 12.65: The attribute hierarchy

In addition to the interfaces and classes for individual attributes, the print service API defines interfaces and classes for attribute sets. A superinterface, `AttributeSet`, has four subinterfaces:

```
PrintRequestAttributeSet
DocAttributeSet
PrintServiceAttributeSet
PrintJobAttributeSet
```

Each of these interfaces has an implementing class, yielding the five classes:

```
HashAttributeSet
HashPrintRequestAttributeSet
HashDocAttributeSet
HashPrintServiceAttributeSet
HashPrintJobAttributeSet
```

Figure 12.66 shows a class diagram of the attribute set hierarchy.

Figure 12.66: The attribute set hierarchy

For example, you can construct a print request attribute set like this:

```
var attributes = new HashPrintRequestAttributeSet();
```

After constructing the set, you are freed from worrying about the `Hash` prefix.

Why have all these interfaces? They make it possible to check for correct attribute usage. For example, a `DocAttributeSet` accepts only objects that implement the `DocAttribute` interface. Any attempt to add another attribute results in a runtime error.

An attribute set is a specialized kind of map where the keys are of type `Class` and the values belong to a class that implements the `Attribute` interface. For example, if you insert an object

```
new Copies(10)
```

into an attribute set, then its key is the `Class` object `Copies.class`. That key is called the *category* of the attribute. The `Attribute` interface declares a method

```
Class getCategory()
```

that returns the category of an attribute. The `Copies` class defines the method to return the object `Copies.class`, but it isn't a requirement that the category be the same as the class of the attribute.

When an attribute is added to an attribute set, the category is extracted automatically. Just add the attribute value:

```
attributes.add(new Copies(10));
```

If you subsequently add another attribute with the same category, it overwrites the first one.

To retrieve an attribute, you need to use the category as the key, for example:

```
AttributeSet attributes = job.getAttributes();
var copies = (Copies) attributes.get(Copies.class);
```

Finally, attributes are organized by the values they can have. The `Copies` attribute can have any integer value. The `Copies` class extends the `IntegerSyntax` class that takes care of all integer-valued attributes. The `getValue` method returns the integer value of the attribute, for example:

```
int n = copies.getValue();
```

The classes

```
TextSyntax
DateTimeSyntax
URISyntax
```

encapsulate a string, a date and time value, or a URI.

Finally, many attributes can take a finite number of values. For example, the `PrintQuality` attribute has three settings: draft, normal, and high. They are represented by three constants:

```
PrintQuality.DRAFT
PrintQuality.NORMAL
PrintQuality.HIGH
```

Attribute classes with a finite number of values extend the `EnumSyntax` class, which provides a number of convenience methods to set up these enumerations in a typesafe manner. You need not worry about the mechanism when using such an attribute. Simply add the named values to attribute sets:

```
attributes.add(PrintQuality.HIGH);
```

Here is how you check the value of an attribute:

```
if (attributes.get(PrintQuality.class) == PrintQuality.HIGH)
    . . .
```

Table 12.5 lists the printing attributes. The second column lists the superclass of the attribute class (for example, `IntegerSyntax` for the `Copies` attribute) or the set of enumeration values for the attributes with a finite set of values. The last four columns indicate whether the attribute class implements the `DocAttribute` (D), `PrintJobAttribute` (J), `PrintRequestAttribute` (R), and `PrintServiceAttribute` (S) interfaces.

Table 12.5: Printing Attributes

Attribute	Superclass or Enumeration Constants	D	J	R	S
`Chromaticity`	`MONOCHROME`, `COLOR`	✓	✓	✓	
`ColorSupported`	`SUPPORTED`, `NOT_SUPPORTED`				✓
`Compression`	`COMPRESS`, `DEFLATE`, `GZIP`, `NONE`	✓			
`Copies`	`IntegerSyntax`		✓	✓	
`DateTimeAtCompleted`	`DateTimeSyntax`		✓		
`DateTimeAtCreation`	`DateTimeSyntax`		✓		
`DateTimeAtProcessing`	`DateTimeSyntax`		✓		
`Destination`	`URISyntax`		✓	✓	
`DocumentName`	`TextSyntax`	✓			
`Fidelity`	`FIDELITY_TRUE`, `FIDELITY_FALSE`		✓	✓	
`Finishings`	`NONE`, `STAPLE`, `EDGE_STITCH`, `BIND`, `SADDLE_STITCH`, `COVER`, …	✓	✓	✓	
`JobHoldUntil`	`DateTimeSyntax`		✓	✓	
`JobImpressions`	`IntegerSyntax`		✓	✓	
`JobImpressionsCompleted`	`IntegerSyntax`		✓		
`JobKOctets`	`IntegerSyntax`		✓	✓	
`JobKOctetsProcessed`	`IntegerSyntax`		✓		
`JobMediaSheets`	`IntegerSyntax`		✓	✓	
`JobMediaSheetsCompleted`	`IntegerSyntax`		✓		
`JobMessageFromOperator`	`TextSyntax`		✓		
`JobName`	`TextSyntax`		✓	✓	
`JobOriginatingUserName`	`TextSyntax`		✓		
`JobPriority`	`IntegerSyntax`		✓	✓	
`JobSheets`	`STANDARD`, `NONE`		✓	✓	

Attribute	Superclass or Enumeration Constants	D	J	R	S
`JobState`	`ABORTED`, `CANCELED`, `COMPLETED`, `PENDING`, `PENDING_HELD`, `PROCESSING`, `PROCESSING_STOPPED`		✓		
`JobStateReason`	`ABORTED_BY_SYSTEM`, `DOCUMENT_FORMAT_ERROR`, many others				
`JobStateReasons`	`HashSet`		✓		
`MediaName`	`ISO_A4_WHITE`, `ISO_A4_TRANSPARENT`, `NA_LETTER_WHITE`, `NA_LETTER_TRANSPARENT`	✓	✓	✓	
`MediaSize`	`ISO.A0` – `ISO.A10`, `ISO.B0` – `ISO.B10`, `ISO.C0` – `ISO.C10`, `NA.LETTER`, `NA.LEGAL`, various other paper and envelope sizes				
`MediaSizeName`	`ISO_A0` – `ISO_A10`, `ISO_B0` – `ISO_B10`, `ISO_C0` – `ISO_C10`, `NA_LETTER`, `NA_LEGAL`, various other paper and envelope size names	✓	✓	✓	
`MediaTray`	`TOP`, `MIDDLE`, `BOTTOM`, `SIDE`, `ENVELOPE`, `LARGE_CAPACITY`, `MAIN`, `MANUAL`	✓	✓	✓	
`MultipleDocumentHandling`	`SINGLE_DOCUMENT`, `SINGLE_DOCUMENT_NEW_SHEET`, `SEPARATE_DOCUMENTS_COLLATED_COPIES`, `SEPARATE_DOCUMENTS_UNCOLLATED_COPIES`		✓	✓	
`NumberOfDocuments`	`IntegerSyntax`		✓		
`NumberOfInterveningJobs`	`IntegerSyntax`		✓		
`NumberUp`	`IntegerSyntax`	✓	✓	✓	
`OrientationRequested`	`PORTRAIT`, `LANDSCAPE`, `REVERSE_PORTRAIT`, `REVERSE_LANDSCAPE`	✓	✓	✓	
`OutputDeviceAssigned`	`TextSyntax`		✓		
`PageRanges`	`SetOfInteger`	✓	✓	✓	
`PagesPerMinute`	`IntegerSyntax`				✓
`PagesPerMinuteColor`	`IntegerSyntax`				✓
`PDLOverrideSupported`	`ATTEMPTED`, `NOT_ATTEMPTED`				✓

Attribute	Superclass or Enumeration Constants	D	J	R	S
`PresentationDirection`	`TORIGHT_TOBOTTOM`, `TORIGHT_TOTOP`, `TOBOTTOM_TORIGHT`, `TOBOTTOM_TOLEFT`, `TOLEFT_TOBOTTOM`, `TOLEFT_TOTOP`, `TOTOP_TORIGHT`, `TOTOP_TOLEFT`		✓	✓	
`PrinterInfo`	`TextSyntax`				✓
`PrinterIsAcceptingJobs`	`ACCEPTING_JOBS`, `NOT_ACCEPTING_JOBS`				✓
`PrinterLocation`	`TextSyntax`				✓
`PrinterMakeAndModel`	`TextSyntax`				✓
`PrinterMessageFromOperator`	`TextSyntax`				✓
`PrinterMoreInfo`	`URISyntax`				✓
`PrinterMoreInfoManufacturer`	`URISyntax`				✓
`PrinterName`	`TextSyntax`				✓
`PrinterResolution`	`ResolutionSyntax`	✓	✓	✓	
`PrinterState`	`PROCESSING`, `IDLE`, `STOPPED`, `UNKNOWN`				✓
`PrinterStateReason`	`COVER_OPEN`, `FUSER_OVER_TEMP`, `MEDIA_JAM`, and many others				
`PrinterStateReasons`	`HashMap`				
`PrinterURI`	`URISyntax`				✓
`PrintQuality`	`DRAFT`, `NORMAL`, `HIGH`	✓	✓	✓	
`QueuedJobCount`	`IntegerSyntax`				✓
`ReferenceUriSchemesSupported`	`FILE`, `FTP`, `GOPHER`, `HTTP`, `HTTPS`, `NEWS`, `NNTP`, `WAIS`				
`RequestingUserName`	`TextSyntax`			✓	
`Severity`	`ERROR`, `REPORT`, `WARNING`				
`SheetCollate`	`COLLATED`, `UNCOLLATED`	✓	✓	✓	

Attribute	Superclass or Enumeration Constants	D	J	R	S
`Sides`	`ONE_SIDED`, `DUPLEX` (= `TWO_SIDED_LONG_EDGE`), `TUMBLE` (= `TWO_SIDED_SHORT_EDGE`)	✓	✓	✓	



Note: As you can see, there are lots of attributes, many of which are quite specialized. The source for most of the attributes is the Internet Printing Protocol 1.1 (RFC 2911).



Note: An earlier version of the printing API introduced the `JobAttributes` and `PageAttributes` classes, whose purpose was similar to the printing attributes covered in this section. These classes are now obsolete.

javax.print.attribute.Attribute **1.4**

- `Class getCategory()`
  gets the category of this attribute.
- `String getName()`
  gets the name of this attribute.

javax.print.attribute.AttributeSet **1.4**

- `boolean add(Attribute attr)`
  adds an attribute to this set. If the set has another attribute with the same category, that attribute is replaced by the given attribute. Returns `true` if the set changed as a result of this operation.
- `Attribute get(Class category)`
  retrieves the attribute with the given category key, or `null` if no such attribute exists.
- `boolean remove(Attribute attr)`
- `boolean remove(Class category)`
  remove the given attribute, or the attribute with the given category, from the set. Return `true` if the set changed as a result of this operation.
- `Attribute[] toArray()`
  returns an array with all attributes in this set.

javax.print.PrintService **1.4**

- `PrintServiceAttributeSet getAttributes()`
  gets the attributes of this print service.

javax.print.DocPrintJob **1.4**

- `PrintJobAttributeSet getAttributes()`
  gets the attributes of this print job.

You have now reached the end of this long chapter covering advanced Swing and AWT features. In the final chapter, we will turn to a different aspect of Java programming: interacting, on the same machine, with “native” code in a different programming language.

Previous chapter

< 11. User Interface Components with Swing

Next chapter

13. The Foreign Functions and Memory API >